

这才是网络安全·第零季：火种

—— 在敲下第一行命令之前，你应该知道的事

目录

- 上部：计算机史——火种从何而来
 - 第一章：从算盘到图灵机——计算的千年长征
 - 第二章：从炸弹到ENIAC——战争催生的巨兽
 - 第三章：晶体管的逆袭——让计算机从房间变成桌面
 - 第四章：互联网的雏形——冷战、ARPANET和“分布式”的野心
 - 第五章：个人计算机的黎明——当电脑不再是政府的玩具
 - 中部：黑客历史——从“电话飞客”到“数字侠客”
 - 第六章：黑客的起源——“hack”本来是件好事
 - 第七章：电话飞客（Phreaking）——最早的“黑客”
 - 第八章：70-80年代——真正的“黑客英雄”登场
 - 第九章：90年代——黑客进入公众视野
 - 第十章：中国黑客的“江湖往事”——从拨号上网到红客联盟
 - 下部：黑客文化与精神——火种如何传递
 - 第十一章：黑客伦理——你的技术信仰
 - 第十二章：黑客的“三大美德”——懒惰、急躁、傲慢
 - 第十三章：技术与道德的分界线
 - 第十四章：守夜者的精神——孤独但有光
 - 附录：守夜者的宣誓词（中文版 & 英文版）
-

上部：计算机史——火种从何而来

第一章：从算盘到图灵机——计算的千年长征

1.1 人类为什么要“算”？

在开始之前，请你做一件事：**把双手伸出来，数一数你有几根手指。**

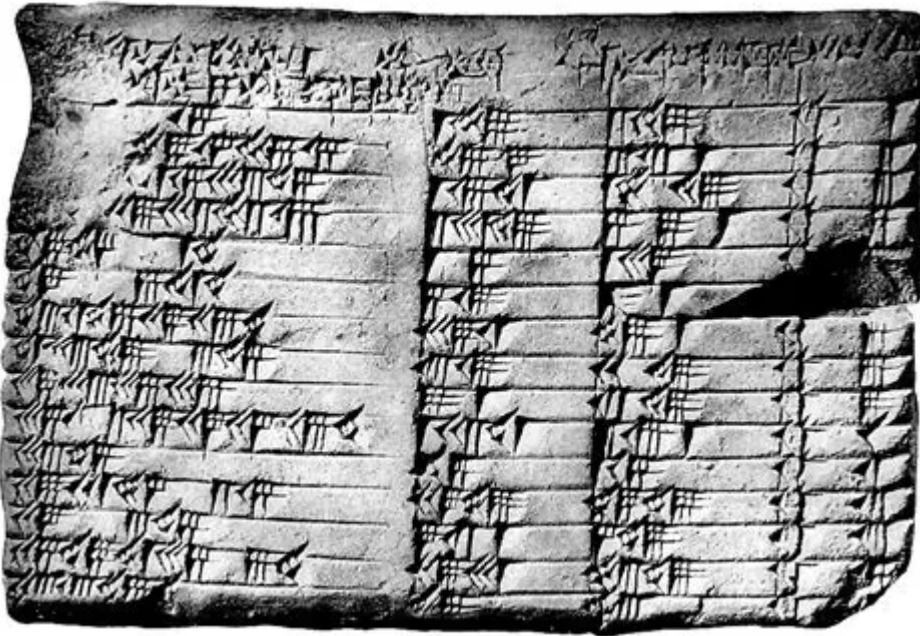
十根。恭喜你，你拥有了人类历史上第一台“计算设备”。这台设备不需要充电，不会死机，唯一的缺点是——你总不能一辈子靠掰手指头过日子。

想象一下这个场景：公元前3000年，两河流域。

一个苏美尔人的村长坐在泥屋门口，面前堆着一堆羊骨头，他需要算出今年全村收了多少大麦、该交多少税、仓库里还剩多少。他的手指头掰完了，脚趾头也掰完了，还是不够用。

怎么办？

苏美尔人的解决方案非常“朴素但有效”：**在地上挖几条平行的沟，往沟里放小石子。**一条沟代表“个位”，一条代表“十位”，一条代表“百位”。一颗石子在个位沟里就是1，挪到十位沟里就是10。



这就是算盘的前身，也是人类第一次**“把数字从脑子里搬到了外面”**。这个动作的意义怎么强调都不过分——它意味着数学不再是“只能想”的东西，变成了“可以看、可以摸、可以操作”的东西。

你可能会问：“不就是数数吗？有什么了不起？”

数数本身就是人类最伟大的发明之一。因为“数”这个东西，在大自然里是不存在的。你见过“3”吗？你见过“100”吗？你见过“-5”吗？没有。树上有三颗果子，但“3”本身是抽象的——它是人类从具体世界中提取出来的**概念**。

而算盘告诉我们另一件事：**抽象的概念，可以用具体的物体来操作。**

你会发现，这和“编程”的本质一模一样。

你写的每一行代码，都是一种“把脑中的逻辑变成机器可以操作的具体指令”的过程。苏美尔人挪石子，你敲键盘。原理没变，只是工具升级了。

从那以后，人类对“计算工具”的需求就像滚雪球一样越滚越大：

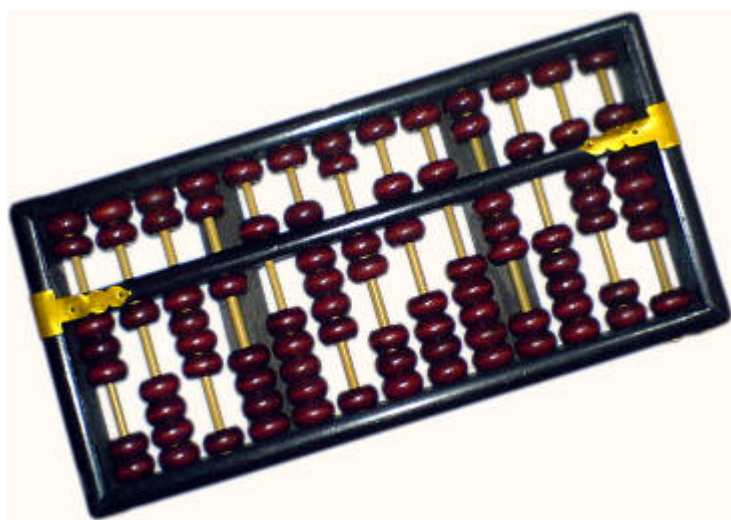
- 商人要算账
- 天文学家要算星星轨迹
- 建筑师要算材料
- 税务官要算……总之就是各种算

“不想自己算”这个念头，推动着人类一步一步走向计算机。

Kate 盯着这段历史看了半天，突然说：“所以人类几千年前就开始想办法‘偷懒’了？我一直以为懒得算是我自己的问题，原来是祖传的。”

1.2 算盘、齿轮、数字的舞会

我们把时间快进到公元5世纪左右。中国人发明了**算盘**——不再是地上挖沟放石子，而是一个精致的木框，串着珠子，上档一颗代表五，下档一颗代表一。



算盘有多厉害？熟练的算盘手，加减法的速度比现代人按计算器还快。一直到上世纪70年代，中国很多商店的柜台上还摆着算盘，算账全靠它。

但算盘有个天花板：**它只能帮你“算”，不能帮你“自动算”**。每一次计算都需要人动手拨珠子，把数字读进去、把结果读出来。人依然是整个过程的中心。

有没有一种机器，能让人彻底“退休”？

答案是：有，但要等很久。

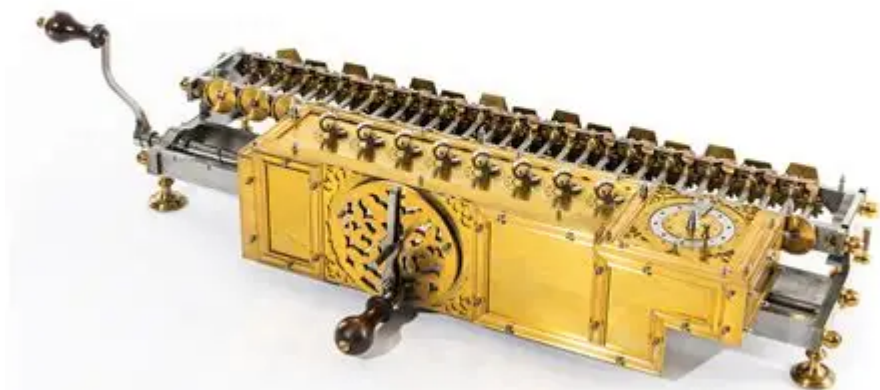
1623年，德国的一个学者叫**威廉·希卡德**，设计了一台“计算钟”——用齿轮来实现加减法。这是历史上第一台机械计算装置。



可惜，他还没造出来，就在战乱中去世了，图纸也失传了。一直到上世纪60年代，后人才从他的手稿里翻出设计图，发现——**这家伙领先了所有人整整二十年。**

然后就是我们熟悉的帕斯卡和莱布尼茨了。

1642年，法国的**布莱兹·帕斯卡**造出了加法机。他的故事特别有意思：帕斯卡他爹是个税务官，每天面对一堆数字算得头昏眼花。帕斯卡心疼老爹，就捣鼓出了一台用齿轮传动的机器，摇一摇手柄就能算加减法。

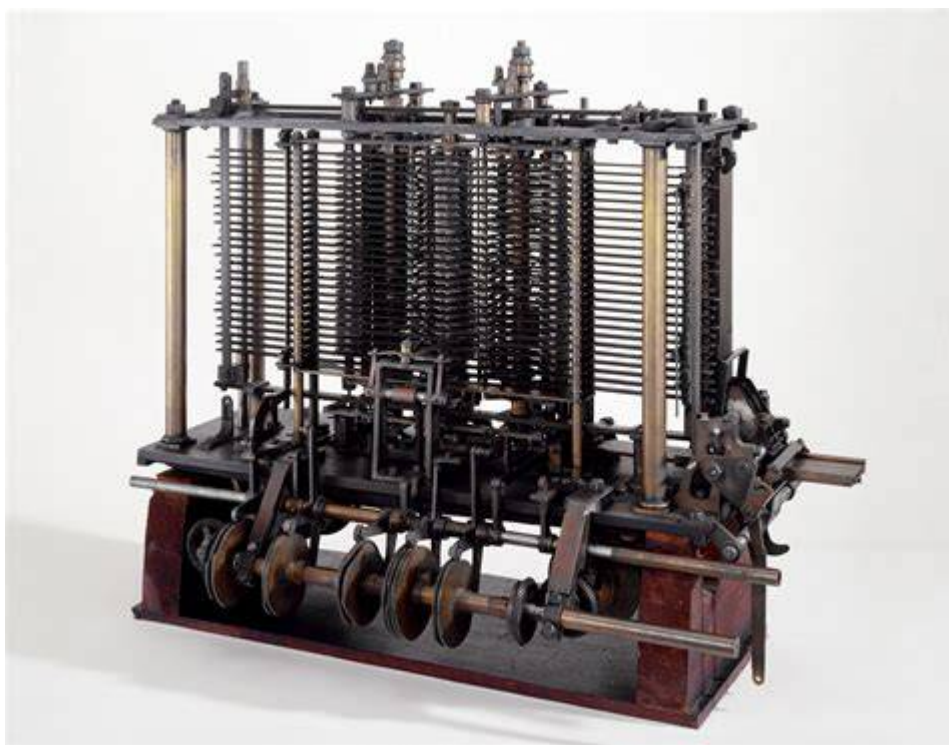


这台机器长得很像一个大号的音乐盒——里面是一排齿轮，每个齿轮上有0到9的数字。当个位齿轮转满一圈，就“咔哒”一声带动十位齿轮转一格——这个机制，你在今天机械手表里还能看到。

帕斯卡的机器一问世，整个欧洲数学圈都炸了。人们第一次看到：**机械可以代替人脑做一部分“脑力劳动”**。

但帕斯卡的机器只能算加减法。1671年，德国数学家**莱布尼茨**（就是微积分里的那个莱布尼茨）在帕斯卡的基础上搞出了能算乘除法的机器。他的秘诀是：**用“重复加法”代替乘法，用“重复减法”代替除法**。

比如 3×4 ，莱布尼茨的机器就帮你算 $3+3+3+3$ 。方法很笨，但机器不嫌累。



莱布尼茨当时说了一句非常狂的话：“让优秀的人把时间浪费在计算上是不值得的，这种工作交给机器就好了。”

这句话放在今天，就是“能自动化的绝不动手”的17世纪版。

不过，这些机械计算机都有同一个问题：**它们是“专用”的，不是“通用”的。**帕斯卡的机器只会加减，莱布尼茨的只会加减乘除，就像今天的计算器只会算数——你没法让它播放视频、也没法让它打游戏。

而人类想要的是：**一台什么都能干的机器。**

这就需要一个人，一套全新的设计，和一个——超前了整整一百年的梦。

Kate 在纸上画了一个小齿轮，旁边写着“咔哒”。她说：“所以齿轮转一圈，就等于掰了一根手指头。机器就是不会喊累的手指头。”

1.3 巴贝奇与分析机：超前一百年梦

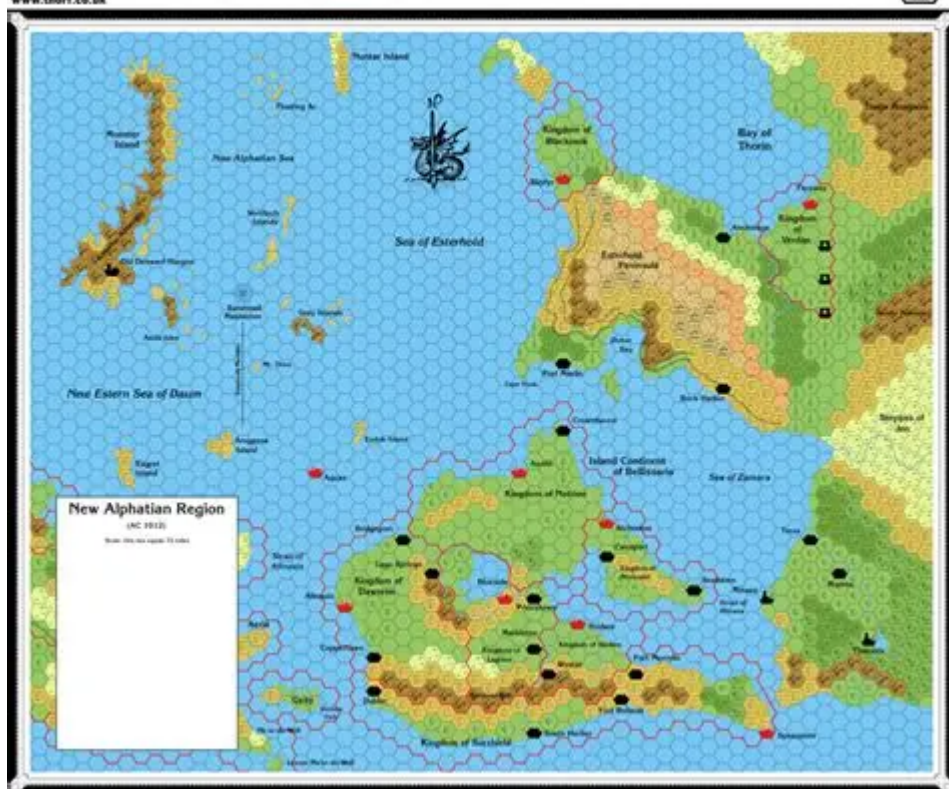
查尔斯·巴贝奇出生于1791年，伦敦一个富裕的银行家家庭。

因为家里有钱，他接受了当时最好的教育——剑桥大学。但巴贝奇在剑桥过得很不爽：他发现，学校教的数学比欧洲大陆落后了一大截，老师讲的都是几十年前的老东西。

他开始自学，最后成果比学校教的东西强得多。24岁就评上了皇家学会院士——相当于今天一个博士生毕业三年就当上了中科院院士。

巴贝奇很快找到了人生的“执念”：**计算表。**

你可能不知道计算表是什么——在计算机出现之前，天文学家、航海家、工程师、数学家做计算时，全靠查“计算表”。就是一本厚厚的表格书，上面印好了各种函数的数值（比如sin、cos、对数表）。要用哪个数，就直接翻到那一页去找。



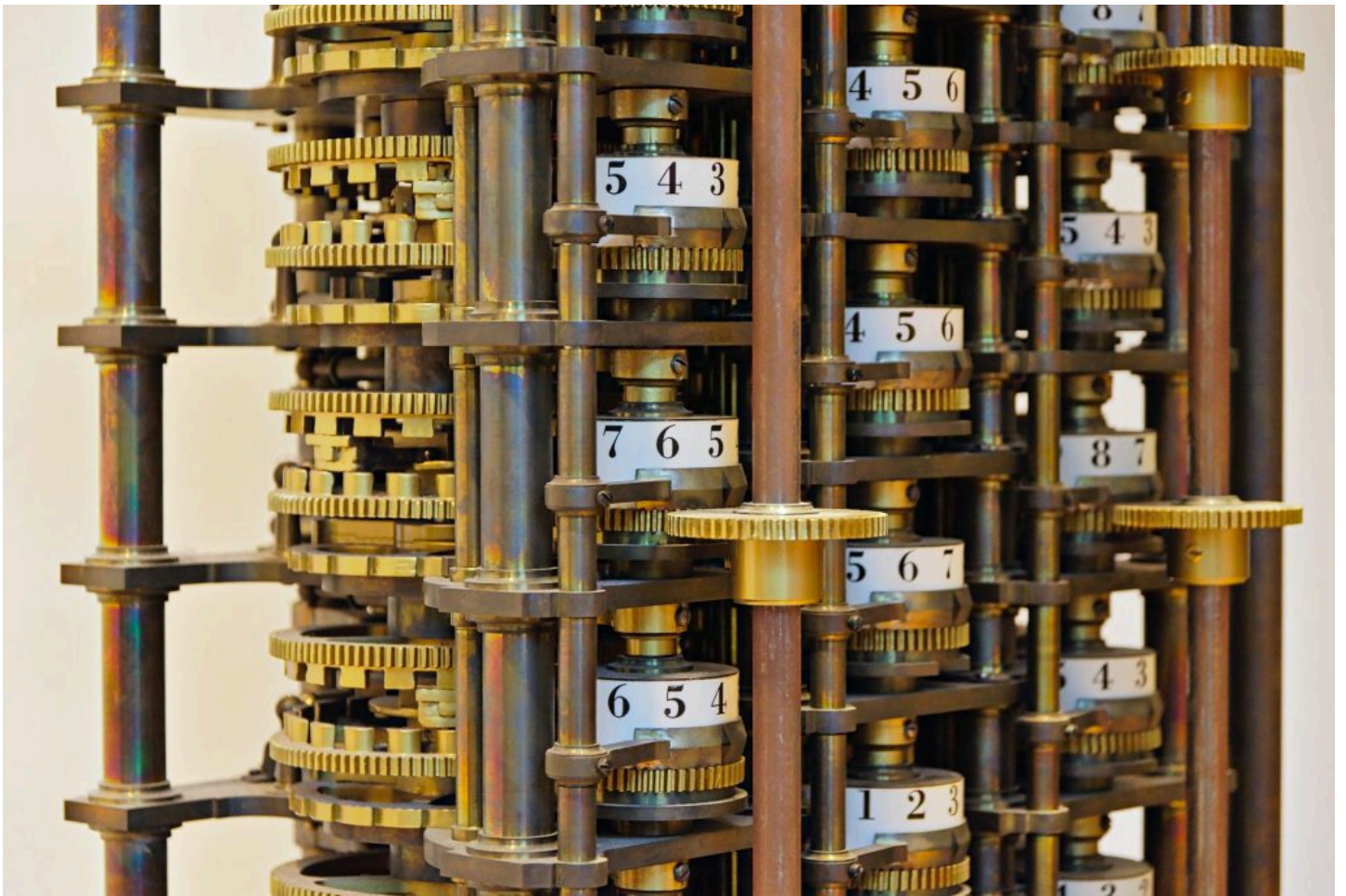
但问题是，这些表格是人算出来的。**是人就会犯错。**一个数算错了，印在书上，整个航海舰队可能偏航几百公里。

1819年，巴贝奇盯着桌上的一本天文表，发现里面至少有几处错误。他把这些错误圈出来，拍在桌子上，大吼一声：

“我向上帝发誓，这些表格必须用机器来算！”

这句话，改变了世界。

1822年，巴贝奇设计出了**“差分机”**。差分机专门用来算多项式——而天文、导航中的很多复杂函数，都可以转化成多项式来近似计算。



英国政府一看，两眼放光：这玩意儿要是造出来，皇家海军的导航精度将提升一个档次！拨款！他们要巴贝奇造一台“差分机二号”——一个能算更多位数、更多变量的超级版本。

巴贝奇拿到钱，开始动手。

然后……这个项目就变成了一个持续20年的噩梦。

巴贝奇是一个“完美主义者中的完美主义者”。他对零件精度的要求，超出了当时英国工业的制造水平。齿轮的齿距必须精确到千分之一英寸，任何偏差都不行。他不停地改设计图纸，不停地要求零件返工。工人恨他，供应商恨他，连政府审计员都恨他。

20年过去了，拨款用光了，机器还是没造出来。

1842年，英国政府正式停止了对巴贝奇项目的资助。那台半成品的差分机，摆在巴贝奇的车间里，落满了灰。

巴贝奇失败了。

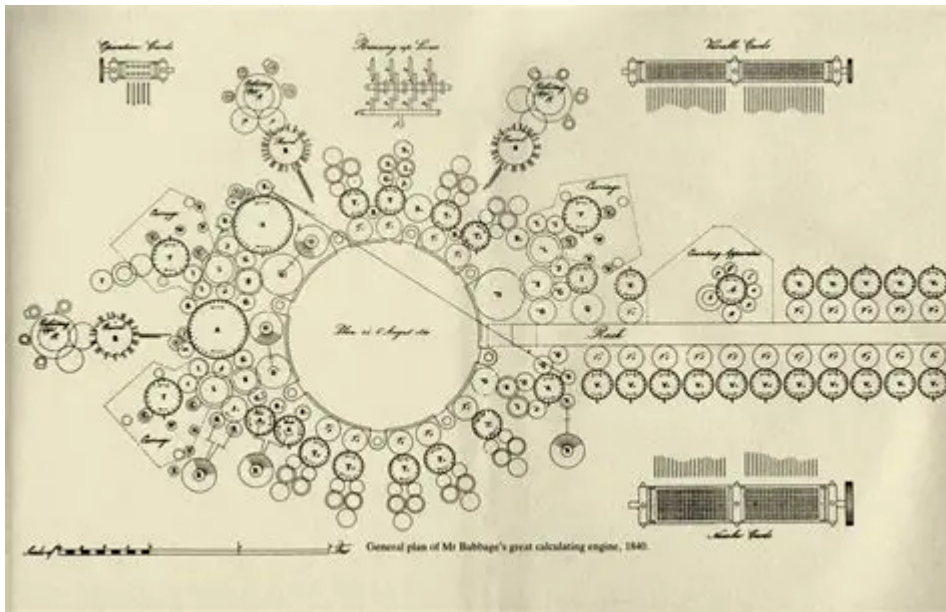
他没有认输。

50岁那年，他开始设计一个比差分机更疯狂的东西——**分析机**。

分析机的设计理念，现在看来简直不可思议：

- **存储单元**：分析机有“存储库”，能存1000个50位十进制数。相当于内存。

- **运算单元**：分析机有“运算室”，能做加减乘除和比较。相当于CPU。
- **控制单元**：分析机通过打孔纸带来“编程”和“读程序”。相当于操作系统+应用软件。
- **输入输出**：分析机有读卡器和打印机。



分析机的总设计师曾说过：“这台机器会自动执行一系列操作，除非你中途手动改变程序，否则它会一直按照打孔纸带的指令运行。”

——这就是现代计算机的完整定义。

巴贝奇在1834年就已经想明白了“内存-CPU-程序-数据分离”的架构。而第一台真正意义上的电子计算机（ENIAC）要到1945年才被造出来。

巴贝奇领先了世界整整**111年**。

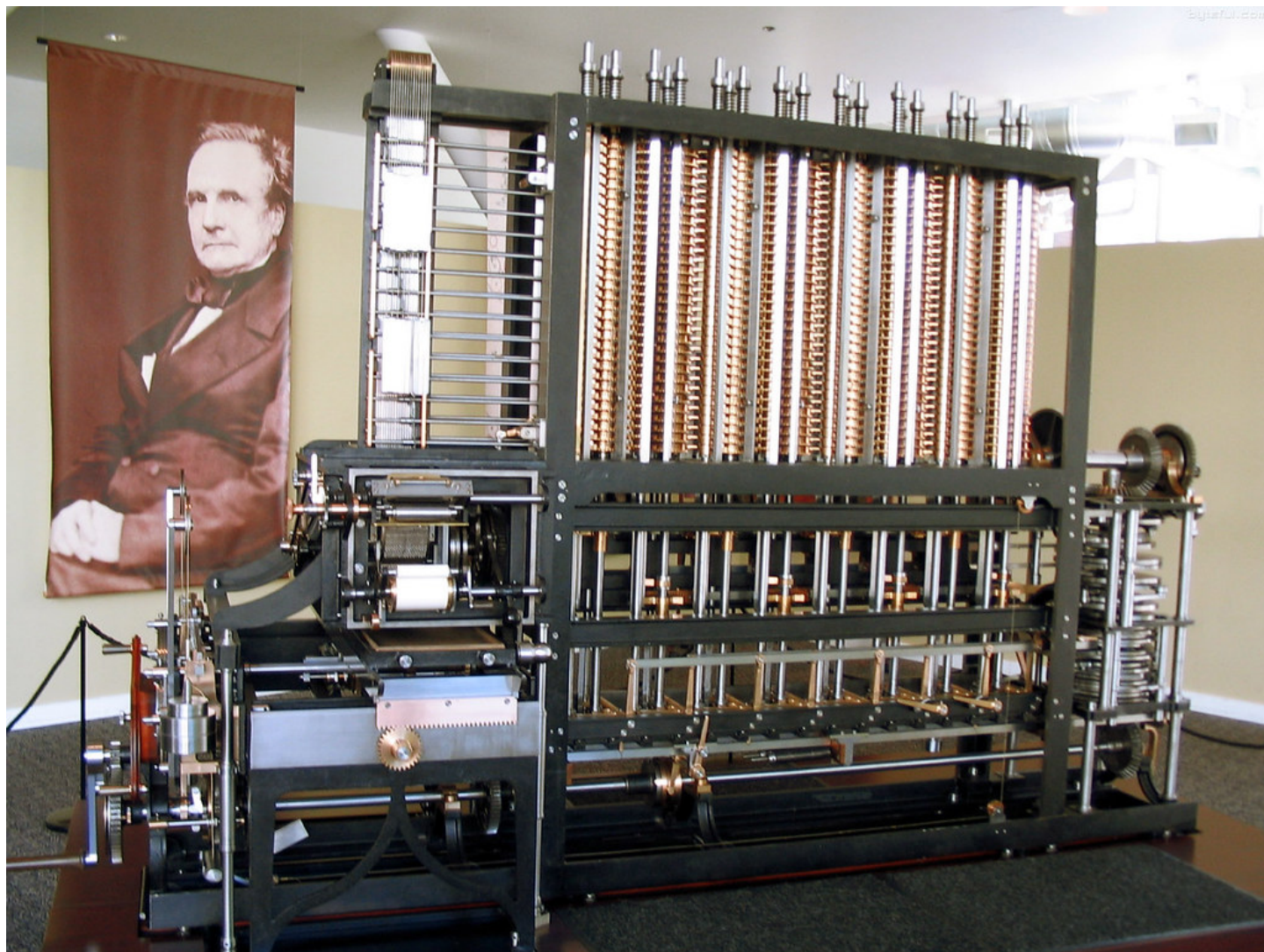
但领先也是有代价的：他那个时代的生产工艺，根本造不出分析机所需的精密零件。这台人类第一台“通用计算机”的蓝图，最终停在了一堆图纸上。

1871年，查尔斯·巴贝奇去世。《泰晤士报》在讣告中这样写道：

“他……浪费了政府巨额的资金，却未能交付任何可用的机器。”

这是一个“失败者”的墓志铭。

一百多年后，伦敦科学博物馆按照巴贝奇的图纸，复原了差分机和分析机——**它们真的能运行。**



巴贝奇不是一个失败者。他是一个被时代困住的天才。

现在我们可以用一句话总结他的贡献：**查尔斯·巴贝奇，用齿轮和蒸汽机，画出了第一张“计算机”的完整设计图。**

——而你坐在电脑前敲的每一行代码，跑的操作系统、编译器、解释器，都建立在“内存-CPU-程序”这个核心架构之上。这个架构，就是巴贝奇和艾达·洛夫莱斯，在没人相信“计算机”这个东西的时候，就已经想明白了。

Kate 盯着分析机的图纸看了很久，说：“这个人用齿轮想出了电脑的结构，然后他那个时代的工人造不出来。他等了20年，没等到。又等了111年，后人才帮他造出来。”她停了一下。“……我以后再也不抱怨编译报错了。”

1.4 艾达·洛夫莱斯：第一位程序员

在分析机的故事里，还有一个重量级角色——**艾达·洛夫莱斯**。

你可能会在历史书上看到她被称为“第一位程序员”。但你想过没有：**在没有计算机的时代，怎么编程？**

答案：写在纸上。

艾达写完的程序，在没有机器的情况下完成，后来机器造出来了，跑通了。

艾达是英国大诗人拜伦勋爵的独生女。但她的母亲（拜伦夫人）痛恨拜伦那种“不负责任的浪漫”，所以从小给艾达安排的课程全是——数学、逻辑、科学。在那个女孩普遍学“礼仪、音乐、绘画”的19世纪初，艾达的教育放在今天依然算“硬核”。

18岁的时候，艾达在伦敦的一次学术沙龙上见到了巴贝奇。当时巴贝奇正在展示差分机的模型。所有人都好奇地围观，但只有艾达一个人走到机器旁边，指着齿轮问出了那个最关键的问题：

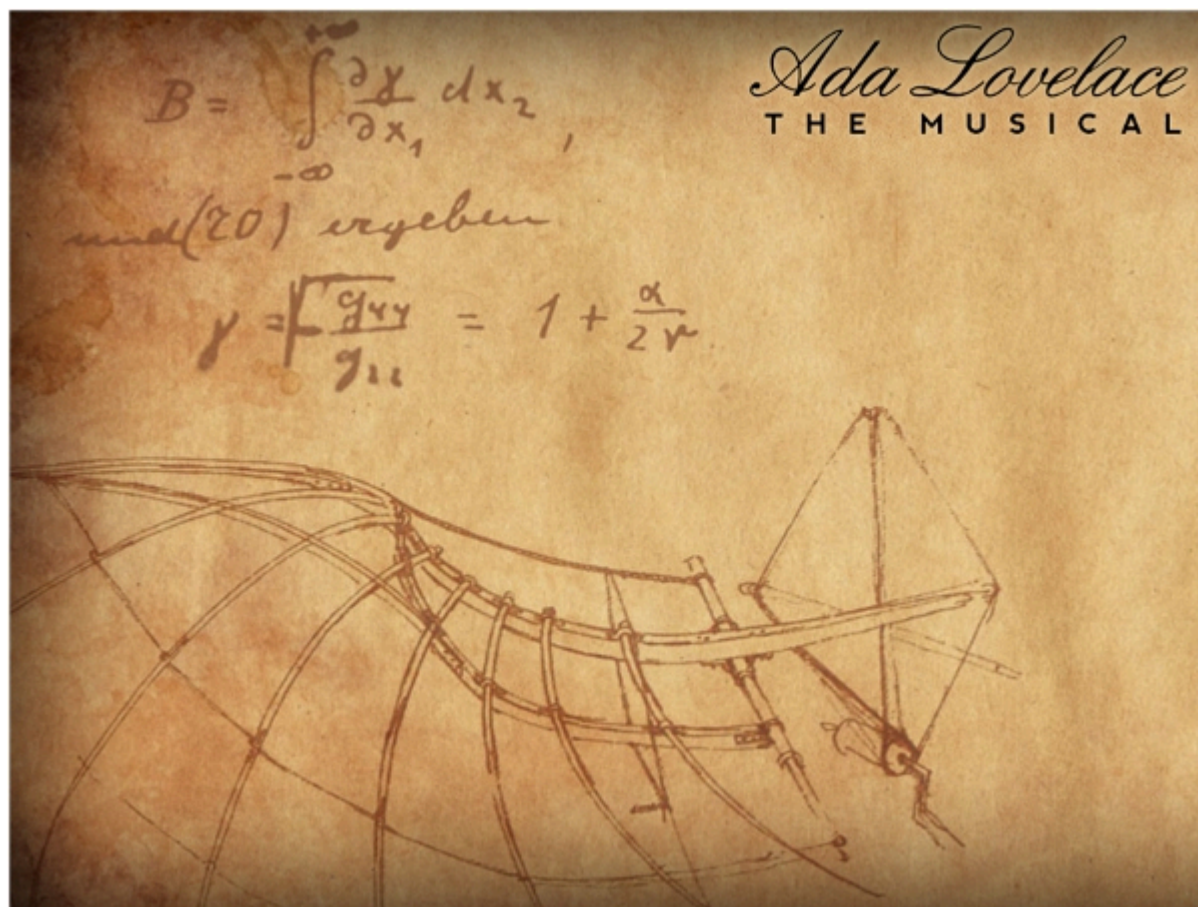
“它的原理是什么？”

巴贝奇愣了一下，然后开始给她讲解。他惊讶地发现：**一个18岁的女孩，比在场所有的绅士都更理解这台机器的设计逻辑。**

从那以后，巴贝奇和艾达成了终生的朋友和合作伙伴。

巴贝奇把分析机的设计稿拿给艾达看。艾达不仅看懂了，还主动承担了一件事：**为这台还没造出来的机器，写一组可以运行的指令。**

她的任务是计算伯努利数列——那是一种在概率论和数论中常见的数列。



艾达是这样写的：

1. 从存储库里取出第N个数值

2. 把数值送入运算室
3. 按照特定的公式进行加减乘除
4. 把结果存回存储库
5. 重复操作，直到算完所有数值

她甚至用了一个“循环”——在分析机上实现了一个重复执行的操作序列。这个思路，和你写的 `for` 循环或者 `while` 循环在概念上毫无区别。

你读了是不是很熟悉？对，就是编程。

这份写在纸上的指令，就是人类历史上的第一段程序。

更厉害的是，艾达在另一篇笔记里还提出：分析机不仅只能处理数字，它还可以处理**任何能用符号表示的事物**——比如音乐旋律、比如图形、比如语言。她说：“这台机器的算法，可以编织出任何复杂的图案。”

这其实就是今天“多媒体计算机”的雏形。艾达在1843年就已经想象到了“电脑能做音乐、能做设计、能处理语言”——那时候连电灯都没发明。

艾达·洛夫莱斯的结局颇为遗憾。她年仅36岁就因癌症去世，走的时候，分析机仍然没有造出来。她穷尽一生写的“程序”始终没有在一台真正的机器上跑过。

她的家族墓志铭上写着她的名字——没有提及“程序员”，没有提及“计算机”。

但历史最终没有忘记她。1953年，图灵的导师——计算机科学的另一位先驱——把艾达的笔记翻出来重印，明确写道：“**这篇论文……预见了现代计算机几乎所有的核心概念。**”

艾达·洛夫莱斯不是“一位女程序员”，她是整个人类历史上第一位程序员。

而她和巴贝奇的故事告诉我们一个最底层的事实：“**写代码这件事，本质上是把“人类脑子里的逻辑”翻译成“机器能听懂的语言”——这个行为本身，和机器有没有造出来、用什么材料造，没有必然关系。**

你现在写代码、写脚本、写任何让机器自动运行的指令——你做的事，和1843年艾达在纸上写的，是同一件事。

Kate 在自己的笔记里画了一行字：“代码可以跑在纸上。程序是一种思想，不是一种材料。”

1.5 1936年：图灵与他的“纸带机”

从艾达到图灵，中间隔着将近一百年。

这一百年发生了什么？

——电被人类驯服了。

电报、电话、无线电——电带来的信息革命，让科学家们开始琢磨一个问题：**能不能用电来做计算？**用齿轮太慢太笨了，电的速度是光速，如果用电来做计算，那速度会快多少个数量级？

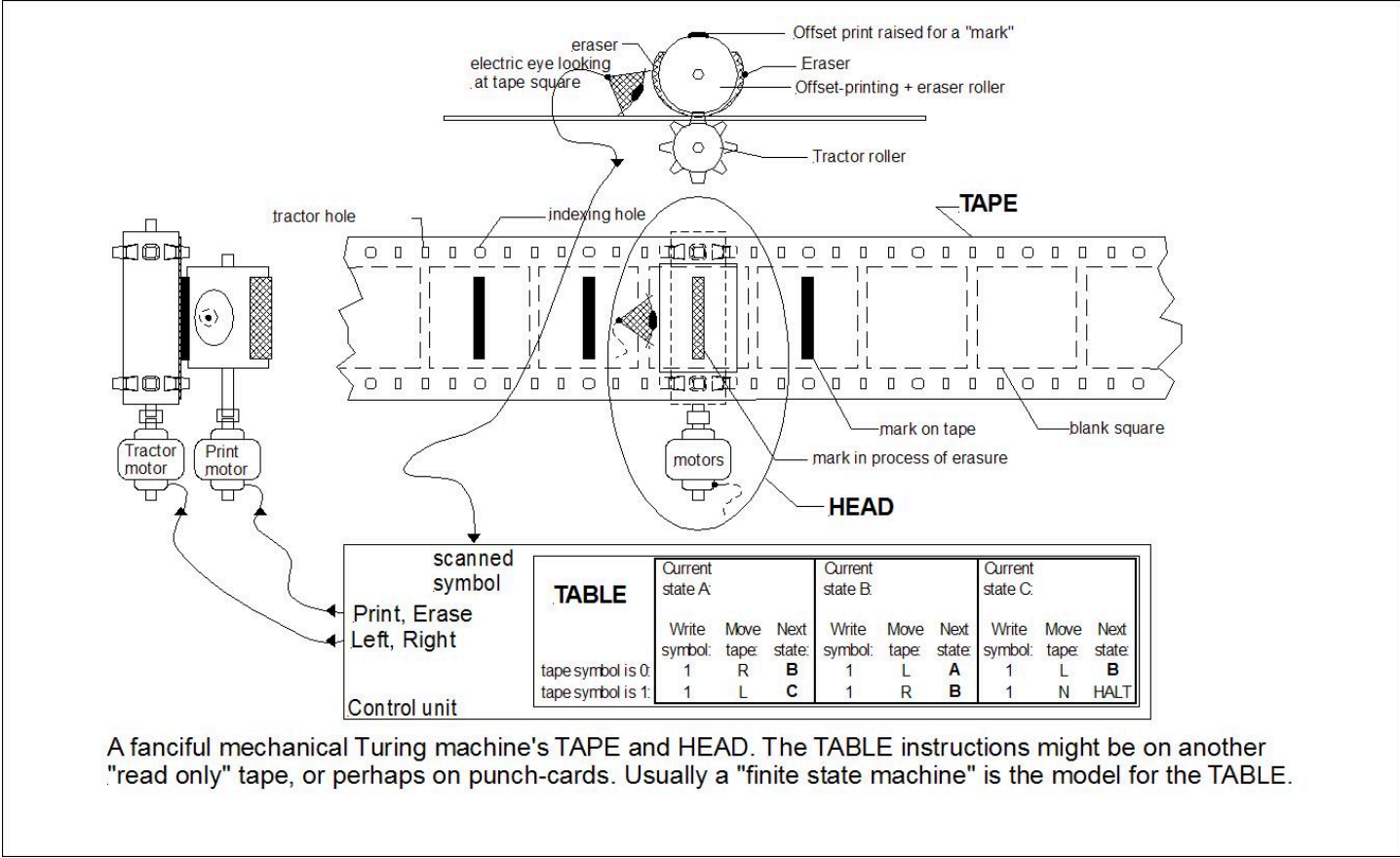
但问题来了：**怎么用电来“算”？**

电动机只会转圈，电灯只会发光，电报只会滴滴答答——你总不能指望电流在导线上自己背乘法表吧？要想让电来执行计算，必须有一套严谨的**逻辑体系**：什么情况接通电路、什么情况断开电路、什么情况下算“是”、什么情况下算“否”。

1936年，24岁的英国人**艾伦·图灵**在一篇论文里解答了这个问题。他提出了一个思想实验——“图灵机”。

这不是一台真的机器。这是图灵在脑子里设想的一个“计算装置”。它的构造极其简单：

- 一条**无限长的纸带**，分成一格一格。每格可以什么都不写，也可以写一个符号（比如0或者1）。
- 一个**读写头**，可以在纸带上左右移动。它能读当前格子里的符号，也能往格子里写一个新的符号。
- 一套**规则表**，告诉读写头“当你看到某个符号、且处在某种状态时，你应该怎么做”——是写一个新符号、还是向左移动、还是向右移动、还是停下来。



就这样？就这三样东西？

对。就这三样东西。

图灵用一个数学证明说：**任何你能想到的计算问题，只要能写出明确的步骤（算法），这台“纸带机”都能帮你算出来。**

不管是加减乘除、解方程、模拟天气、加密文件、还是打游戏——本质上都可以被拆解成一堆“读格子-写格子-移动-判断”的重复操作。图灵机的神奇之处在于，**它不需要任何“智能”，只需要机械地执行规则**，就能实现所有可计算的逻辑。

这就是**艾伦-图灵论题**的核心：计算机的本质不是硅、不是电子、不是齿轮——是**规则**。

只要有足够长的纸带、足够多的时间、一套完整的规则表，你就可以计算任何问题。

一个让你细思极恐的细节：图灵证明了“存在一个通用图灵机”——一种特殊的图灵机，可以模拟任何其他图灵机的工作。

换句话说：**你不需要为每个任务造一台不同的机器。你只需要一台机器，给它装不同的“程序”（规则表），它就能做任何事。**

这，就是“通用计算机”的数学模型。

你手里的电脑，本质上就是一台“电子版的通用图灵机”。

而图灵机的核心思想——**机器只是机械地执行规则**——也埋下了一颗种子。当你以后学到缓冲区溢出、SQL注入、权限提升这些攻击手法的时候，回想一下：所有网络安全问题，本质上都是“规则被绕过或违背了”。攻击者没有“破解”计算机，他只是找到了规则表上的一个漏洞，然后让读写头跳到了它不该去的位置。

Kate 听完图灵机的原理，说：“所以计算机不是‘聪明’，只是‘听话’。它不知道自己在做什么，它只是在执行规则。”她停了一下。“那黑客做的事，就是篡改规则？”

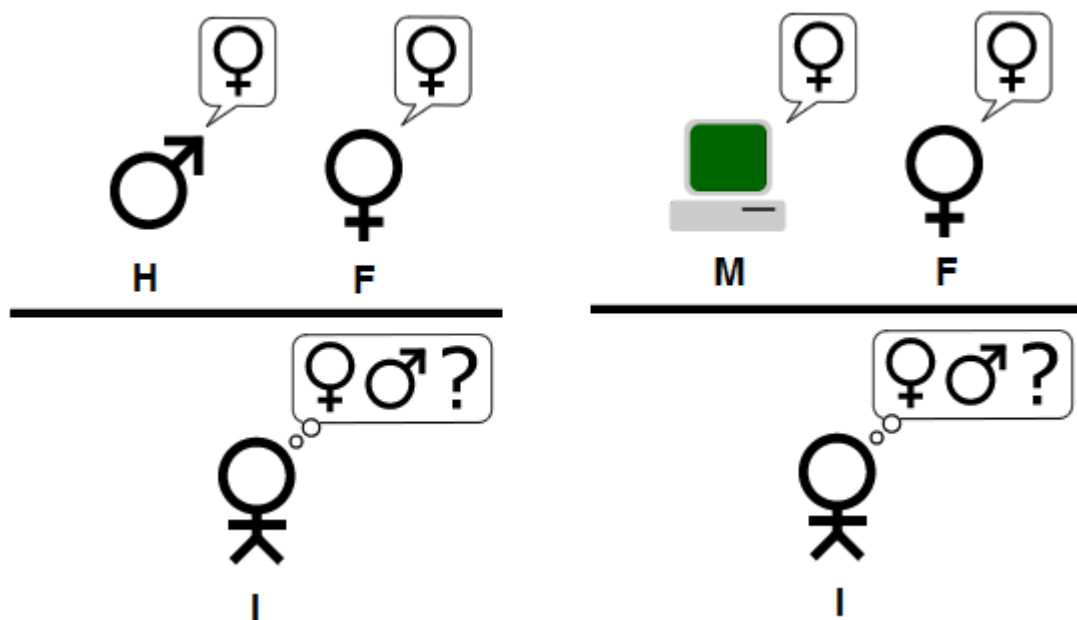
1.6 图灵测试：机器能思考吗？

图灵并没有止步于“机器能算”。他还想知道另一个更哲学的问题：**机器能思考吗？**

1950年，他在论文《计算机与智能》中提出了一个实验，后来被称为“**图灵测试**”：

把一个人和一台机器放在两个不同的房间里。测试者通过打字机（注意，不是直接说话，只能打字交流）分别和这两个对象对话，问任何问题。

如果测试者在对话结束后，无法区分哪个是真人、哪个是机器——那么这台机器就可以被认为是“具有智能”的。



图灵当时预测：到2000年，会有一台机器能在5分钟的对话中，骗过至少30%的测试者。

后来的历史证明他这个预测相当准确。2014年，一个叫“尤金·古斯特曼”的聊天机器人，在英国皇家学会组织的图灵测试中，让33%的评委以为它是一个13岁的乌克兰男孩。

当然，它骗过人的手段也很“鸡贼”——它装成一个非英语母语的13岁小孩，语法错误、逻辑跳跃都可以甩锅给“英文不好”和“年纪小”。

不过，图灵测试到今天还备受争议。很多人说它只是“模仿游戏”，不是真正的“思考”。但图灵要的根本不是“思考”——他要的是**可用标准**。

他告诉我们：如果你非要问“机器有没有灵魂”，那这个问题没有标准答案，永远扯不清。但如果你把问题换成“机器能不能骗过一个人”，那就有了明确的可检验标准——我们可以测试，可以重复，可以验证。

这种思维方式，叫做**可操作性定义**。它也是整个科学实验的基础——一个命题没有可观测的验证方法，就没有科学意义。

图灵的这一刀，把“人工智能”从哲学争论里切出来，变成了一门**工程学科**。

你现在可以自己去找“Turing test online”，找到网站扮演“测试者”——和机器聊几句，你会发现：那些没有通过测试的AI往往聊两句就露馅了（比如胡言乱语或者回答前后矛盾），而当你遇到一个能骗过你的AI时，你真的会后背发凉。

Kate 去网上找了一个图灵测试网站，聊了五分钟。回来之后她说：“那个AI一直在说‘我是个13岁的乌克兰男孩’。聊到后来，我都不知道该信它还是不该信它——相信呢，它可能是假的。不相信呢，万一人间真有个乌克兰小孩被冤枉了……”

1.7 图灵的结局

图灵的故事讲到这里，我不得不提那场悲剧。

1952年，图灵的住所被盗。警方调查时发现他是同性恋——这在当时的英国是刑事犯罪。他面临一个选择：要么坐牢，要么接受化学阉割（注射雌性激素，持续一年）。

图灵选择了后者。

他的安全许可被撤销，无法继续为国家工作。之前接触的最高机密被一刀切断。两年后，1954年6月7日，图灵在家中咬了一口注射了氰化物的苹果，结束了自己的生命。

他只有41岁。

2009年，英国时任首相戈登·布朗代表政府发表正式道歉。2013年，英国女王伊丽莎白二世颁发了对图灵的“皇家赦免”。

“他是世界级的数学家、密码破译者、计算机科学的奠基人。我们对不起他。”
——英国首相，2009年

历史欠图灵一个公正，但他留给世界的遗产，谁也抢不走。

他发明的图灵机模型，成了整个计算机科学的基石。没有他，就没有通用计算机，也就没有你正在用的VS Code、没有GitHub、没有网络协议——没有你正在读的这些书。

Kate 沉默了很长时间。然后她说：“.....所以我写的每一行代码，都应该感谢这个人。”

1.8 本章小结

这一章，你认识了三位奠基人：

人物	年代	核心贡献	一句话总结
巴贝奇	1834年	设计分析机：齿轮+蒸汽机的“通用计算机”蓝图	超前了111年的机械计算机之父
艾达·洛夫莱斯	1843年	写了第一段程序（循环+数学计算）	人类第一位程序员
图灵	1936年	提出图灵机模型（计算的数学模型）	计算机科学之父
图灵	1950年	提出图灵测试（判断机器能否思考的标准）	人工智能的奠基人

他们共同证明了三件极其重要的事：

1. **“程序”是一种可以脱离机器存在的纯逻辑。** 艾达的代码写在纸上，不需要电脑就能被验证。你今天学编程的时候，也先画流程图、写伪代码——一样的事。
2. **“通用计算机”不是靠技术进步才“出现”的，是靠逻辑设计才“想出来”的。** 巴贝奇用齿轮想通了这件事，冯·诺依曼用电子管把它造出来了。你用的任何一台计算机，底层架构依然可以“回溯”到巴贝奇和艾达的思想。
3. **“计算”的本质是“规则执行”，不是“智慧”。** 图灵证明了机器不需要“懂”自己在干什么，只要严格按照规则执行，就能完成复杂任务。这也意味着——**所有网络安全问题，本质上是“规则被绕过或违背了”。** 当你以后挖漏洞的时候，想想图灵机：如果攻击者能篡改“规则表”，或者让读写头跳到它不该去的位置，那整台机器就被人控制了。

这三位，是你成为网络安全工程师之前，必须先认识的人。 你以后学了TCP/IP、学了SQL注入、学了内网渗透——那些冷冰冰的命令和协议，都需要“历史”和“人文”来赋予它们温度。

下回当你觉得“学这些基础好无聊”的时候，想一想巴贝奇：**他连一台能跑代码的机器都没有，却用齿轮想出了现代计算机的完整架构。**

你只是学点Linux命令而已，比巴贝奇幸福多了。

下一章预告： 战争是最好的催化剂——二战如何催生了真正的电子计算机（ENIAC），并教会了人类“0”和“1”的力量。

守夜者手记

读完这一章，在继续之前，花几分钟回答这三个问题（写在你的Markdown笔记里）：

1. 艾达·洛夫莱斯的程序跑在什么“机器”上？她写的指令和今天你写的代码有什么本质相同之处？
2. 图灵机为什么是“思想实验”而不是实物？它的三要素是什么？
3. 巴贝奇的设计被《泰晤士报》评价为“浪费”——但历史给了他公正的评价。你怎么看待“失败”和“超前”之间的边界？

写完之后，用 `git add . && git commit -m "完成第一章读后感"` ——这不是为了交作业，这是为了**让你在未来回头的时候，能看到现在的自己在想什么。** 这是你身为守夜者记录的第一笔“自我观察”。

第二章：从炸弹到ENIAC——战争催生的巨兽

2.1 当计算遇到战争

在第一章里，我们认识了三位“纸上谈兵”的天才：巴贝奇在图纸上画出了计算机，艾达在纸上写出了程序，图灵在论文里证明了通用计算的可能性。

但你可能也注意到一个问题：**他们都没有真正“造”出一台能用的电子计算机。**

为什么？

答案是两个字：**材料。**

巴贝奇的时代只有齿轮和蒸汽机，根本造不出他设计的精密机械。图灵的时代有了电路和继电器，但距离“能跑程序”的大规模电子系统还差得很远。一台通用计算机需要成千上万个开关元件——用继电器的话，整台机器会像一栋楼那么大，而且每秒钟只能“啪嗒”几次，慢得要命。

那真正的电子计算机，到底是什么时候出现的？

——**战争，而且是人类历史上规模最大的战争。**

你可能听过一句话：“战争是科技进步的催化剂。”这句话很残酷，但很真实。二战（1939-1945）对计算机诞生的推动作用，怎么强调都不过分：

- **军方需要快速计算弹道轨迹：**一颗炮弹打出去，要算风速、角度、温度、湿度、地球自转……一个炮兵计算员用纸笔算一条弹道要花好几天，而前线每天要打几千发炮弹。
- **军方需要破解敌方密码：**德国的恩尼格玛密码机每天换一次密钥，用人工破译的话，等算出来，情报早就过期了。
- **军方需要模拟原子弹爆炸：**曼哈顿计划里那些核物理方程，靠笔算得算到天荒地老。

于是，政府开始砸钱——**天文数字的钱，投入到了“造一台会算数的机器”上。**

以前那些“花政府钱造机器”的项目（比如巴贝奇）被嘲笑为浪费。但在战争时期，这件事突然变成了“国家安全”级别的优先级。

电子计算机，就是在这样的背景下，从实验室走向了战场。

2.2 布莱切利公园：图灵的“秘密工厂”

在说美国的ENIAC之前，我们必须先讲英国的故事——因为那里有一个人，用一台“半电子、半机械”的机器，把二战缩短了至少两年。

这个人是艾伦·图灵。这个地方叫**布莱切利公园**。

布莱切利公园在伦敦西北约80公里，表面上是一个普通的庄园，实际上——**它是英国战时密码破译的总部。** 几千人在那里日夜工作，唯一的目标：破解德国的恩尼格玛密码。



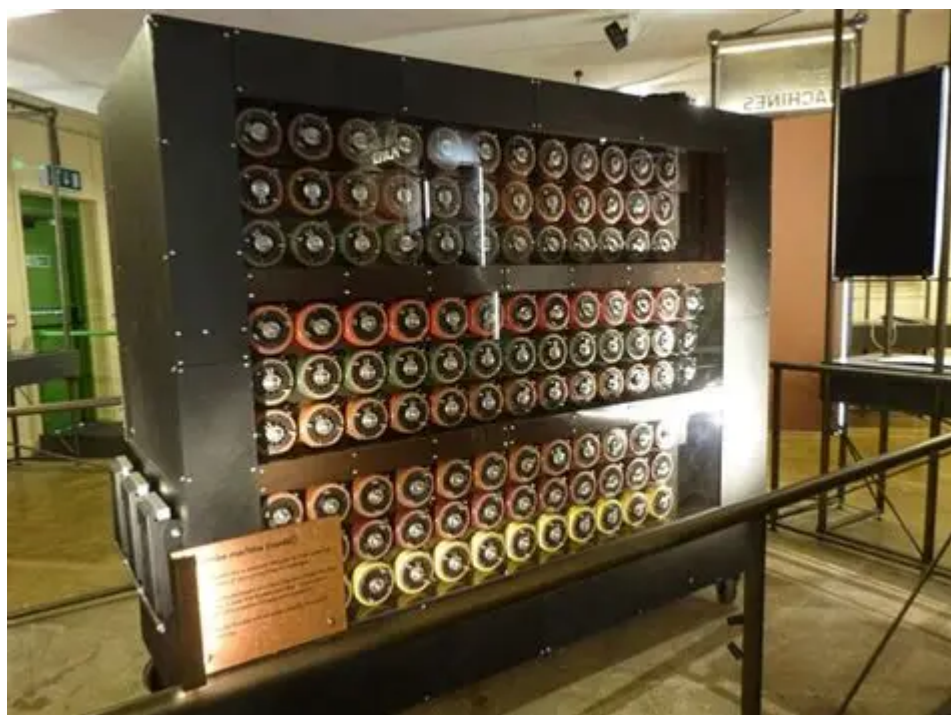
恩尼格玛密码机是德国人引以为傲的加密设备，长得像一台打字机，里面有三到四个转子。每按下一个字母，电流通过转子，加密成另一个字母输出。

关键点是：**每次按键，转子都会转动一格**。这意味着同一个明文字母，在不同的位置会被加密成完全不同的密文字母。德军每天换一次密钥设置，理论上用人工暴力破解所有组合需要**几千年**。



而图灵和他的同事们，设计了一台叫“**炸弹**”（Bombe）的机器来破解它。

“炸弹”不是一台通用计算机。它是一台**专用破解机**——专门用来寻找恩尼格玛的密钥设置。它模拟了多台恩尼格玛机同时运行，通过逻辑排除法，把可能的密钥组合从天文数字缩小到人力可以验证的范围。



“炸弹”是电动的，但不是完全电子化的——它里面有很多继电器和转盘，运行起来咋嗒作响，像一台巨型自动收银机。

但它**有效**。到战争结束时，布莱切利公园已经部署了超过200台“炸弹”机器，每天破解数千条德军通讯。

历史学家估计，布莱切利公园的工作将二战至少缩短了**两年**，挽救了**千万级**的生命。

而图灵在这其中扮演的角色，不只是一名工程师，更是一位“系统设计师”——他设计了一套完整的“输入-处理-输出”流程，把密码破译这件事变成了一条可重复、可扩展的“工业流水线”。

如果你仔细看，图灵做的事和今天的渗透测试流程几乎一模一样：

他的“信息收集”是截获德军无线电，“分析”是找出恩尼格玛的密钥生成规律，“利用”是用“炸弹”机暴力破解当天密钥，“提权”是破解之后阅读德军最高机密的通讯。

你在《这才是网络安全》后面会正式学到这套“信息收集→分析→利用→提权”的渗透方法论——图灵在1940年就已经把它用在战场上了。

他的“渗透目标”是恩尼格玛，他的“漏洞”是德国人的密钥生成方式有固定模式。

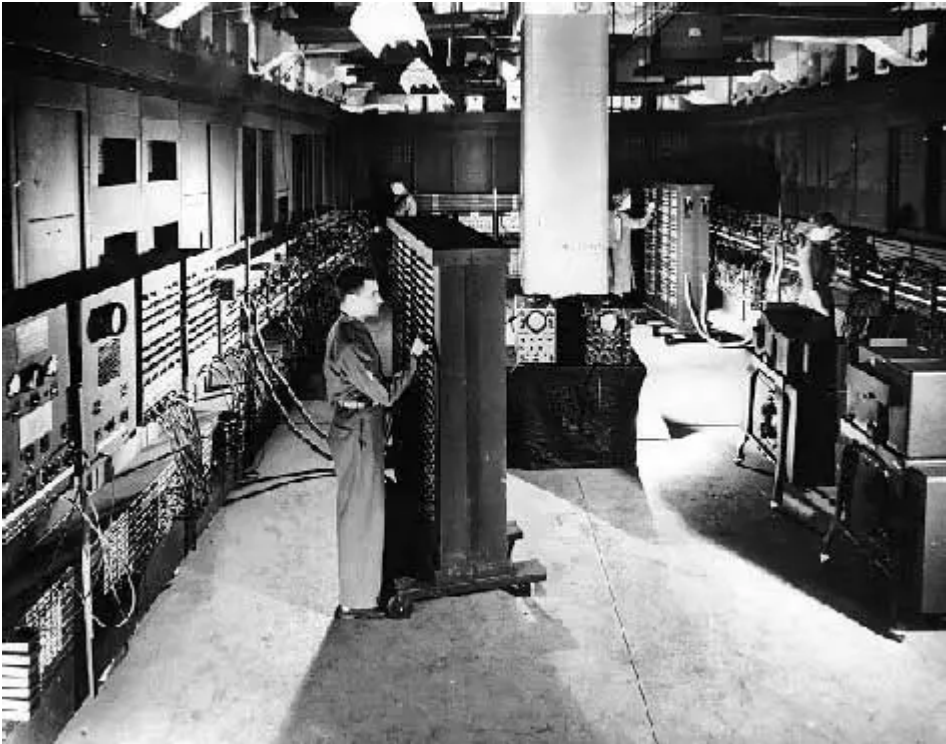
Kate 看完这段，说：“所以黑客思维不是从电脑黑客开始的。图灵在没有电脑的时候，就已经在用黑客的思路解决问题了。”

“对。他不是‘破坏者’，他是‘破译者’。这两者的区别，我们在后面的章节里会反复讲到。”

2.3 ENIAC：占地167平方米的“计算怪物”

当你听到“第一台电子计算机”这个词，多数人会想到ENIAC。

ENIAC全称是“电子数字积分计算机”（Electronic Numerical Integrator and Computer），1945年在宾夕法尼亚大学诞生。



它的数据有多夸张，我们来感受一下：

指标	ENIAC的数据	对比你手里的东西
占地面积	167平方米	约等于两个车库并排
重量	27吨	相当于5头成年大象
电子管数量	17,468个	你电脑里一个CPU集成了几十亿个晶体管
耗电量	150千瓦	够点亮1500个灯泡，一度电够它跑6分钟
运算速度	每秒5000次加法	你手机里的A系列/骁龙芯片是它的 几十亿倍
编程方式	人工插拔线缆+拨开关	键盘打字，VS Code自动补全

ENIAC是一头巨兽。它启动的时候，整个费城的灯光都会**变暗**，因为电网扛不住它的瞬时功率。它运行的时候，17,000多个电子管发出的热量足以让房间变成桑拿房，必须用大型空调强制降温。

Kate 盯着表格最下面一行看了半天：“等一下。编程方式是插线？不是打字？”

“对。没有键盘，没有屏幕。你要告诉ENIAC做什么，就得在它的面板上插几百根线缆，拨动几百个开关。”

Kate 低头看了一眼自己的键盘，然后说：“……我以后再也不抱怨快捷键不好使了。”

但你知道吗？**ENIAC最早的设计目标，根本不是“造一台通用计算机”——它的任务是：计算炮弹弹道。**

这是ENIAC最容易被误解的地方：它不是被“发明”出来的，而是被“需求”逼出来的。美国陆军需要一张炮弹射程表，而人工计算一条弹道需要一个人算**30个小时**——等算完，炮弹都已经打完了。ENIAC能在**30秒**内算出一条弹道。

30小时 → 30秒。这就是电子计算机带来的第一波冲击。这个“算力提升”的幅度，放在今天相当于你把一个大项目从一个月缩短到几分钟。

ENIAC在1946年正式向公众展示时，场面极其轰动。《纽约时报》管它叫“巨大的电子大脑”。当时的人们第一次看到：一群闪烁的电子管，能在几秒钟内算出人类要算几个月的东西。

电子计算机从“概念”变成了“实物”——就在这里，就在这一刻。

2.4 ENIAC 的“程序员”：六位被遗忘的女性

ENIAC的故事里有一个细节，我必须在你的书里专门拿出来讲。

ENIAC的“编程”和今天完全不同。没有键盘、没有屏幕、没有操作系统——你要让ENIAC干活，就得**手动插拔线缆、拨动上百个开关**，把机器内部的电路“连接”成你需要的运算路径。

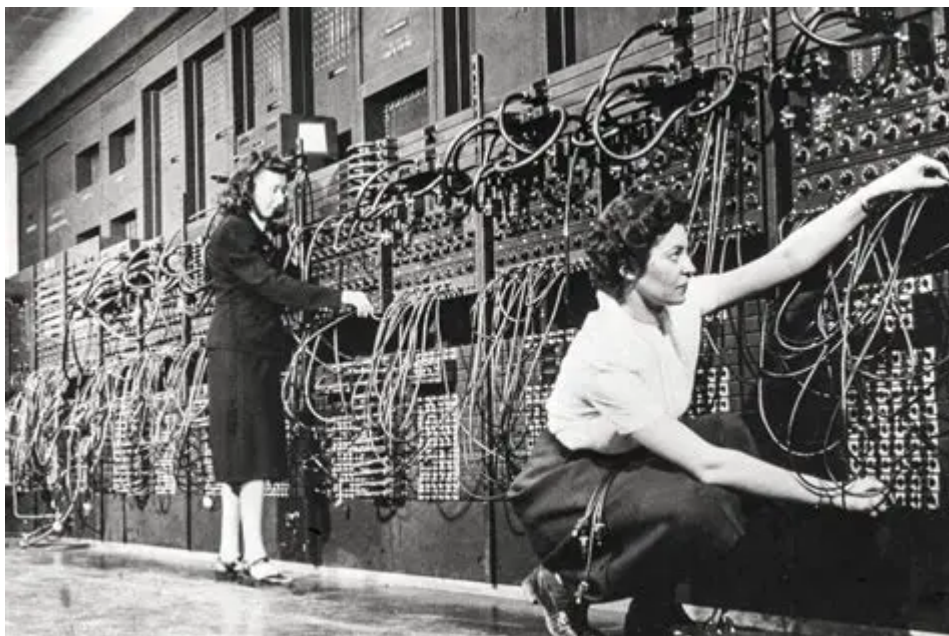
这是一项极其繁琐、极其考验耐心的工作——线路插错一根，整台机器算出来的就是垃圾。

而做这件事的人是谁？

六位女性数学专业毕业生。

- Kay McNulty
- Betty Jennings
- Betty Snyder
- Marlyn Wescoff
- Fran Bilas
- Ruth Lichterman

她们当时被叫做“**计算员**”（computers）——在电子计算机出现之前，“computer”这个词指的就是“做计算的人”，而且绝大多数是女性。（是的，这个词的原意和性别有关。在你之后的职业生涯里，如果遇到有人管你叫“computer”，记得骄傲地告诉Ta：最早那批computer是有名字的。）



这六位女性做的事情包括：

1. **理解炮弹弹道的数学方程**——把物理问题翻译成数学公式
2. **把数学公式拆成ENIAC能执行的“步骤”**——设计计算流程
3. **把步骤转成“电路连接方案”**——在ENIAC的面板上插拔几百根线缆
4. **调试**——ENIAC运行出错的时候，钻进机器里面找哪根线松了、哪个电子管烧了

你没有看错。她们既是“程序员”，也是“硬件工程师”。

照片上她们站在ENIAC旁边合影，笑容灿烂——但在当时，她们的工作没有被公开表彰。ENIAC揭幕的时候，报纸上只提到了机器的设计师，六位女性甚至没有出现在媒体报道中。她们的名字在很多年里消失在历史档案里，直到几十年后计算机历史学家才把她们的故事挖掘出来。

Kate 说：“所以她们做了所有实际的工作——写程序、插线、调试——但报纸上只写了设计师的名字。”

“对。而且这种不公平持续了很多年。历史学家翻了几十年的档案，才把她们的名字找回来。”

Kate 沉默了几秒，然后把那六个名字敲进了她的笔记里：“……那我们至少能在书里把她们的名字写出来。”

Kay McNulty后来在一次采访中回忆说：

“我们当时不觉得自己在做‘开创性’的事。我们只是觉得，这是一份工作，而且很有趣。”

她丈夫是ENIAC团队的另一位工程师。1985年她丈夫去世后，她翻出一堆旧资料，整理后交给计算机历史博物馆——才让这段被尘封的历史重见天日。

2.5 冯·诺依曼架构：计算机的“宪法”

ENIAC能跑能算，但有个致命问题：**它没法“记住”程序。**

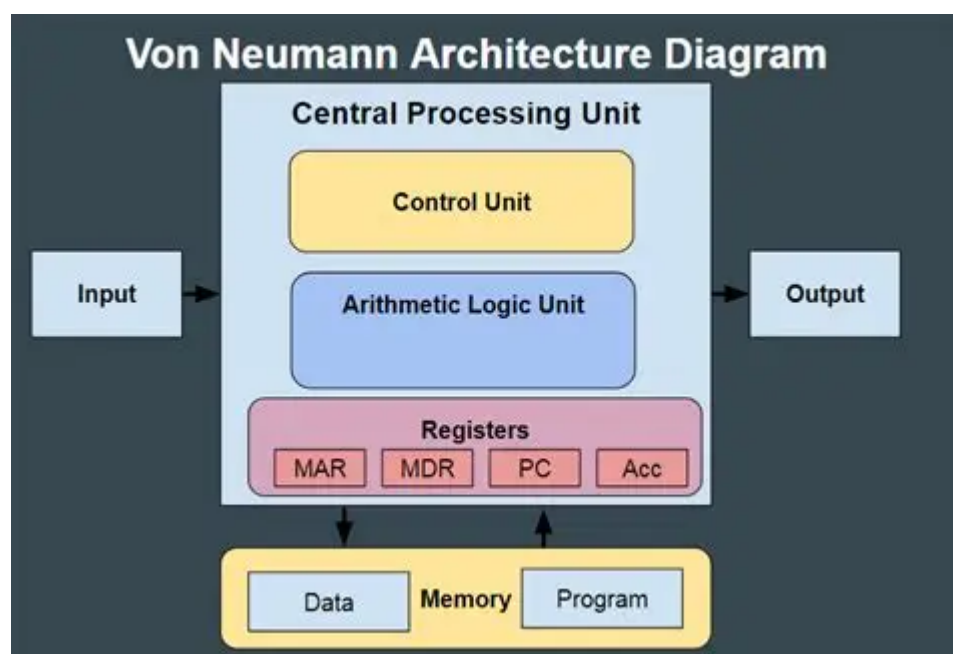
每一次换任务，都需要重新插拔线缆、重新拨动开关——相当于每次换一个程序，你就要把电脑拆开重装一遍。一个任务算完了，需要整整几天才能切换成下一个任务。

这是一个很“蠢”的设计。ENIAC的“程序”是电路连接，而不是存储在内存里的数据。

这个问题被一个人解决了——**约翰·冯·诺依曼**。

冯·诺依曼是匈牙利裔美国数学家，天才中的天才。他参与过曼哈顿计划，也参与过ENIAC的设计讨论。他在1945年写了一份报告（后来被称为“First Draft of a Report on the EDVAC”），提出了一个极其重要的设计思想：

把“程序”也当作“数据”一样，存储在内存里。



这个想法在今天看来理所当然——程序存在硬盘里，开机时加载到内存，CPU从内存里取指令执行——但在1945年，这是一个颠覆性的飞跃。

Kate 看着这张架构图，试图用自己的话解释：“运算是脑子，控制器是神经，存储器是记忆，输入设备是眼睛耳朵，输出设备是嘴巴。电脑跟人其实差不多。”

“精准。而且这个架构从1945年到现在，八十年没变过。你用的每一台设备——手机、电脑、服务器——都长这个样。”

冯·诺依曼架构的核心只有五个部件：

1. **运算器**（ALU）：做加减乘除和逻辑运算
2. **控制器**（Control Unit）：解读指令，指挥各部件工作
3. **存储器**（Memory）：存程序和数据

- 4. **输入设备**（Input）：把程序和数据放进机器
- 5. **输出设备**（Output）：把结果拿出来

这五个部件，构成了今天每一台计算机的基础框架。

不管你是用Windows、macOS还是Linux，是台式机、服务器、手机、还是智能手表——它们都遵循冯·诺依曼架构。你在《这才是网络安全》后面会学到内存溢出、代码注入、缓冲区攻击——这些攻击手法，都是针对这个架构的“弱点”设计的。

为什么说“存储程序”这个概念如此重要？

因为一旦程序可以像数据一样存进内存，就意味着：

- **程序可以自我修改**（早期的病毒就是这样干的）
- **程序可以被“偷换”**（攻击者可以把恶意代码塞进内存，伪装成合法程序——这就是缓冲区溢出攻击的雏形）
- **程序可以被“重载”**（你重启电脑就是重新加载程序）

换句话说：“存储程序”架构是计算机史上最伟大的设计之一，也是网络安全史上最大的“攻击面”之一。

你后面学渗透测试的时候，会反复听到“内存地址”“栈溢出”“ROP链”——它们都建立在冯·诺依曼架构的基础上。你现在记住这张图，以后会很省力。

2.6 从 ENIAC 到现代计算机

ENIAC之后，计算机的发展进入了“指数级”的爆发：

年代	里程碑	意义
1947年	晶体管发明（贝尔实验室）	代替了笨重、发热的电子管
1958年	集成电路发明（德州仪器）	把多个晶体管做在一个芯片上
1971年	英特尔4004微处理器	世界上第一颗CPU，一个芯片就是一台计算机的“大脑”
1975年	Altair 8800	第一台个人计算机，普通人可以买回家
1977年	Apple II	第一台“开箱即用”的彩色个人电脑
1981年	IBM PC	开放架构，兼容机时代开启

如果你把ENIAC（1945年）和你的手机（哪怕是一台三年前的旧款）放在一起对比：

- ENIAC占地167平方米，你的手机塞进口袋

- ENIAC重27吨，你的手机不到半斤
- ENIAC每秒5000次加法，你的手机每秒**几十亿次**运算
- ENIAC耗电150千瓦，你的手机电池只有几瓦时

70年间，计算机的性能提升了万亿倍。这在人类历史上是绝无仅有的——没有任何一项技术发展得如此之快。

而推动这一切的，是一个“正反馈循环”：

有了更好的计算机 → 可以设计更好的计算机 → 有了更好的计算机 → 可以设计……无限循环。

你现在用的VS Code、Git、Markdown、还有你正在写的这本《守夜者之书》的笔记——它们都是这个循环的产物。

2.7 小结：战争、巨兽与“0”和“1”

这一章讲了四件关键的事：

事件	年代	意义
布莱切利公园和图灵的“炸弹”机	1940年前后	专用密码破译机，用电子和机械混合的方式把二战缩短了两年
ENIAC诞生	1945年	第一台通用电子数字计算机，占地167平方米，宣告电子计算时代来临
ENIAC的六位女性程序员	1945-1946年	世界上第一批“程序员”，用插拔线缆的方式“编程”
冯·诺依曼架构提出	1945年	确定了“存储程序”的设计思想，成为所有现代计算机的“宪法”

而这四件事，最终共同指向一个结论：

计算机不再是“梦想”，它变成了“工具”。

——从巴贝奇的齿轮蓝图，到图灵的纸带机，到ENIAC的电子管巨兽——计算机从“理论上存在”变成了“房间里摆着一台真实能跑的机器”。

它不再只是数学家的玩具，而是**赢得战争的力量、科学家的大脑增强器、改变世界的基础设施。**

对于你来说，这件事意味着：

你是在一个“计算机已经真实存在”的世界里出生的。你不需要像巴贝奇那样用齿轮去想象，你不需要像艾达那样把代码写在纸上等一百年——你打开电脑，它就在那里。

你的任务不是“发明计算机”，而是“用好它、保护它、掌控它”。

后面的章节，我们会进入“黑客”的世界——那些比任何人都更早看透计算机本质的人。当其他人只会用计算机查资料、打字的时候，第一批黑客已经在研究“这东西还能干点别的什么”。

下一章预告：从真空管到沙子——晶体管如何让计算机从“国家机密”变成“人人都买得起”。

守夜者手记

1. ENIAC的“编程”方式和今天有什么根本不同？为什么“存储程序”这个概念如此重要？
2. 冯·诺依曼架构的五个部分分别是什么？你能用一句话向不懂技术的人解释清楚吗？
3. 你觉得——如果没有二战，电子计算机会在什么时候出现？战争是“催化剂”还是“助产士”？

写完后，`git add . && git commit -m "第二章读完：从巨兽到架构"`。

等你积累到第十次commit，回头看第一条笔记——你会看到自己的思考在加速，它本身就是你成长最真实的证明。

第三章：晶体管的逆袭——让计算机从房间变成桌面

3.1 电子管的天花板

我们在第二章看到，ENIAC是一头“吞电巨兽”。

但它的问题远不止“费电”两个字那么简单。我们来复盘一下电子管时代的“基础设施危机”：

- **电子管是玻璃做的**，里面是真空，靠加热灯丝发射电子。像灯泡一样，会发热、会老化、会坏。ENIAC平均每两天就有一个电子管烧毁——17,468个电子管里，你永远不知道下一个是谁。
- **电子管工作时温度极高**，整个机房跟烤箱一样，空调系统必须24小时运转。如果是夏天，外面三十五度，房间里能到五十度——程序员在里边插拔线缆，跟蒸桑拿没区别。
- **电子管体积大**，一个电子管跟你的拇指差不多粗。要想把计算机缩小到“一台能搬进办公室”的尺寸，用电子管是不可能的任务。

那时的人们已经意识到一个问题：**用玻璃管和铁架子搭计算机，这条路走到头了。**

但计算机不能停在实验室里。科学家们想要更快的运算速度，军方想要更小、更耐用的机器，而商业公司已经在琢磨“这东西能不能卖钱”。

必须找到一个**能代替电子管的、更小更稳定更省电的东西**。

答案——藏在沙子里面。

Kate 盯着电子管的照片看了半天，说：“所以ENIAC的程序员一边插线一边蒸桑拿？夏天五十度？”

“对。而且随时有个电子管会炸。你能想象你写代码的时候，电脑上17,000个灯泡，随时灭一个，你就得自己去找是哪个。”

“.....我收回之前对VS Code所有的抱怨。”

3.2 肖克利与晶体管的“意外”

1947年12月23日，美国新泽西州，贝尔实验室。

三个物理学家——**约翰·巴丁、沃尔特·布拉顿、威廉·肖克利**——在实验室里鼓捣一个用锗（一种灰色的半导体材料）做的小装置。他们想验证一个理论：用半导体材料能不能实现“放大信号”的功能。

结果，它工作了。

那是一个只有几毫米大的小玩意儿，长得像一块灰扑扑的石头，插着三根金属丝，能完成电子管的功能——放大电信号、开关电流——但**不需要加热、不消耗大功率、不会动不动就烧掉**。

他们管它叫“**晶体管**”（transistor）——名字来自 transfer + resistor，也就是“传输电阻”。



注意一个细节：他们做出第一个晶体管的时间是**1947年12月23日，圣诞节前两天**。贝尔实验室的官方记录里，试验日志上只写了一行字：

“发现了在锗表面的点接触方式可以实现电压放大。”

字迹很平淡，看不出任何情绪。但这一行字，后来被写进了每一本计算机教科书的“最重要时刻”列表里。

晶体管出现之后，计算机终于从“玻璃时代”进入了“固体时代”。

不过，贝尔实验室的专利部第一次见到这东西的时候，根本不知道该怎么分类。他们问肖克利：“这算什么？机械装置？电气装置？”肖克利想了想说：“你们就写‘电子装置’吧。”

——你看，就连发明它的人，当时也没完全意识到自己手里握着什么级别的“武器”。

Kate 说：“圣诞节前两天，别人在做姜饼人，这三个人在做晶体管。而且他们还不知道自己做出来的东西后来统治了世界。”

“历史最奇妙的地方就在这里——改变世界的那一刻，往往看起来非常不起眼。”

3.3 肖克利其人（和“硅谷”的起源）

说晶体管，就必须说它的发明者之一——**威廉·肖克利**。

肖克利是个极其复杂的角色。一方面，他是天才中的天才：物理学博士、贝尔实验室的明星研究员、晶体管专利的共同持有人。另一方面，他的性格里带着一种“令人窒息的自大”。他觉得自己什么都懂，其他同事的贡献在他眼里都不值一提。

这种性格在他后来的职业生涯里埋下了巨大的雷。

1956年，肖克利离开贝尔实验室，跑到加利福尼亚州一个叫**帕罗奥图**的小镇，创办了“肖克利半导体实验室”。

他选择这个地方是有原因的：帕罗奥图紧邻斯坦福大学，气候宜人，而且当时的斯坦福工程学院院长叫**弗雷德·特曼**——他有一个宏大计划：把斯坦福周围的空地租给高科技公司，打造一个“知识+产业”的聚集区。

特曼跟肖克利一拍即合。

肖克利招来了一批全美国最聪明的年轻科学家和工程师，其中就有后来英特尔公司的创始人——**戈登·摩尔**和**罗伯特·诺伊斯**。

结果呢？

肖克利的管理风格极其专制、多疑、控制欲爆棚。他动不动就怀疑手下员工“背叛他”。他甚至要求员工接受**测谎仪测试**，来证明他们没有背着他做别的项目。

你能想象一群顶级的物理学家和工程师，被老板逼着做测谎仪的感觉吗？

于是，1957年，肖克利手下最优秀的八位员工——包括诺伊斯和摩尔——集体辞职。

这八个人后来被称为“**叛逆八人组**”（Traitorous Eight）。肖克利气得暴跳如雷，骂他们是“叛徒”。



Courtesy of Special Collections, Stanford University Libraries

这八个“叛徒”出去之后，成立了一家新公司——**仙童半导体**（Fairchild Semiconductor）。而仙童半导体，在接下来的十几年里，成了整个硅谷的“黄埔军校”。

仙童的人辞职后创办的公司，包括：英特尔、AMD、国家半导体、阿尔特拉……几乎所有后来统治半导体行业的巨头，都能追溯到仙童血统。

这就是“硅谷”的真正起源。不是从某个政府规划开始的，是从八个人受不了老板的测谎仪、决定出来单干开始的。

而你手里的每一块芯片——包括你现在用的手机、电脑、键盘上的主控——都可以追溯到1957年那个“反叛”的冬天。

3.4 集成电路：把一堆晶体管塞进一粒沙

晶体管比电子管小得多，但也引来了一个新问题：**如果把成千上万个晶体管组装成一台计算机，那么把它们“连起来”的导线会多得吓人。**

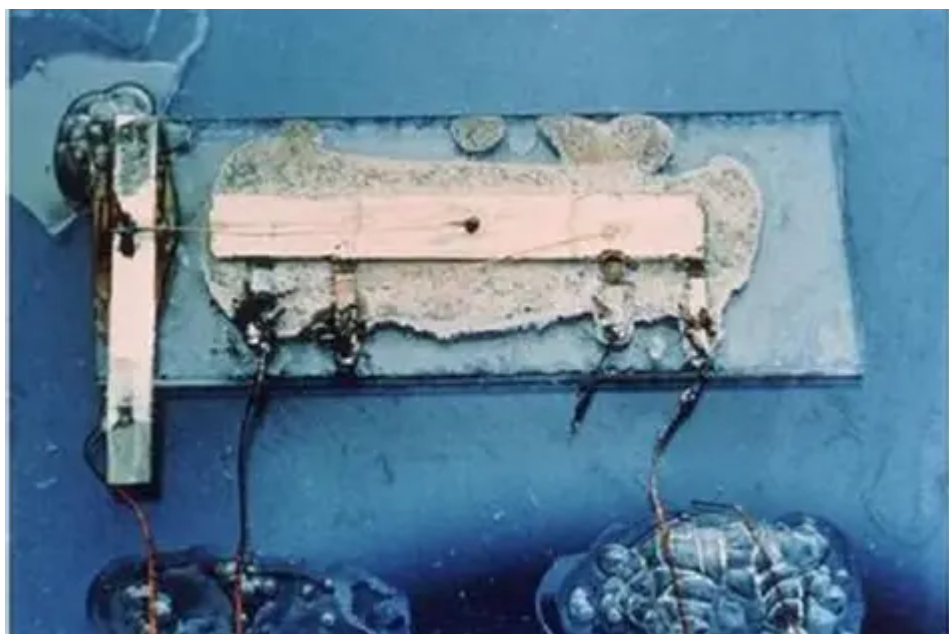
你想象一下：一台电脑里有几百万个晶体管，每个晶体管有三根脚，如果你用导线一根根把它们焊在一起……那个布线量足够让整个地球的铜线价格翻倍。

怎么办？

答案是：**别“焊”了，把它们“印”上去。**

1958年，德州仪器的工程师**杰克·基尔比**，突发奇想：能不能把晶体管、电阻、电容这些元件“做在同一个半导体材料块上”，用内部的薄膜线路代替外部的导线？

他做出了第一块**集成电路**（IC，Integrated Circuit）——在一块比指甲盖还小的锗片上，集成了几个晶体管和电阻。



几乎在同一时间，仙童半导体的**罗伯特·诺伊斯**也独立设计了类似的方案——而且诺伊斯的版本用的是硅（而不是锗），并且他用“平面工艺”解决了大规模量产的问题。

（又是“几乎同时”的独立发明——在技术史上这种事情非常常见，说明当时“集成电路”的构想已经呼之欲出。）

基尔比和诺伊斯后来都因为集成电路发明获得了极高的荣誉。2000年，基尔比还拿到了诺贝尔物理学奖——诺伊斯当时已经去世，否则大概率会并列。

你今天的手机里，用一块几毫米见方的芯片，里面集成了**几十亿个晶体管**。这几十亿个晶体管，就是基尔比和诺伊斯在1958年那个“奇思妙想”的直接后代。

3.5 英特尔 4004：世界上第一颗 CPU

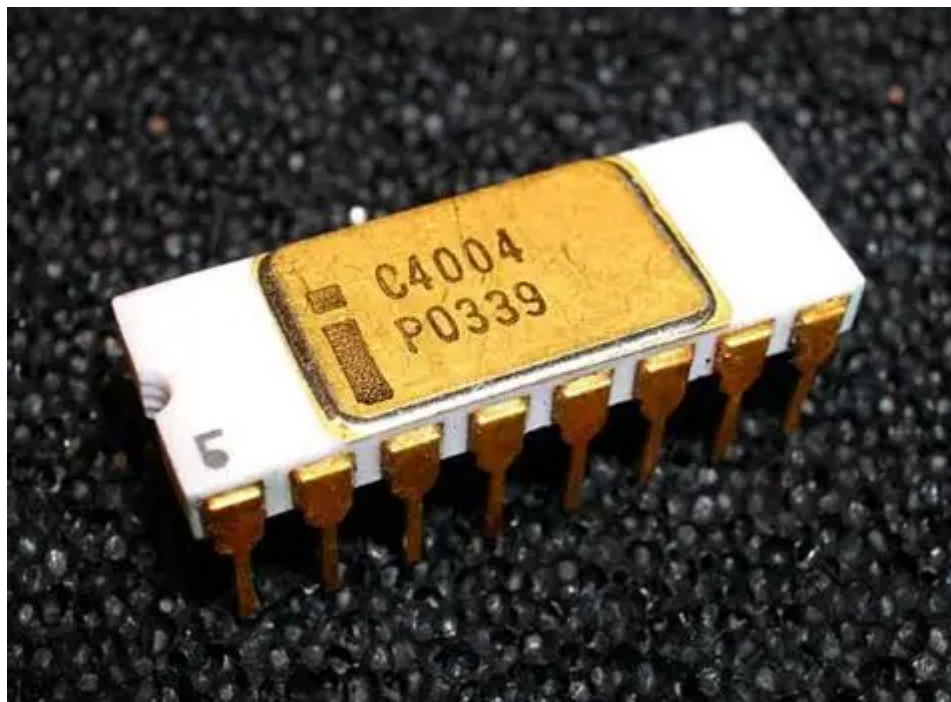
晶体管变小了，集成电路把它们整在了一起，但还有一个问题没解决：**怎么把“整个计算机的大脑”塞进一颗芯片里？**

这个问题在1971年得到了回答。

英特尔公司（就是摩尔和诺伊斯从仙童创办的那家）推出了**英特尔4004微处理器**。

4004有多小？**3毫米×4毫米**，大约是你小拇指指甲盖的三分之一。

4004有多强？它每秒能执行**9万次**运算。看起来很少？没错，比你手机慢了十几万倍——但1971年，它比ENIAC快了**18倍**，而且体积不到ENIAC的**百万分之一**，功耗不到它的**万分之一**。



这是人类历史上第一次：**一台计算机的“大脑”——运算器+控制器——可以被放进一个芯片里。**

从此，计算机不再是一个“房间里的巨兽”，它可以被做成一颗芯片，塞进收银机、计算器、微波炉、游戏机……以及后来的个人电脑。

英特尔4004的诞生，意味着两件事：

1. **计算机从“昂贵的大型设备”变成了“可以批量生产的日用品”。**
2. **个人计算机的时代，正式拉开序幕。**

3.6 “摩尔定律”：每18个月翻一番

在讲个人计算机之前，我们需要花30秒认识一条著名的“经验法则”——**摩尔定律**。

1965年，戈登·摩尔（就是仙童那八位“叛徒”之一，英特尔的联合创始人）在一篇杂志文章里预言：

“集成电路上的晶体管数量，大约每18个月到24个月会翻一番。”

这只是一条基于当时技术趋势的“经验观察”。但它被后来的半导体产业奉为圭臬——在接下来的几十年里，它的确做到了。

这意味着什么？

- 你手里的电脑，每一年半就比同价位的上一代快一倍；
- 存储容量每一年半翻一番；
- 同价位能买到的东西，每一年半就是上一代的“两倍性能”。

摩尔定律持续了约半个世纪，驱动了互联网、智能手机、云计算、人工智能——一切的一切。

Kate 问：“那我们学安全的人，摩尔定律跟我们有什么关系？”

“好问题。摩尔定律让计算机每一年半就变快一倍——防守方有更强的防火墙、更快的加密算法，但攻击方也有更强的暴力破解、更大的字典、更快的漏洞挖掘工具。你在后面几季会看到，很多攻击手法之所以‘现在能用了’，就是因为算力终于够用了。”

“……所以摩尔定律是两边同时发武器。”

在《这才是网络安全》后面几季你会看到：攻击者的工具变得更强、更快、更隐蔽，也是因为摩尔定律在悄悄给两边“发武器”。这是一场“算力军备竞赛”。

3.7 小结：从“房间里的大象”到“口袋里的玩具”

总结一下本章的核心变化：

技术节点	年代	体积	意义
电子管	1940年代	拳头大小	第一代计算元件，但笨重、耗电、易坏
晶体管	1947年	几毫米	更小、更稳定、不发热，为计算机小型化铺路
集成电路	1958年	指甲盖大小	多个晶体管做在同一块材料上，为微处理器奠基
微处理器 (4004)	1971年	小指甲盖的三分之一	整个CPU塞进一颗芯片，个人计算机的“大脑”诞生

你现在用的每一台设备——电脑、手机、平板、耳机、智能手表——里面至少有一颗微处理器，而那颗微处理器里的晶体管数以亿计，每一个都在执行着你赋予的指令。

你手里握着的，不是一个“工具”，而是七十年来“把计算变小”这场宏大工程的最终成果。

下一章，我们将真正进入“网络”的时代——计算机做好了，接下来要让它们彼此对话。

下一章预告： 冷战、ARPANET和“分布式”的野心——当计算机不再孤独。

守夜者手记

1. 晶体管相比电子管，最核心的优势是什么？（不要只回答“更小”，想想“为什么更小就重要”？）
2. 肖克利的“测谎仪事件”你觉得反映了什么？技术天才和优秀管理者是不是同一个人？
3. 如果摩尔定律有一天失效了，你觉得计算机还会继续变快吗？会用什么方式？

写完后：`git add . && git commit -m "第三章读完：晶体管与微处理器"。`

第四章：互联网的雏形——冷战、ARPANET和“分布式”的野心

4.1 当计算机还是“孤岛”

我们在前三章讲完了计算机的“肉身”进化史：从齿轮到电子管，从晶体管到微处理器。到 1960 年代末，计算机已经不再是实验室里仅供瞻仰的“神兽”——它开始出现在大学、研究所、政府机构和大型企业里。

但有一个问题，可能你从来没想到过：

这些计算机之间，不能互相说话。

一台在加州大学洛杉矶分校（UCLA）的计算机，和一台在斯坦福的计算机，直线距离不到 400 公里——但它们之间唯一的“通信方式”是：把数据打在纸带上，装进信封，开车送过去。

你没看错。1960 年代，“计算机网络”这个概念不存在。每台计算机都是一座孤岛。

如果你是一个当时的程序员，你想用另一台机器上的数据库？对不起，你要么亲自跑过去，要么打电话让那边的同事帮你查完再把结果念给你听。

你现在打开浏览器就能看全世界网页的这个“习惯”，在 1960 年代的人看来，属于科幻小说范畴。

而改变这一切的，是一个你意想不到的推力：**冷战**。

4.2 冷战与“去中心化”的恐惧

1957年10月4日，苏联成功发射了人类第一颗人造卫星——“**斯普特尼克1号**”。

这颗篮球大小的铁球，什么也没做，只是在太空里“嘀嘀嘀”地发无线电信号。但它在整个西方世界引发的恐慌，堪比一场战争。

为什么？因为它意味着：**苏联的导弹可以打到美国任何地方了。**

美国国防部立刻成立了一个新机构——**高级研究计划局（ARPA）**，专门资助“高风险、高回报”的前沿科技项目，确保美国在科技上不会再次被苏联“打脸”。

ARPA 最开始把钱砸向了太空、材料、武器系统。但到了 1960 年代中期，ARPA 的一个分支——信息处理技术办公室（IPTO）——把目光转向了一个更具体的目标：**让全国的计算机连起来。**

为什么军方对“计算机联网”这么感兴趣？理由非常“冷战”：

- **分布式的指挥系统**：如果核战争爆发，华盛顿的指挥中心被摧毁了，其他地方的计算机还能继续指挥战斗——前提是它们之间能通信。
- **资源共享**：全美国有几十台巨型计算机，每台造价几百万美元。如果能把它们连起来，一个人不管在哪，都能用上最远的那台机器——不用再长途跋涉去用特定的某台机器。

这就是 ARPANET 最初的动力：**不是为了让人们互相发消息，而是为了让计算机共享资源，同时让整个系统“打不死”。**

“打不死”这件事，在当时的冷战语境下，叫作**“去中心化”**。

4.3 保罗·巴兰的“热土豆”构想

在 ARPANET 正式立项之前，有一个人的想法深刻影响了它的设计——**保罗·巴兰**。

巴兰是一位波兰裔美国工程师，1960 年代初在兰德公司（RAND Corporation）工作。兰德公司是一个为美国政府提供军事战略研究的智库，冷战期间专门研究“如何打赢核战争”。

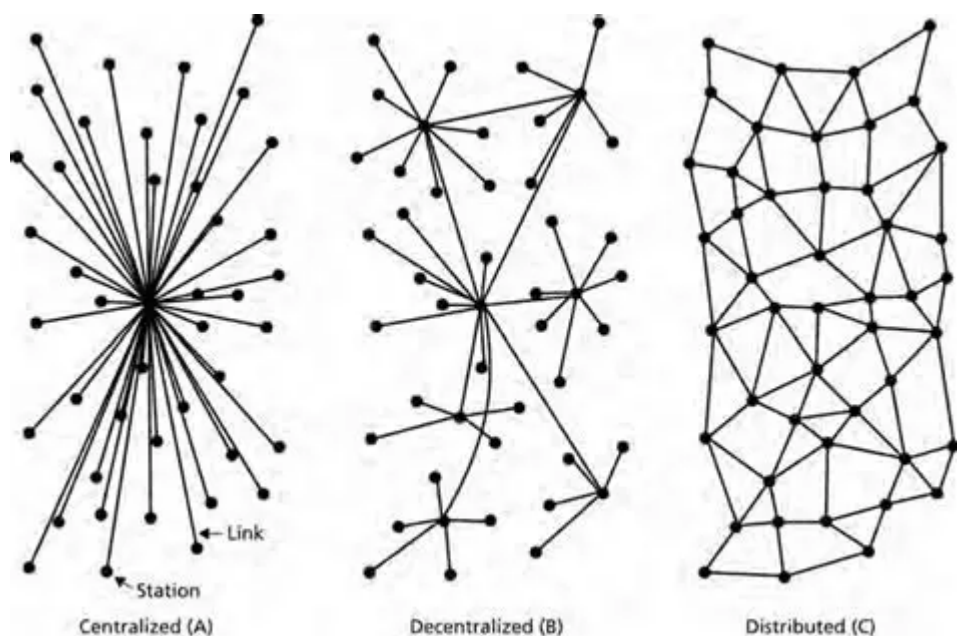
巴兰接到的任务是：**设计一个通信系统，即使遭到核打击，还能继续工作。**

他的答案是——**分布式网络**。

巴兰提出了一个今天听起来简单、但在当时极其反直觉的想法：

不要用“中央交换机”来连接所有节点。而是让每个节点都连接到多个其他节点，形成一个“网”。信息在网里不是“点对点”地直达，而是**像热土豆一样，从上一个节点甩到下一个节点，直到抵达终点。**

如果某条路径被炸了，信息会自己“绕路”走别的路径。



巴兰画了三张对比图：

1. **集中式**：一个中心节点连接所有终端——中心一炸，全完蛋。
2. **分布式**：多个节点互联，没有绝对中心——任何一处被炸，网络照样跑。
3. **网状**：每个节点都互相连接——最抗打，但成本最高。

巴兰推荐的是第二种：**分布式**。它不是最坚固的，但它是性价比最高的“不死之身”。

巴兰在 1964 年就写出了完整的技术报告。但当时 AT&T（美国电话电报公司，垄断全美通信网络）觉得他的想法太疯狂——他们花了上百年铺铜线建“中心交换”系统，你说要全部推倒重来？做梦。

于是巴兰的报告被束之高阁。

但五年后，ARPA 的年轻人翻出了他的报告，拍案叫绝：“这就是我们要的！”

4.4 ARPANET 的诞生：四个节点与一条“LO”

1966 年，ARPA 的一位年轻的计算机科学家——**拉里·罗伯茨**——接手了“计算机互联”项目。他读遍了巴兰的报告、英国的类似研究，然后拿出一份方案：

在全美国建立若干个“**接口消息处理器**”（IMP，Interface Message Processor），每台 IMP 连接一台大型计算机。IMPs 之间用专用电话线连接，形成一个**子网**。所有大型计算机通过 IMP 接入这个子网，彼此就能通信了。

IMP 本质上是一台“路由专用计算机”——它负责拆包、转发、寻址。它是现代路由器的“爷爷”。

1969 年，ARPA 正式拨款，在四个大学节点上部署第一张 ARPANET 子网：

1. **加州大学洛杉矶分校（UCLA）**
2. **斯坦福研究所（SRI）**

3. 加州大学圣塔芭芭拉分校 (UCSB)

4. 犹他大学

这四个节点，就是整个互联网的“创世四城”。

1969 年 10 月 29 日晚上 10 点 30 分，UCLA 的年轻程序员**查利·克莱恩**坐在一台 SDS Sigma 7 主机前，准备向 600 公里外斯坦福研究所的 IMP 发送第一条消息。

他输入的命令是“LOGIN”——第一个字母“L”，发过去了。第二个字母“O”，发过去了。第三个字母“G”——系统崩溃了。

这就是互联网历史上第一条消息：不是完整的“LOGIN”，只是一个半残的“LO”。

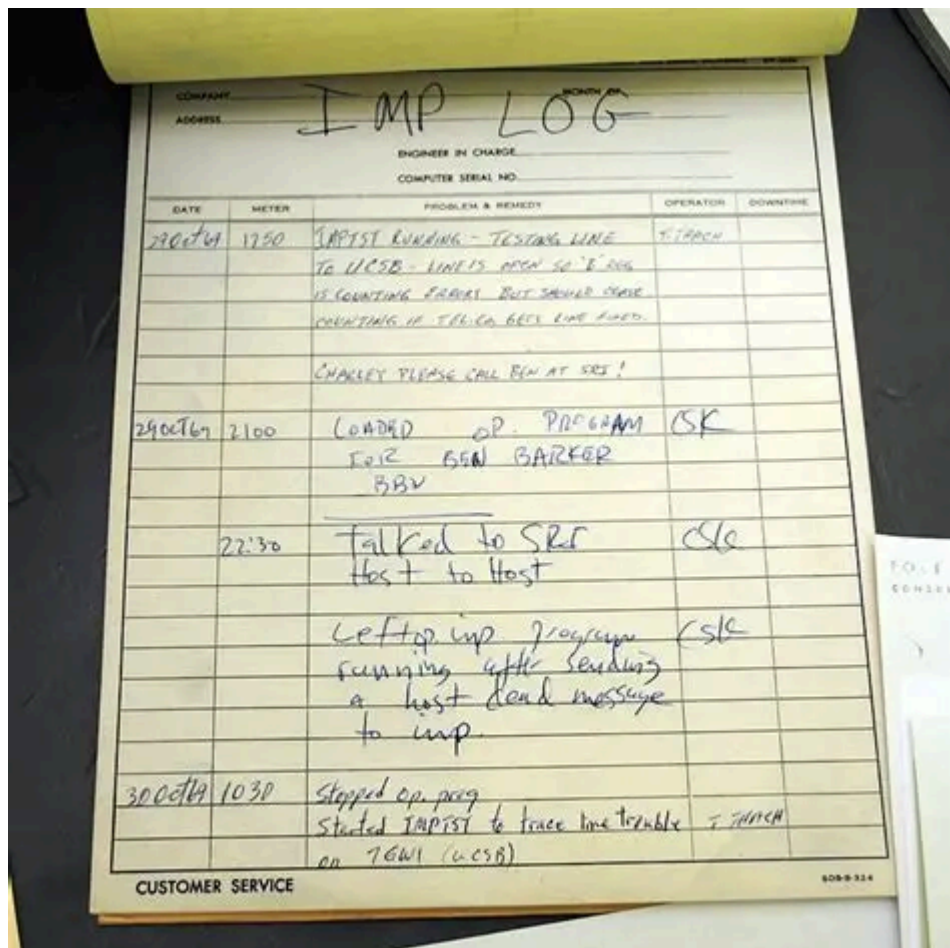
你可能期待一个更伟大的开场白——比如“Hello, World”或者“这是人类的一大步”——但现实比戏剧更戏剧：**互联网的第一句话，是一句没说完的“LO”，然后死机了。**

Kate 看到这里，说：“所以互联网的第一句话不是‘你好世界’，不是‘人类的一大步’，是一个没打完的‘LOGIN’——而且打到第三个字母就崩了。”

“对。而且他们一个小时候才修好。”

“……这比我的代码第一次上线还惨。我那次至少跑完了三分钟才崩。”

你以后写代码的时候如果遇到了“先跑起来再崩”的情况，记住：你并不孤单，互联网本人就是这样的。



不过别担心，一个小时后他们修复了系统，完整地发送了“LOGIN”。人类史上第一个“远程登录”成功了。ARPANET 正式诞生。

4.5 分组交换：把信息切块寄出去

ARPANET 能工作，靠的不是“打电话式”的连接——它用的是一种全新的通信方式：**分组交换 (Packet Switching)**。

分组交换的逻辑，和传统电话完全不同：

对比	电话系统（电路交换）	ARPANET（分组交换）
建立连接	先拨号，等对方接通，建立一条专线	不需要预先建立连接
资源占用	通话期间这条线路一直占着	数据按“包”发送，每包走自己的路，线路空闲时可被别人用
效率	占线率低，但浪费严重	利用率极高
容错性	中间断了，通话就断了	某条路径断了，包自动绕路

简单来说，分组交换就是：**把一封长信剪成几十个小纸条，每个小纸条写上收件人地址，分别通过不同的邮差送出去，到了收件人那边再重新拼起来。**

如果某个邮差的路上遇到堵车（网络拥塞），别的邮差还能从别的路送过去。

这个概念如今听起来天经地义，但在 1960 年代，它和“分布式网络”一样，被电信巨头嘲笑为“学术界的玩具”。AT&T 的工程师看了巴兰的论文后说：“这东西不可能工作——太复杂了，而且我们不想要它。”

事实证明，巨头们错了。

1970 年代：ARPANET 上跑的第一个“非学术应用”

ARPANET 最初只用于远程登录和文件传输——都是学术用途。但很快，用户发现了一件更有趣的事：**可以在 ARPANET 上聊天。**

1971 年，一位叫**雷·汤姆林森**的程序员，写了一个程序叫“SNDMSG”，可以让同一台机器上的用户互相留消息。然后他把 SNDMSG 和另一个文件传输程序（CPYNET）拼在一起，做了一个关键改动：**让消息可以发到其他机器上。**

他需要用个符号把“用户名”和“主机名”隔开。他在键盘上扫了一圈，选了“@”——今天全人类都在用的电子邮件地址的核心符号，就是这么随便选出来的。

Kate 说：“所以‘@’不是经过什么委员会讨论决定的，就是一个程序员低头看了一眼键盘，觉得这个符号最顺眼，就用它了。”

“对。而且他发的第一封邮件是发给他自己的。内容大概就是‘测试测试’之类的。”

“……测试邮件。互联网历史上第一封邮件，是一封测试邮件。我们组的测试用例比这个体面多了。”

第一封电子邮件，是汤姆林森发给他自己的——内容他后来记不清了。大概率只是一行测试文字。

但电子邮件诞生后，ARPANET 的使用量开始爆炸式增长。人们发现：**连计算机算力，远不如连计算机上面的“人”重要。**

这一下，ARPANET 从“军方的算力共享平台”变成“科学家们的聊天工具”。它的使用曲线在 1972 年后急剧上扬——从每月几万条消息，到每月几十万条。

4.6 TCP/IP：让所有网络“说同一种语言”

ARPANET 很成功，但出现了一个新问题：**它是封闭的。**

只有接入 ARPANET 的机构才能互相通信。别的网络（比如夏威夷大学的 ALOHA 网络、欧洲的科研网络）用着不同的协议标准，和 ARPANET 鸡同鸭讲。

这就像你开了个聊天室，只有你家小区的人能进，隔壁小区的人只能用对讲机。然后全世界的“网络孤岛”越来越多，却彼此不通。

谁来解决这个问题？

——**文特·瑟夫和鲍勃·卡恩。**

1974 年，他俩发表了一篇论文《分组网络互联协议》，提出了一个被称为 **TCP/IP** 的新协议族：

- **TCP（传输控制协议）**：负责把数据拆成小包、确保对方收到了、如果丢包就重发。
- **IP（网际协议）**：负责给每个小包写上“从哪里来、到哪里去”的地址，让它能穿过不同的网络找到自己的路。

TCP/IP 最核心的设计哲学是：“**网络只管传，应用自己处理错误。**”这听起来很糙，但正是这种“糙”，让它具备了极强的可扩展性——它不对下层网络做任何假设，不管是铜线、光纤、无线电、还是卫星链路，它都能跑。

关键一步发生在 1983 年 1 月 1 日。那天，ARPANET 正式从原来的 NCP 协议切换到 TCP/IP。所有节点必须在当天完成切换，否则就会被踢出网络。

这是互联网历史上最重要的“割接日”。这一天，标志着互联网从一个“单一网络”变成了一个“由无数不同网络组成的网络”——也就是“网际网”（Internet）。

从这一天起，世界不再有“多个网络”，只有一个“互联网”。

4.7 小结：从“打不死”到“连得上”

第四章讲了四件事：

事件	时间	意义
巴兰的分布式网络构想	1964 年	奠定了互联网“去中心化”的底层哲学
ARPANET 第一条消息“LO”	1969 年 10 月 29 日	人类第一次实现了“远程登录”
汤姆林森发明电子邮件和@符号	1971 年	人们发现“网络”最有价值的用途是“连接人”
TCP/IP 协议正式启用	1983 年 1 月 1 日	互联网诞生，一个“网络的网络”从此统治世界

你可能注意到了：互联网的整个发展史，本质上就是“去中心化”不断战胜“中心化”的过程。

- 巴兰想摆脱中心交换机——他赢了。
- 分组交换让信息不依赖单一线路——它赢了。
- TCP/IP 让任何网络都可以接入——它赢了。

而“去中心化”这个思想，也会在《这才是网络安全》后面反复出现——Web3、区块链、零信任架构、甚至黑客社区的运行方式——本质上都是对“中心化权力”的一种反抗。

Kate 说：“所以互联网不是被‘发明’出来的，是‘去中心化’一步一步打赢的。”

“对。巴兰画了三张图，AT&T嘲笑他。分组交换被说成‘学术玩具’。TCP/IP刚出来的时候也没什么人理。但现在，整个地球都在用。”

“……所以分布式网络是技术史上的‘逆袭剧本’。”

你现在正在读的《守夜者之书》，从第零季的理念到整个系列的开源模式，本身也是“去中心化”精神的一种表达：信息不锁在某个机构里，知识不垄断在课堂里。

这，才是网络安全。

下一章预告：当电脑不再是政府的玩具——个人计算机的黎明。

守夜者手记

1. 为什么 ARPANET 的第一条消息会是“LO”而不是“OK”或者别的什么？
2. 分组交换的核心理念是什么？为什么它让“网络抗打”？
3. 你有没有想过：为什么电子邮件用了“@”这个符号，而不是别的？

写完后： `git add . && git commit -m "第四章读完：从LO到互联"`。

第五章：个人计算机的黎明——当电脑不再是政府的玩具

5.1 1975年以前：电脑是“神坛上的圣物”

在1975年之前，如果你想碰一下电脑，你需要满足以下条件中的至少一条：

- 你是大学计算机系的教授或研究生（而且得是顶尖大学）；
- 你是军方或政府机构的研究人员；
- 你在某家超级有钱的大公司（比如IBM）的研发部门工作；
- 你有一张“准许使用计算机中心”的门禁卡，而那张卡比你的命还难拿。

换句话说，**电脑是“集体财产”，不是“个人物品”**。

当时的电脑长这样：一整间空调房里，立着一排排灰白色的金属机柜，上面闪烁着一排排小灯。操作员穿着白大褂（是的，白大褂！），在控制台上插拔线缆、拨动开关、放入纸带。整个过程看起来不像“用电脑”，更像“在核反应堆里做手术”。

如果你想写一个程序，流程是这样的：

1. 在纸上写出代码（用铅笔，因为要反复改）；
2. 把代码敲到穿孔卡片上（一张卡片一行代码，80个字符上限）；
3. 抱着一沓穿孔卡片（可能有几百张）走到计算机中心；
4. 把卡片交给操作员，让他帮你“排队”运行；
5. 等几个小时甚至一整天，回来取打印出来的结果；
6. 如果发现错了，重复以上全部步骤。

你写错一个逗号，就是一天的“往返跑”。

这种模式，后来被形象地称为“**批处理**”（batch processing）。你提交任务，机器帮算，过会儿来取结果——像去洗衣房洗衣服，而不是像在家用洗衣机随洗随取。

而在当时的计算机中心外面，有一群年轻人开始“造反”了。

5.2 Altair 8800：第一台个人电脑的“爆炸”

1975年1月，美国《大众电子》杂志的封面文章，预告了一台名叫“**Altair 8800**”的机器。

Altair 8800 是一台“组装套件”——你买回来的不是一台完整的电脑，而是一盒零件：一块电路板、一堆芯片、一个电源、一个金属机箱……你得自己焊、自己装、自己调试。

HOW TO "READ" FM TUNER SPECIFICATIONS

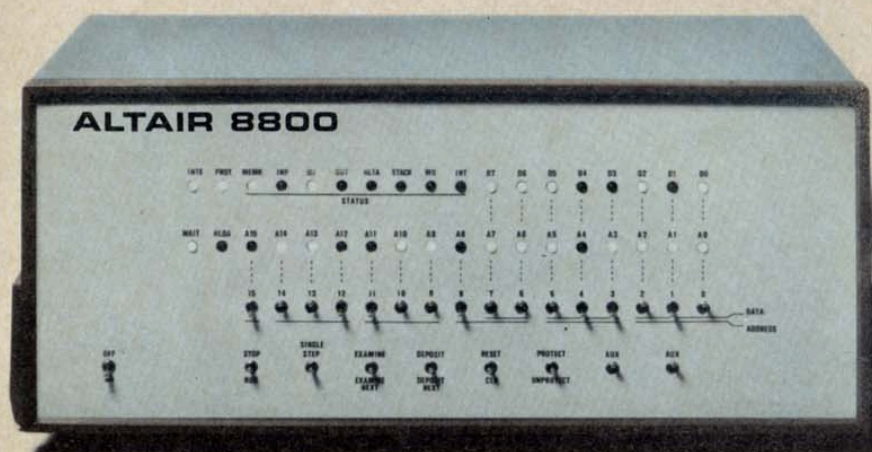
Popular Electronics

WORLD'S LARGEST-SELLING ELECTRONICS MAGAZINE JANUARY 1975/75¢

PROJECT BREAKTHROUGH!

World's First Minicomputer Kit to Rival Commercial Models...

"ALTAIR 8800" SAVE OVER \$1000



ALSO IN THIS ISSUE:

- An Under-\$90 Scientific Calculator Project
- CCD's—TV Camera Tube Successor?
- Thyristor-Controlled Photoflashers



TEST REPORTS:

Technics 200 Speaker System
Pioneer RT-1011 Open-Reel Recorder
Tram Diamond-40 CB AM Transceiver
Edmund Scientific "Kirlian" Photo Kit
Hewlett-Packard 5381 Frequency Counter

18101

Altair 8800 的配置在现在看来简直是笑话:

项目	配置
CPU	Intel 8080（比 4004 强一点，但也就那样）
内存	256 字节（你没看错，是 字节 ，不是 KB，更不是 MB）
输入输出	前面板一排拨动开关 + 一排小灯
存储	没有硬盘，没有软驱，什么都没有
价格	439 美元（套件价）

它怎么用？你要用拨动开关输入“机器码”——把二进制的指令一个比特一个比特地拨进去，然后按“运行”，看小灯亮灭判断结果。

如果你想让它算 1+1，步骤是：

- 1. 查手册，找到“加法指令”对应的二进制代码；
- 2. 把开关拨成对应的 0 和 1；
- 3. 按“存入”；
- 4. 再把数据（1 和 1）也拨进去；
- 5. 按“运行”；
- 6. 看小灯——如果亮的方式表示“2”，恭喜你，你“用电脑了”。

非常反人类，对吧？

Kate 盯着这段操作步骤看了三遍。“等一下。算 1+1，要拨开关、查手册、看小灯——六个步骤。我在 VS Code 里算 1+1，只需要打开终端，输入 Python，然后敲 1+1 回车。”

“对。你花 2 秒。Altair 用户花 2 分钟——而且他得先花几周学会怎么拨开关。”

“……我觉得我以后再也没有资格抱怨‘VS Code 启动太慢’了。”

但 Altair 8800 在发布后，订单像雪崩一样涌来。第一个月就卖了 1000 多台——在那个年代，1000 台对于一台“需要自己焊接”的电脑来说已经是天文数字了。

为什么它能火？

因为它**第一次证明了“电脑不是只能属于机构，个人也可以拥有”**。

一个高中生省下半年打工的钱，就能买一套 Altair 套件，在自己卧室里组装一台“真正的电脑”。这在 1975 年以前是不可想象的。

Altair 8800 的另一个贡献是：**它无意中创造了一个产业**。

买 Altair 的人很快发现：这台机器只有硬件没有软件。除了拨开关、看小灯之外，你几乎干不了任何事。

这时候，两个来自西雅图的年轻人看到了机会——他们一个叫**比尔·盖茨**，一个叫**保罗·艾伦**。他们给 Altair 8080 写了一个 **BASIC 语言解释器**，让用户可以用 BASIC 语言写程序，而不是拨开关输机器码。

这是微软公司的起点。

（你也许听说过“比尔·盖茨是首富”这件事，但你可能不知道——他的第一桶金，是从一台连屏幕都没有的 DIY 电脑开始的。）

5.3 家酿电脑俱乐部：硅谷的“地下网吧”

Altair 8800 点燃了个人计算机的第一把火。而真正把这个火堆烧旺的，是一个叫“**家酿电脑俱乐部**”（Homebrew Computer Club）的组织。

家酿电脑俱乐部 1975 年 3 月成立于加州门洛帕克，就在硅谷的核心地带。每周三晚上，一群电子爱好者、工程师、程序员、大学生，挤在一间车库里，传阅零件、展示作品、交换想法。气氛很随意——有人带一箱可乐，有人带一堆电路板。

这个俱乐部的成员，后来创办了一长串改变世界的公司：

- **史蒂夫·沃兹尼亚克**和**史蒂夫·乔布斯**在这里展示过 Apple I 的原型机；
- **亚当·奥斯本**后来创办了奥斯本电脑公司（第一台便携式电脑）；
- 还有很多你叫不出名字但影响了整个行业的“无名之辈”。

家酿电脑俱乐部的第一条规则是：“**你在这里学到的东西，必须在下次会议时分享给大家。**”

Kate 说：“这条规则，跟守夜者社区的规定好像。”

“本质上一样。学会了，分享出去。你写笔记、写教程、Fork 仓库——都是‘分享’的现代版本。”

“……所以我们每个写教程的人，其实都是家酿俱乐部的精神后代。”

——这就是黑客精神早期的“实物版本”。它不是学术会议那种“听完给个证书”，而是**互相帮对方做东西、修东西、想办法让机器跑起来**。

如果你在那个年代走进俱乐部，你会看到这些：

- 有人拿烙铁焊主板上的电容；
- 有人围着一台 Altair 讨论为什么它老是死机；
- 有人在黑板上画电路图争论不休；
- 有人用磁带录音机“存储”程序（把程序转成音频信号录在磁带上，读的时候再放一遍——像传真机一样）。



家酿俱乐部最宝贵的东西不是技术，而是一种**氛围**：“你可以做一台自己的电脑——这没什么不可能的。”

5.4 沃兹和乔布斯：车库里的革命

1976年，家酿俱乐部里出现了两个面孔——**史蒂夫·沃兹尼亚克**和**史蒂夫·乔布斯**。

这两个人，性格完全相反：

- **沃兹**：技术天才，性格腼腆，动手能力极强，写代码和设计电路板是他最大的乐趣。他设计东西的时候会把自已关起来，连续几天不眠不休地工作，然后带着成品出现，像一个正在展示发明的小学生。
- **乔布斯**：商业天才，极其强势，对审美有近乎偏执的要求，能说服任何人做任何事。他当时留着长发，穿破洞牛仔裤，说话语速很快，看一眼沃兹的电路板就能列出一堆改进意见。

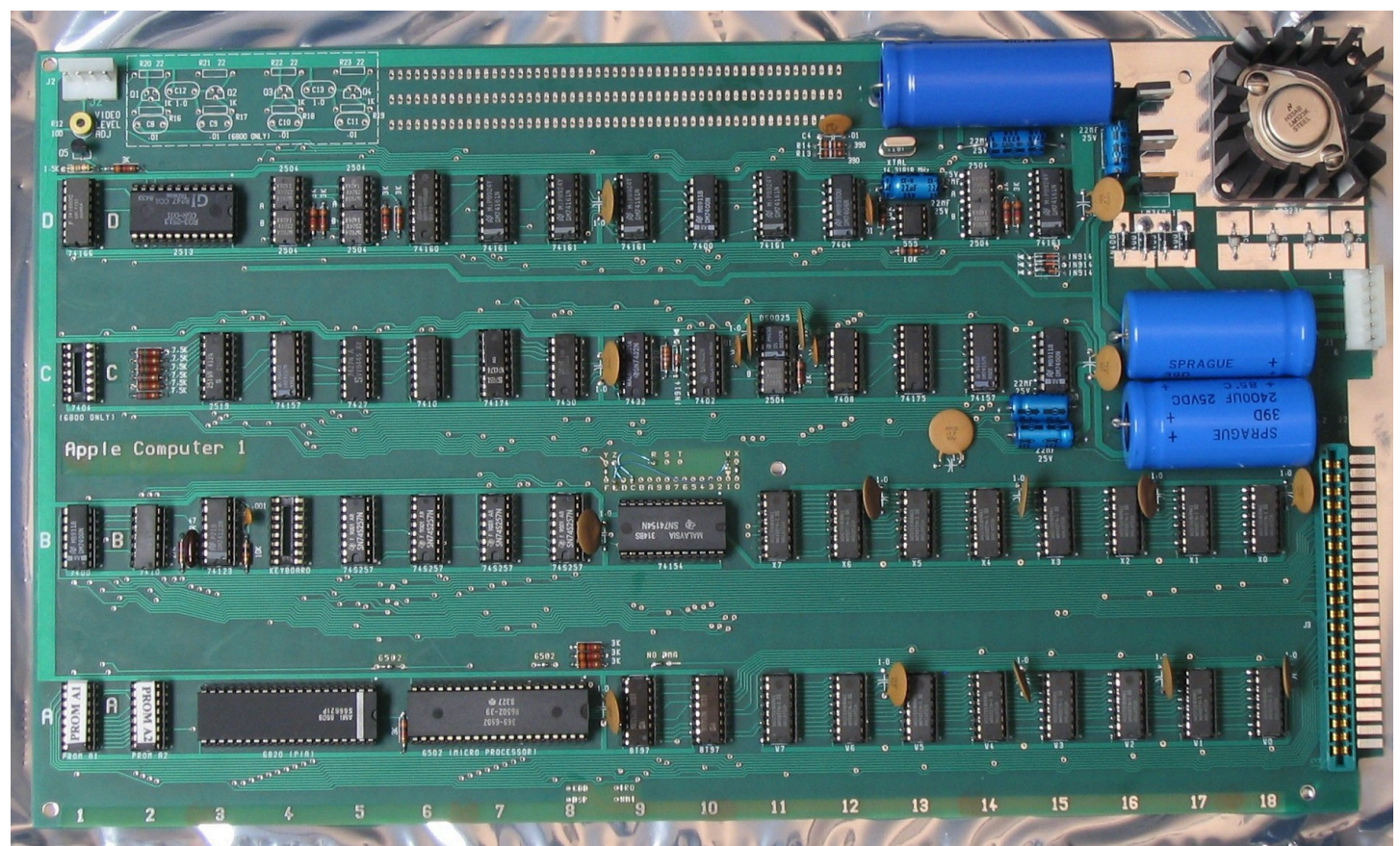
Kate 说：“所以沃兹是那个‘把东西做出来的人’，乔布斯是那个‘让全世界都想买它的人’。”

“精准。沃兹造了 Apple I 和 Apple II。乔布斯把它卖给了五百万个家庭。”

“……最完美的组合，也是最危险的组合。没有沃兹，乔布斯只能卖空气。没有乔布斯，沃兹的东西可能永远停在家酿俱乐部的车库里。”

他们俩的关系，很像“科学家 + 推销员”的完美组合。

沃兹设计了一台电脑——**Apple I**。它是一块裸电路板，没有键盘、没有显示器、没有外壳。插上电源后，你必须自己连接一个键盘和一台电视机才能用。



乔布斯说：“不能只卖电路板。我们要卖整台电脑。”

他去找了一个电子元件零售商，拿到了第一笔订单——50台。他们借了乔布斯家的车库当装配车间，沃兹焊电路板，乔布斯找人做机箱、电源、键盘。三周后，50台 Apple I 交付了。

Apple I 的售价是 666.66 美元——乔布斯选这个价格，是因为他喜欢重复数字。

Apple I 卖得不火，但足以让两兄弟确信一件事：**个人电脑有市场。**

1977年，沃兹完成了他的“终极作品”——**Apple II**。

Apple II 是史上第一台“开箱即用”的个人电脑：

- 自带键盘；
- 自带彩色显示输出（你可以接到电视机上）；
- 自带 BASIC 语言解释器（开机就能编程）；
- 自带两个软驱插槽（可以存数据了）；
- 一个漂亮的米色塑料外壳——不再是一堆裸露的电路板。



Apple II 发布的时候，现场的人惊呆了。他们第一次看到一台电脑“长得像个家用电器”，而不是像实验室设备。你把它买回家，接上电视，插上电源，开机——屏幕上出现一个闪烁的光标，你可以开始打字了。

这是人类历史上第一台“真正卖给了普通人”的电脑。

Apple II 卖了整整 16 年，累计销量超过 500 万台。它也是很多人的第一台“启蒙计算机”——

- 第一批电子表格软件（VisiCalc）跑在 Apple II 上；
- 第一批文字处理器跑在 Apple II 上；
- 第一批电脑游戏跑在 Apple II 上；
- 甚至，第一批“密码破解实验”也在 Apple II 上跑过。

乔布斯后来有一句非常有名的话：

“计算机是一个‘思想的自行车’——它让你的大脑跑得更快、走得更远。”

而沃兹的回应更朴实，也更“黑客”：

“我只是想让我自己能做更多的事——然后顺便让别人也能做更多的事。”

5.5 IBM PC：蓝色巨人的入场

Apple II 火了之后，所有人都开始相信“个人电脑是门大生意”。

1970 年代后期，市场上出现了一大堆品牌：Commodore、Tandy、Atari.....但最大的那个玩家——**IBM**——还在观望。

IBM 是“计算机行业”的代名词。从 1950 年代起，IBM 就是“大电脑”的代名词：主机、银行系统、政府数据库。他们卖一台大型机的价格，可以买一栋楼。

在 IBM 看来，个人电脑是“玩具”——他们觉得这种小东西撑不起多少营业额。但苹果在 1980 年上市后市值突破 10 亿美元，IBM 的高层终于坐不住了：

“玩具”变成了“战场”。

IBM 决定加入战局。但他们面临一个问题：大公司的内部流程太慢了。按照 IBM 的标准流程，做一台新电脑至少要三到五年，等做出来，苹果已经卖了几百万台了。

于是 IBM 做了一个大胆决定：**绕过自己的内部流程，成立一个“独立小组”——这个小组可以不受公司内部规章限制，可以自由选择供应商，甚至可以用非 IBM 标准的零件。**

这个小组被派到佛罗里达州博卡拉顿市的一间小办公室里，关起门来加班加点干活。他们只有一个要求：**快。**

他们用了不到一年时间，完成了 IBM 个人电脑的设计——**IBM PC。**

1981 年 8 月 12 日，IBM PC 正式发布。



IBM PC 有几个关键设计：

1. **使用英特尔 8088 CPU**——和苹果用的摩托罗拉芯片不同，这个选择后来成了“PC 兼容机”的基石。
2. **使用微软的 MS-DOS 操作系统**——IBM 没有自己写操作系统，而是找微软买了一个。比尔·盖茨当时保留了一个关键权利：他可以把这个系统卖给其他厂商。（这个决定让微软后来成为软件巨头。）
3. **采用了“开放架构”**——IBM 把整个技术规格公开了。任何人都可以生产兼容的部件：扩展卡、外设、甚至整台“克隆机”。

为什么 IBM 要“公开”自己电脑的设计？

因为 IBM 当时的商业模式是“卖硬件赚钱”，他们不靠软件、不靠配件赚钱。他们觉得：公开设计反而会让更多人购买 IBM 标准的设备——然后他们就买 IBM 的电脑。

但他们算错了一件事：**别人可以做得比他们更便宜。**

IBM PC 发布后，大量兼容机厂商涌入市场。最著名的是**康柏**（Compaq）——他们用逆向工程（合法地分析并复制 IBM 的 BIOS 功能，而不是直接复制代码）造出了“完全兼容 IBM PC”的电脑，价格却比 IBM 便宜很多。

IBM 开放了自己电脑的“设计规范”，却没有办法阻止别人用它。于是“兼容机”时代开始了。

而兼容机时代，最终带来了我们如今熟悉的局面：**一台电脑可以用任何厂商的零件组装——CPU 可以买英特尔或者 AMD，内存可以买金士顿或者海盗船，硬盘可以买希捷或者西部数据。**

这个“兼容”模式，让电脑从“特定品牌的专属产品”变成了“标准化的大众消费品”。价格不断下探，性能不断提升，最终走进了每一个家庭——包括你家。

5.6 从“玩具”到“必备品”

个人计算机从 1975 年的 Altair 到 1981 年的 IBM PC，只用了 6 年时间——这 6 年，完成了从“极客的玩具”到“家庭的必备品”的蜕变。

但真正让电脑成为“必需品”的，是 1980 年代中后期出现的几款软件：

- **VisiCalc**（1979）：第一代电子表格软件，让商业用户纷纷掏钱买 Apple II——因为算账比手算快一百倍。
- **WordStar**（1979）：第一代真正可用的文字处理器，让写作者彻底告别了打字机和涂改液。
- **Lotus 1-2-3**（1983）：更强大的电子表格，成为 IBM PC 上第一个“杀手级应用”。
- **AutoCAD**（1982）：让建筑师和工程师可以在电脑上画图。

这些软件的出现，让计算机从“能用来写程序的东西”变成了“能帮所有人解决实际问题的工具”。

一个工具只有成为“工具”的时候，它才会被所有人接受。

而计算机一旦被所有人接受，它在“家庭”里就有了立身之地。于是有了后来的“多媒体电脑”“上网电脑”“智能手机”……一步一步，走到你现在手里这个设备。

你不需要知道这些软件具体怎么用，你只需要知道一件事：

如果没有这些“普通人需要的东西”，你今天看的《守夜者之书》可能还在某本纸质杂志的连载专栏里，而不是像现在这样，用 Markdown 写在你的 VS Code 里，用 Git 管理，推送到 GitHub 上——然后免费开源给全世界。

你的“写作方式”，就是个人计算机时代最直接的成果。

5.7 小结：从“政府的玩具”到“你的玩具”

第五章讲了四件事：

事件	年代	意义
Altair 8800 发布	1975 年	第一台个人电脑套件，点燃了 PC 革命
家酿电脑俱乐部	1975 年起	“共享-分享-动手”的精神大本营
Apple II 发布	1977 年	第一台“开箱即用”的个人电脑，把电脑带进了家庭
IBM PC 发布 + 开放架构	1981 年	标准化+兼容，让电脑从“贵价专供”变成“大众平价”

而在这些“冰冷的硬件”背后，有一根贯穿始终的线：

从 Altair 的拨动开关，到 Apple II 的开机即用，到 IBM PC 的开放兼容——每一步都在降低门槛、扩大参与、让更多人拥有“算力”。

而你——作为 21 世纪的科技特长生——已经站在了这些巨人的肩膀上。你不用拨开关、不用焊电路、不用背机器码。你打开 VS Code，写一行 Markdown，全世界都能看到。

这就是个人计算机给你的“特权”。

下一章预告：“hack”本来是件好事——黑客的起源与真正含义。

守夜者手记

1. 如果你回到 1975 年，你会买一台 Altair 8800 套件自己焊吗？还是觉得“太折腾了”？
2. 家酿电脑俱乐部的“分享精神”和黑客伦理有什么关系？
3. Apple II 和 IBM PC 对“普通人使用电脑”的影响有什么不同？

写完后： `git add . && git commit -m "第五章读完：个人计算机的黎明"`。

中部：黑客历史——从“电话飞客”到“数字侠客”

第六章：黑客的起源——“hack”本来是件好事

6.1 在“黑客”被污名化之前

我们今天说起“黑客”这个词，脑子里浮现的是什么画面？

可能是这样的：一个戴着兜帽的人，坐在漆黑的房间里，脸上映着荧幕的绿光，手指在键盘上飞速敲击，然后屏幕上一行行绿色字符滚动，最后出现一行大字——“ACCESS GRANTED”。

好莱坞在过去三十年里，把这个画面重复了大概一万遍。

但在“黑客”被塑造成这种形象之前，它完全是另一个意思。

“hack”这个词，最早跟“犯罪”“入侵”“网络攻击”没有任何关系。它的原始含义，朴素到让你觉得不可思议：

“hack” = 一个巧妙的、非常规的、但极其有效的解决方法。

你可能听过“生活小妙招”这种说法——用橡皮筋固定手机支架、用回形针当书签、用电吹风加热让胶水松动……这些东西，本质上就是“hack”在日常生活中的体现。

而在计算机世界里，“hack”的含义更具体一些：**用创造性的方式，让机器做出原本设计者没想过它能做的事。**

不一定是“坏事”。也不一定是“好事”。它是一种**思维方式**。

而我们要讲的，就是这种思维方式是怎么来的，以及它如何从一群大学实验室里的“恶作剧爱好者”演变成了我们今天既敬畏又警惕的“黑客”。

6.2 “hack”这个词的源头——MIT 的火车模型

要追“hack”这个词的源头，最公认的说法指向一个地方——**麻省理工学院（MIT）**。

时间大约在 1950 年代末到 1960 年代初。

MIT 有一群学生，成天泡在学校的“模型铁路俱乐部”（Tech Model Railroad Club，简称 TMRC）里。这个俱乐部做的事情很简单：搭一个超大型的火车模型沙盘，让火车在上面跑。

但 MIT 的学生不是普通人。他们不满足于“让火车跑起来”——他们想的是：

- 能不能让火车**自动**跑？
- 能不能让道岔**自动**切换？
- 能不能让信号灯根据火车的位置**自动**变化？
- 能不能让整个系统像一台真正的铁路调度系统一样运行？

于是他们动手了。他们用电话系统的继电器（对，就是那种“咔嗒咔嗒”响的老式继电器）搭了一套控制系统，让整个火车模型实现了自动化。

这套系统叫“**信号与动力系统**”（Signals and Power，简称 S&P）。它用了成百上千个继电器，铺满了整个俱乐部的地下室，电线密密麻麻像蜘蛛网一样。

而操作这套系统的人，TMRC 俱乐部的核心成员，给自己起了一个称呼——“**hackers**”。

在当时的 TMRC 语境里，“hacker”的意思是：**一个能用继电器和电线搭出复杂系统的人，一个能把东西做得比设计图纸更巧妙的人，一个“会折腾”的人。**



这个群体里有一个后来非常出名的人物——**理查德·格林布拉特**。他是 TMRC 的第一代“hacker”之一，也是后来 MIT 人工智能实验室的第一批程序员。他的名言是：

“当你看到一个问题，你不去想‘这个能不能解决’，你只去想‘怎么解决’。”

你看，这种思维方式跟“攻击”没有任何关系。它只跟一件事有关：**好奇心和动手能力。**

Kate 说：“所以最早的‘黑客’，其实是一群玩火车模型的大学生。他们觉得自己是 hacker，是因为他们用继电器让火车自动变道——不是因为他们在网上入侵了谁。”

“对。hack 的原意，跟犯罪没有任何关系。它就是一种‘我看看这玩意儿还能干点啥’的好奇心。”

“.....那我们每天在 VS Code 里写代码、调 bug，其实就是在 hack。”

你可能在想：“火车模型跟网络安全有什么关系？”

关系很大。因为 TMRC 的这群“hacker”后来大多进了 MIT 的计算机实验室，成了最早一批与计算机“亲密接触”的人。他们把在火车模型上练出来的那种“让系统做超出设计者预期的事情”的思维，带到了计算机上。

于是，“hack”的定义从“让火车自动变道”扩展到了“让计算机做有趣的事情”——而这个扩展，打开了潘多拉的盒子。

6.3 第一批“计算机黑客”：MIT 人工智能实验室

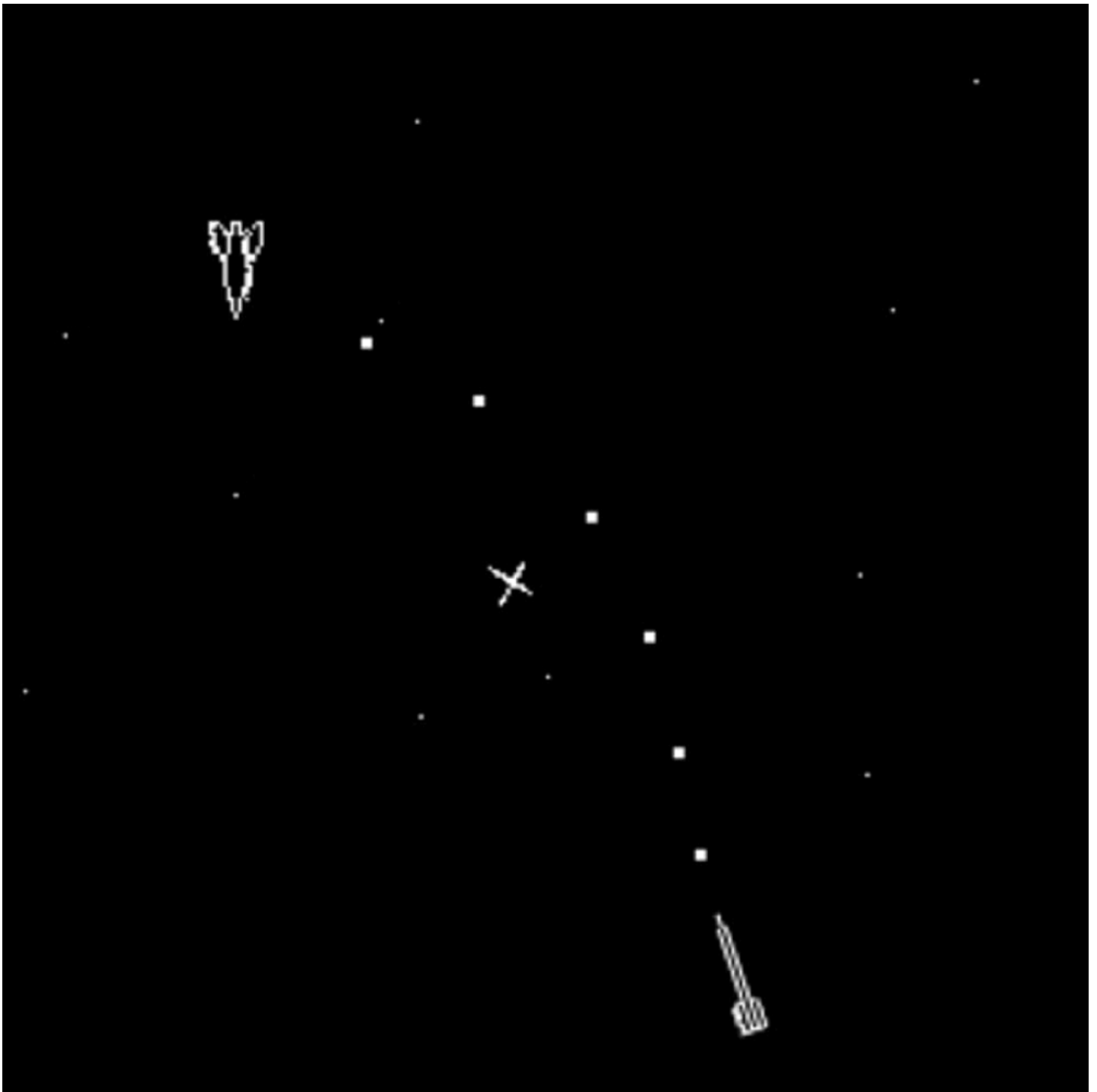
1960 年代，MIT 得到了两台当时最先进的计算机——**PDP-1** 和 **PDP-6**（都是 DEC 公司生产的，大小相当于一个文件柜，比 ENIAC 小太多了）。

TMRC 的那群火车模型爱好者，顺理成章地成了这两台计算机的第一批“用户”。他们不是计算机专业的学生（当时还没有这个专业），他们是电子工程、数学、物理系的学生，纯粹出于兴趣去“玩”电脑。

而“玩”的方式是这样的：

- PDP-1 配有一个**CRT 显示器**和一个**键盘**——这在当时是极其奢侈的配置。大多数计算机还在用穿孔卡片和打印纸输出，PDP-1 已经可以让你在屏幕上“实时”地看到自己输入的内容了。
- 这群学生写了一个程序，让屏幕上显示一个**点**。然后他们写了一个程序，让这个点可以**移动**。然后他们写了一个程序，让这个点变成一个**飞船**，可以用键盘控制方向，可以发射子弹，可以躲避障碍物。

这个程序，就是人类历史上第一款电子游戏——《**太空战争！**》（**Spacewar!**）。



你没有听错。世界上第一款电子游戏，是黑客“玩”出来的。它不是商业产品，不是游戏公司的立项，就是一群 MIT 的学生想在 PDP-1 上“看看能做点什么好玩的东西”。

Kate 说：“等一下。世界上第一个电子游戏，不是游戏公司做的，是一群 MIT 的大学生用实验室的电脑写的？”

“对。而且那台电脑的官方用途是‘科学计算’，不是‘打游戏’。”

“.....所以第一个黑客项目，是一个电子游戏。不是病毒，不是木马，是一个太空飞船对射的游戏。”

而这款游戏的创造者——**史蒂夫·罗素**和他的朋友们——就是“计算机黑客”的第一代代表人物。

他们做的事，用现在的标准来看，就是“hacking”的经典模板：

- 你有一台机器。
- 这台机器的官方用途是“计算”。
- 你发现它还能干别的。
- 你把它干成了。

这种做法，和 TMRC 的火车模型自动化一脉相承。

在 MIT 人工智能实验室里，这些早期黑客形成了一套不成文的“工作准则”：

1. **所有信息都应该是自由的。** 如果你写了一个好程序，你应该把源代码分享给所有人，而不是藏起来。
2. **不要重复造轮子。** 如果别人已经写了某个功能的代码，你可以直接用，也可以改进它，但不要从头开始写。
3. **用手解决问题。** 机器是用来帮你做事的，不是用来限制你的。如果你想让它做某件事，你就自己动手改代码。
4. **做有趣的事情。** 如果你做的事情不够“酷”，那你就没抓住黑客精神的精髓。

这里面没有任何一条提到“入侵”“破坏”“盗取数据”。这些准则的核心是：**探索、创造、分享。**

而这，就是黑客精神最纯粹的“原始版本”。

6.4 “hack”不是“break”——一个重要的概念区分

在 MIT 黑客们的语境里，他们从来不用“hack”这个词来表示“入侵”或“破坏”。

他们有另一个词来表示“恶意搞破坏”——“**crack**”。

- **Hacking** = 创造性地解决问题，让机器做有趣的事。
- **Cracking** = 恶意破坏系统，盗取数据，搞恶作剧。

但 1960 年代没有“网络安全”这个概念，因为这些计算机压根就不联网（ARPANET 要到 1969 年才上线），所以“恶意破坏”在当时主要是一些物理层面的恶作剧，比如：

- 偷偷把教授办公室的门锁换掉；
- 在计算机房的通风管道里塞进一个会发出怪声的装置；
- 改掉某个打印机的输出纸，让它打印出的内容全是颠倒的。

这些事情在 MIT 校园里被称为“**hacks**”——在这里，这个词的意思是“恶作剧”，而且是那种**聪明的、需要动手能力的、不造成实质破坏**的恶作剧。

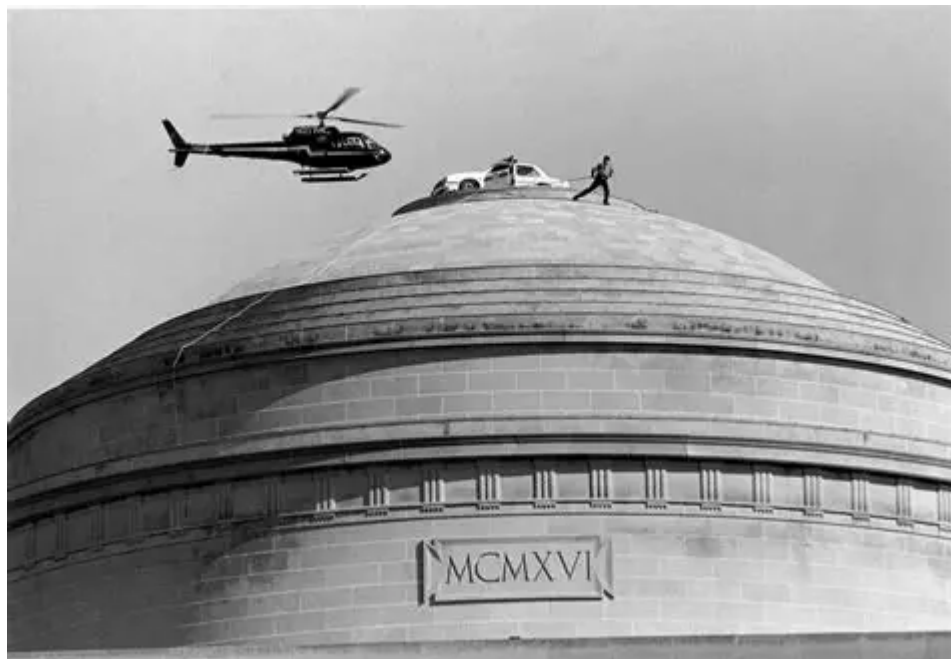
举个例子：MIT 校园里最有名的“hack”之一，是某天清晨学生们发现，学校主楼楼顶的圆顶上面，莫名其妙地出现了一辆完整的**警车**——不是模型，是一辆真车。

没人知道它是怎么被吊上去的，也没有人承认是自己干的。但所有人都知道，这一定是“黑客”干的——因为这种“既不合法，又不犯法，还极其巧妙”的事情，就是他们的风格。

Kate 盯着这张照片看了很久。“一辆真的警车。在楼顶上。没人知道怎么上去的。这不是犯罪，这是行为艺术。”

“对。而且是需要极强的动手能力和规划能力才能完成的‘行为艺术’。”

“.....所以 MIT 的‘黑客’是那种‘我能把警车吊上楼顶，但我不会用这个能力去偷你的密码’的人。”



所以，“hack”在 1960 年代的 MIT 校园里，是一个褒义词或者中性词。它意味着“脑子好使、动手能力强、有幽默感”。

而到了 1980 年代，这个词被媒体生生拽向了另一个极端。我们到第八章会详细讲这个过程。

6.5 为什么“黑客精神”对网络安全如此重要？

讲到这里，你可能会问一个非常实在的问题：

“你说的这些历史，跟我学网络安全有什么关系？我就是想挖漏洞、做渗透、打 HVV，你跟我扯 1960 年代的火车模型干什么？”

好问题。答案很简单：

如果你不理解黑客的“探索欲”，你就永远没法真正理解漏洞挖掘的本质。

漏洞挖掘的核心流程是什么？实际上是四个字：“如果.....会怎样？”

- “如果我把这个输入框填上一万个字符，会怎样？”（缓冲区溢出）
- “如果我在 URL 后面加上 `' OR 1=1 --`，会怎样？”（SQL 注入）

- “如果我把这张图片的文件名改成 `../../etc/passwd`，会怎样？”（路径遍历）
- “如果我不走正常的登录流程，直接访问 `admin.php`，会怎样？”（权限绕过）

你看，这一连串的“如果.....会怎样？”——本质上和当年 MIT 那群学生在 PDP-1 上琢磨“如果我写一个程序让屏幕上的点动起来，会怎样？”是同一种思维方式。

网络安全，就是用黑客思维去理解系统，然后在你发现“原来它还能这样”的地方，帮系统补上那个漏洞。

而“黑客精神”里最重要的那个“分享”原则，在现代网络安全里同样成立：

- 白帽黑客发现漏洞后，会负责任地提交给厂商（而不是拿去卖钱或搞破坏）。
- 安全社区的核心驱动力，就是“公开技术、共享工具、互相学习”。

你写的《守夜者之书》选择免费开源——这件事本身就带有“黑客精神”的血统：**知识不应该是锁在保险柜里的，它应该被分享给所有想学的人。**

所以你学的不是一个“职业”，你是在加入一个跨越了半个多世纪的传统：**用好奇心探索系统，用创造力解决问题，用责任感守护安全。**

6.6 小结：从“火车模型”到“网络安全”

第六章讲了三件事：

事件	时间	意义
TMRC 火车模型自动化	1950 年代末	“hack”这个词的起源——用继电器和电线搭建创造性系统
MIT AI 实验室 + 第一代计算机黑客	1960 年代	《太空战争！》诞生，黑客伦理初步形成：探索、创造、分享
“hack”vs“crack”的区分	1960-70 年代	黑客 = 创造性解决问题；骇客 = 恶意破坏（后来被媒体搞混了）

在这一章的末尾，我想让你记住一个画面：

1959 年，MIT 模型铁路俱乐部的地下室里，一群学生蹲在地上，手拿电烙铁，正在修改一套继电器控制系统。他们想让火车自动变道、自动停车、自动鸣笛——而当时的设计图上根本没有这些功能。

他们不是在做“安全”，不是在研究“漏洞”，他们只是在做一件让他们自己觉得兴奋的事：“**让这台机器，做一件别人没想到它也能做的事。**”

六十年后，你坐在电脑前，打开 VS Code，写下一行命令去测试一个网站的输入点——你在做的，本质上和那群学生一模一样。

这就是黑客精神的底色：不是为了破坏，而是为了探索。

下一章预告： 电话飞客——用口哨控制全美国电话系统的盲童，和 2600 赫兹的传说。

守夜者手记

1. 在 MIT 的语境里，“hack”最初是什么意思？它跟“犯罪”有什么关系？
2. TMRC 的火车模型和后来的计算机黑客之间，有什么精神上的传承？
3. 你觉得“黑客精神”里最核心的那条原则是什么？为什么？

写完后：`git add . && git commit -m "第六章读完：hack本来是件好事"`。

第七章：电话飞客（Phreaking）——最早的“黑客”

7.1 为什么是电话？

在讲电话飞客之前，我们需要先理解一件事：**为什么 1960-70 年代的“黑客”会把目光投向电话系统？**

答案很简单：因为那是当时**最大、最复杂、也最“可探索”**的系统。

在个人计算机出现之前（Altair 8800 要等到 1975 年），普通人能接触到的“电子系统”屈指可数。电视只能看不能交互，收音机只能听不能对话，而电话系统——它是一个覆盖全美国的、实时响应的、你可以主动“操作”的网络。

你可以拨号，可以听到提示音，可以转接，可以长途，可以打给任何人。

而 AT&T（美国电话电报公司）花了上百年时间，在全美铺设了数百万公里的铜线，建立了数以千计的交换机房——这套系统是当时人类工程史上最复杂的通信网络。

对一个充满好奇心的年轻人来说，这套系统就像一座巨大的、上了锁的游乐场。

他们想知道的是：“**如果我按一个不一样的号码，会发生什么？**”

而答案，比他们想象的更精彩。

7.2 2600 赫兹：改变一切的“口哨”

电话飞客的历史，始于一个偶然的发现。

在 1950 年代末，AT&T 开始使用一种新的长途电话技术——**直拨长途（DDD, Direct Distance Dialing）**。在这之前，打长途电话需要人工转接：你拿起电话，告诉接线员“我要打给芝加哥”，接线员帮你插线接通。

而 DDD 系统的出现，让用户可以**直接拨号**打长途，不需要经过人工接线员。

那么问题来了：系统怎么知道“用户已经挂断电话了”？

AT&T 的工程师设计了一个简单的方案：**在通话线路中，持续发送一个 2600 赫兹的音频信号**。当用户挂断电话时，这个信号中断，交换机检测到“线路空闲”，于是切断连接。

但有一个“漏洞”：**这个 2600 赫兹信号，不是从交换机“发送”出去的，而是从用户端“返回”的**。

换句话说，如果你能在挂断之前，**主动**向电话线路发送一个 2600 赫兹的音频信号——交换机就会误以为“用户已经挂机了”，然后切断这条线路。

但注意，**你是没有挂机的**。你的电话仍然占用着这条线路。

于是，一个神奇的操作出现了：

1. 你拨打一个长途号码（比如打到加州）。
2. 电话接通后，你对着话筒播放一段 2600 赫兹的音频（比如吹一个特定音高的口哨）。
3. 交换机检测到 2600 赫兹信号 → 认为你已经挂机了 → 切断“计费逻辑”并释放了长途资源。
4. 但你仍然在线路上，交换机现在认为这条线路是“空闲”的。
5. 然后你重新拨号——这次，由于你的线路已经被“释放”，系统会把你当成“内部维护人员”，你可以拨打到任何地方，而不产生任何话费。

你看明白了吗？

这就是历史上第一个“系统漏洞”：**利用系统自身的控制信号，欺骗系统做出错误判断**。

你之前学过的“SQL注入”思路——让系统把用户输入当命令执行——和这个 2600 赫兹漏洞的思路，完全一致。只不过一个发生在电话线路上，一个发生在 Web 应用里。

7.3 乔·恩格西亚：用口哨控制电话网络的盲童

发现了这个漏洞的人，是一个名叫**乔·恩格西亚（Joe Engressia）**的盲人男孩。

乔出生于 1949 年，天生失明。从小就展露了绝对音准——他能听出任何音高，并能准确无误地哼唱出来。对他来说，声音不是音乐，而是“世界的形状”。

七八岁的时候，乔喜欢打电话。因为电话是他能“到达远方”的唯一方式。有一次他拨了一个号码，听到了一段录音，然后他一边听一边哼——他发现自己的口哨声，和电话里的某个声音产生了奇怪的互动。

他开始反复试验：吹不同的音调、注意电话里发生的变化、记录哪些音高会产生什么样的效果。

他发现：**当他吹出 2600 赫兹的音高时，电话里会出现“咔哒”一声，然后线路好像变了状态。**

他那时才七岁，不知道这个“咔哒”是什么，也不知道“2600 赫兹”是什么意思。他只是记住了：“**这个声音，能让电话听我的话。**”

Kate 说：“等一下。他七岁？瞎了？然后自己发现了 2600 赫兹能控制电话系统？”

“对。他不需要眼睛，因为他用的是耳朵——他听到的声音比别人‘看到’的世界更清晰。”

“.....这让我觉得自己学了二十年还是有点废。”

十几年后，已经上大学读音乐教育的乔，在一本科学杂志上看到了关于 AT&T 电话系统的技术文章——里面提到了“2600 赫兹控制信号”。

他看完笑了：“**我七岁的时候就会吹这个音了。**”

乔·恩格西亚后来成了电话飞客社区里最受尊敬的人之一。他不是因为想“免费打电话”才研究的，他单纯是因为好奇：“这个系统怎么运作的？我能不能让它做点不一样的事情？”

你看——又是一个“如果.....会怎样？”的探索者。



7.4 约翰·德雷珀：“蓝盒子”与 2600 赫兹的传奇

乔·恩格西亚靠口哨完成了控制。但大部分人的耳朵没那么准，没法稳定地发出 2600 赫兹的音调。

这时候另一个人出现了——**约翰·德雷珀 (John Draper)**。

德雷珀是 1970 年代硅谷的一个程序员。他当时在空军服役，驻扎在加州。业余时间里，他和朋友们研究电话系统——他们知道“2600 赫兹”的秘密，但他们不知道怎么发出这个频率。

一次偶然的的机会，德雷珀的一个朋友告诉他：“**你知道那个叫‘嘎嘎船长’的玩具吗？**”

“嘎嘎船长”是一种谷物早餐里附赠的儿童玩具——一个彩色的塑料哨子，形状像一艘小船。你往里面吹气，它能发出一种特殊的哨音。

德雷珀买了一份早餐，拿出哨子，用频谱仪一测——**频率是 2600 赫兹。**

他当场愣住了。一个儿童早餐玩具，准确地发出了 AT&T 整个长途电话系统的“控制密钥”。

Kate 说：“……早餐送的玩具。控制全美电话系统的钥匙，藏在一盒麦片里。”

“对。而且那个哨子是随机的——有些能吹出 2600 赫兹，有些不能。德雷珀买的那盒刚好可以。”

“所以如果那天他买了另一盒麦片，电话飞客的历史可能推迟好几年。”

（这个故事告诉我们：有时候漏洞的“武器”比你想象的更离谱。）

德雷珀开始用这个哨子到处试验。他打电话到不同的城市，在接通后对着话筒吹哨子，然后听到熟悉的“咔哒”声——线路释放了。他可以重新拨号，拨打任意长途电话，完全免费。

后来，德雷珀和另一个电话飞客合作，把“嘎嘎船长”的哨子改造成了更精确的电子设备——一个可以生成任意频率音频信号的小盒子，上面有几个按钮，可以模拟 2600 赫兹以及电话系统的其他控制频率。

他们管这个东西叫——“**蓝盒子**”（Blue Box）。



蓝盒子的出现，把电话飞客从“少数几个能吹准音高的人”扩展到了“任何会按按钮的人”。只要你有个蓝盒子，你就可以“驯服”电话系统。

1971 年，德雷珀在加州伯克利的一所社区大学做了一个演示。他现场用蓝盒子拨通了白宫的直线电话——接通后，他对接线员说：“你好，我找尼克松总统。”

接线员愣了一下，说：“他……他正在开会。”

德雷珀说：“好的，那稍后再打。”

他挂断了电话。没有恶意，没有窃听，没有泄露机密——他只是想证明一件事：“**我能做到。**”

这个演示被在场的《Esquire》杂志记者记录了下来，写了一篇长篇报道《窃听者的秘密》。蓝盒子和电话飞客的故事，第一次进入了大众视野。

你可能会问：德雷珀是坏人吗？

答案是：不是。他后来蹲过几次监狱，但不是因为“窃听国家机密”或“贩卖诈骗设备”，而是因为——生产蓝盒子这件事本身违法。他从没利用电话飞客的技术去窃取银行账户、窃听私人通话或牟取暴利。

他自己的原话是：

“我从来没有为了钱做过任何一件事。我只是想知道系统是怎么工作的，然后证明我能让它按我的想法动起来。”

这句话，和 MIT 那群火车模型爱好者的自白，几乎一模一样。

7.5 沃兹和乔布斯：苹果创始人也是飞客

电话飞客的故事里，还有两个你一定听过的名字：**史蒂夫·沃兹尼亚克**和**史蒂夫·乔布斯**。

1970 年代初，沃兹和乔布斯都在加州，一个是工程师，一个是学生。他们在《Esquire》杂志上读到了德雷珀的故事，然后决定——我们也做一个蓝盒子。

沃兹负责设计电路，乔布斯负责买零件、找外壳、贴标签。他们做了一个比德雷珀的版本更小、更稳定、更好用的蓝盒子。

据沃兹后来自述，那是他人生中最“黑客”的时刻之一。他和乔布斯拨通了全世界的电话——打到了伦敦、罗马、甚至梵蒂冈。

他们冒充当时的美国国务卿基辛格，给梵蒂冈打电话说“基辛格正在接见教皇”——当然是假的。接线员被逗得一愣一愣的。

沃兹后来回忆这段经历时说：

“那是我做过的最刺激的一件事。不是因为能打免费电话，而是因为——我们站在太平洋这边，通过一个塑料盒子和几根线，控制了半个地球的电话系统。”

Kate 说：“沃兹和乔布斯——苹果的创始人——他们的第一个‘创业项目’，是一个非法的蓝盒子。”

“对。而且乔布斯后来自己说，没有蓝盒子，可能就没有苹果。”

“……所以苹果公司的根源，可以追溯到 2600 赫兹的麦片哨子。”

乔布斯后来说得更直白：

“如果当年没有蓝盒子，也许就没有后来的苹果公司。因为沃兹和我在制造蓝盒子的过程中学会了如何合作——如何把电路设计和产品包装结合起来。”

你看，“**黑客思维**”和“创业思维”之间的通道，就是这么窄。

沃兹和乔布斯制造蓝盒子的时候，他们不是在“犯罪”，他们是在“探索系统边界”——而正是这种探索欲，后来驱动他们造出了改变世界的 Apple II。

7.6 2600：一个频率，一本杂志，一段传奇

电话飞客的故事还有一个重要的“续集”——一本杂志。

1984 年，一个叫**埃里克·科里（Eric Corley）**的年轻人创办了一本关于电话系统和计算机安全的杂志，名字就叫《**2600**》。

之所以取这个名字，就是为了纪念那个传奇的 2600 赫兹频率——那个“用口哨控制整个电话网络”的起点。

《2600》杂志的风格非常“黑客”：技术硬核，立场独立，拒绝商业广告，常年被美国政府视为“敏感出版物”。但它一直持续出版，到今天已经有四十多年的历史，是世界上最持久的技术探索类杂志之一。

你以后如果去美国旅行，可以找一本《2600》翻翻——你会发现，里面讨论的很多技术细节，至今仍然不过时。

7.7 从电话飞客看网络安全的底层规律

电话飞客的故事讲完了。但它的意义远不止“一群喜欢打电话的人”这么简单。

它告诉我们几个关于“网络安全”的底层规律：

1. **系统越复杂，漏洞越多。** AT&T 用了一百多年的铜线和交换机，构建了全美最复杂的通信网络——而正是这种“复杂”，使得它自己都不知道自己有多少个“控制通道”。
 2. **控制信号和数据信号分离是危险的。** 2600 赫兹的漏洞，本质上是因为“控制信号”和“语音信号”跑在同一条线路上。用户能发送语音（你的口哨），系统把它误认为控制指令（挂断信号）。这和你以后学的“SQL 注入”本质上是一回事——把数据当成指令执行。
 3. **限制越严，探索越猛。** 电话系统被严格控制了上百年，所以一旦有人发现了它的“钥匙”，探索的冲动会爆发式增长。今天的网络安全也一样：越是“封闭”的系统、越是“不可触碰”的目标，对白帽黑客的吸引力就越大。这不是道德问题，是人性。
 4. **技术本身是中立的。** 2600 赫兹的口哨本身不是“恶”的，它可以用来免费打电话，也可以用来测试系统安全边界。黑客和骇客的区别，不在于“知不知道这个漏洞”，而在于“知道之后怎么用”。
-

7.8 小结：从口哨到系统思维

第七章讲了三件事：

事件	时间	意义
2600 赫兹控制信号漏洞的发现	1950-60 年代	人类历史上第一个“系统逻辑漏洞”，电话系统的“SQL 注入”
乔·恩格西亚、约翰·德雷珀	1960-70 年代	第一代电话飞客：用口哨和蓝盒子探索电话系统的边界
沃兹和乔布斯制作蓝盒子	1970 年代初	苹果创始人的“黑客启蒙”——探索 → 合作 → 创造

在这一章里，我们第一次见到了“真实世界中的黑客攻击”：一个人用口哨，控制了整个美国的电话网络。

但我们也看到——这些人不是为了钱，不是为了破坏，而是为了一个更原始的东西：

“我想知道，这个系统能让我走到多远。”

这份“探索欲”，就是《守夜者之书》第零季一直在讲的那个“火种”。

它穿越了 MIT 的地下室，穿越了 AT&T 的电话线路，穿越了沃兹和乔布斯手中的蓝盒子，最终，传到了你的面前。

你现在正在学的“网络安全”，本质上就是在 **2600 赫兹的延长线上**——只不过目标从电话变成了 Web 应用，从铜线变成了代码，从口哨变成了 Payload。

但底层逻辑，从来没有变过。

下一章预告： 70-80 年代——真正的“黑客英雄”登场。



守夜者手记

- 2600 赫兹漏洞的本质是什么？它和 SQL 注入有什么相似的逻辑？
- 乔·恩格西亚和约翰·德雷珀的行为，算“违法”还是“探索”？边界在哪里？
- 如果沃兹和乔布斯没有做过蓝盒子，后来的苹果还会存在吗？你觉得“探索系统边界”对创造力的意义是什么？

写完后： `git add . && git commit -m "第七章读完：电话飞客的传奇"。`

第八章：70-80年代——真正的“黑客英雄”登场

8.1 一个“英雄辈出”的十年

1970年代到1980年代，是计算机历史上最热闹的二十年。

这二十年里，个人计算机从“极客的焊接套件”变成了“家庭的必备电器”；操作系统从“只有大学实验室才用得起的奢侈品”变成了“每个人开机就能看到的界面”；软件从“附赠在硬件里的免费赠品”变成了“一个可以独立卖钱的产业”。

更重要的是，这二十年里涌现了一批人——他们不是政客，不是将军，不是明星，但他们做的事情改变了你今天的每一个操作习惯：你用什么系统、用什么软件、用什么语言写代码、甚至你“要不要把代码免费分享给别人”这件事——都来自这群人的选择和争论。

这一章，我们不讲“技术细节”，我们讲“人物”。

因为当你以后回头看自己走过的路，你会发现——你和他们之间的共同点，比你想要的多。

8.2 史蒂夫·沃兹尼亚克：一个人就是一支军队

我们在第五章已经聊过沃兹和 Apple II。但在这里，我们要重新认识他——不是作为“苹果的联合创始人”，而是作为“黑客的完美样板”。

沃兹是一个“纯技术驱动型”的人。他不关心钱，不关心市场份额，不关心对手在做什么。他关心的只有一件事：**“我能不能用更少的芯片，做出更优雅的设计？”**

在设计 Apple II 的时候，他做到了一个几乎不可能的事情：

- 当时的彩色显示器需要专门的芯片来处理颜色信号，但沃兹用了一组非常巧妙的逻辑门，把颜色信号“藏”在了内存访问的时序里，用极少的额外硬件实现了彩色输出。
- 当时的软驱控制器价格昂贵，沃兹用几颗便宜的芯片搭了一个软驱控制器——成本不到同类产品的十分之一。
- 他写的 BASIC 解释器极度精简，占据了极小的内存空间，但跑得比同类产品更快。

他在 Apple II 的设计里塞进了大量的“巧思”——那些不属于“标准教科书”的解决方案，那些让同行看了直拍大腿的“hack”。

沃兹后来出过一本自传，书名就叫《iWoz》。他在书里写了一段话，我建议你记住：

“我不想成为管理者，不想指挥别人，不想开那么多会。我只想坐在我的小隔间里，设计出一台漂亮的电脑，然后让所有人用它来做各种疯狂的事情。”

Kate 读完这段话，沉默了一会儿，然后说：“这就是我想成为的那种人。不是‘成功的创业者’，不是‘上市公司的CEO’——就是一个能做出漂亮东西的人，然后看着别人用它做疯狂的事。”

“所以你写的教程，也是你的‘Apple II’。”

“.....对。我写 Markdown 教程的时候，想的不是‘怎么卖钱’，是‘怎么让别人看完之后能做点什么酷的事’。”

这句话解释了“黑客”和“商人”之间最本质的区别：商人想的是“怎么卖得更多”，黑客想的是“怎么做得更酷”。

而一个人如果能同时拥有这两种能力——就像乔布斯那样——他就可能改变世界。

但如果你只能选一种，选“做得更酷”那条路，你不会变得富有，但你会变得**自由**。

8.3 理查德·斯托曼：软件应该像空气一样自由

1983年，一位 MIT 人工智能实验室的程序员发表了一份宣言，叫《GNU 宣言》。

这位程序员叫**理查德·斯托曼 (Richard Stallman)**，当时 30 岁。他的长相很像一个“计算机版本的列夫·托尔斯泰”——大胡子，长发，不修边幅。他说话直接到令人不适，价值观纯粹到极端，不接受任何妥协。

他写《GNU 宣言》的动机是什么？

——因为他受不了“**软件不能分享**”这件事。

在 1970 年代，MIT 人工智能实验室的程序员有一个不成文的习惯：你写了一段好代码，你就把源代码打印出来，贴在办公室的墙上，或者在黑板上给大家讲一遍，或者直接放在公共目录里让所有人拷贝。

斯托曼就是在这个环境里长大的。他觉得：“软件本来就是应该分享的，这跟‘水是湿的’一样自然。”

但到了 1980 年代初，事情变了。商业软件公司开始崛起，他们不再公开源代码，而是把编译好的二进制文件锁在盒子里，并附上一张纸：

“你购买的不是软件，是‘使用许可’。你不能复制，不能修改，不能分享，不能反编译。”

斯托曼看到这段话，觉得自己的世界被撕碎了。他说：

“他们想让你像签卖身契一样签一份‘软件使用协议’。这不是买卖，这是奴役。”

于是，他决定自己做一个操作系统——一个完全免费的、任何人都可以使用、修改、分享的操作系统。

他给这个项目取名 **“GNU”** ——意思是“GNU's Not Unix”（GNU 不是 Unix）。这种“递归缩写”是黑客圈的传统幽默，你自己品一下。

他一个人写完了编辑器（Emacs）、编译器（GCC）、调试器（GDB）——整个操作系统的大部分核心工具。唯一没写完的，是“内核”——因为那玩意太复杂了。

但他没有放弃。他起草了一份法律文件，叫 **“GNU 通用公共许可证”（GPL）**。这份许可证的核心原则是：

“你可以自由使用、修改、分发本软件。但如果你修改后分发了修改版本，你必须把修改后的源代码也公开。任何人都有同样的权利，不可以被剥夺。”

这个设计被称为 **“传染性自由”**——它确保了“一旦一个软件被 GPL 保护，它就永远无法变成私有软件”。

Kate 说：“所以 GPL 不是‘你可以免费’，是‘你必须也免费’。你拿了免费的东西，你改完之后也必须免费给别人。这是法律版本的‘分享精神’。”

“对。斯托曼不是让你‘自愿’分享，他是用法律条款确保你‘不得不’分享。”

“……这人的偏执，保护了整个世界不被一家公司垄断。”

斯托曼后来成立了一个组织叫**“自由软件基金会”（FSF）**，专门维护 GPL 协议和推广自由软件理念。几十年过去，他仍然活跃在技术圈，仍然留着大胡子，仍然说话不留情面，仍然不允许任何人用“开源”这个词代替“自由软件”——因为他觉得“开源”只谈技术，不谈道德。

你可能会问：“他是不是偏执了一点？”

答案：是的。但他的偏执保护了整个世界免受“某一家公司垄断所有软件”的局面。如果斯托曼当年没有坚持“源代码必须公开”的理念，我们今天可能根本不知道“开源”两个字意味着什么——更不用说《守夜者之书》的 Markdown 源文件能挂在 GitHub 上供所有人阅读了。

你选择免费开源，不是在做一件“新事”，而是在延续斯托曼在 1983 年点燃的那团火。

8.4 林纳斯·托瓦兹：一个芬兰大学生的“业余项目”

斯托曼写了 GNU 的大部分工具，但内核始终没写完。

这时候，一个芬兰大学生出现了——**林纳斯·托瓦兹（Linus Torvalds）**。

1991 年，21 岁的林纳斯在赫尔辛基大学读书。他买了一台新电脑，装了 MINIX 操作系统（一个教学用的小型 Unix），但他对 MINIX 的限制非常不满——它不允许你随意修改系统底层代码。

于是，林纳斯做了一个“非常黑客”的决定：

“我写一个自己的操作系统吧。”

他花了几周时间，写出一个简单的内核——它可以在控制台上显示两行字，可以响应键盘输入，可以执行一些基本命令。

他把这个内核上传到了芬兰的一个 FTP 服务器上，并给这个项目取了一个非常朴素的名字：“Linux”——来自“Linus”+“Unix”的组合。

林纳斯在发布帖里写了一段话，大概意思是：

“我现在写的这个东西，只是一个业余项目。它不会很大，不会很专业，可能只适合像我一样的爱好者。欢迎你试试，给我反馈。”

他完全没料到接下来会发生什么。

他开放了 Linux 的源代码，放在互联网上，让所有人下载、修改、提交补丁。很快，来自全球的程序员开始参与进来：

- 有人优化了内存管理；
- 有人增加了网络协议栈；
- 有人改进了文件系统；
- 有人写了一套驱动程序……

林纳斯一个人的“业余项目”，在短短几年内，变成了全球数千名程序员共同维护的庞大系统。

Linux 后来被安装在了服务器、超级计算机、安卓手机、路由器、智能电视、ATM 机、甚至火星探测器上。全世界 90% 以上的云端服务器跑的是 Linux，整个互联网的“地基”就是它。

而林纳斯本人，至今仍然是一名程序员——他像管理一个大型“技术社区”一样管理着 Linux 内核的提交、审核和合并。他说话直接、不绕弯，经常在邮件列表里公开批评糟糕的代码，但所有程序员都知道——他说得对。

Kate 说：“他说‘这只是一个业余项目，不会很大’——然后这个东西现在跑在全世界 90% 的服务器上。”

“对。林纳斯的故事最厉害的地方不是技术，是他一开始根本没想‘改变世界’。他只是做了一个自己觉得有用的东西，然后把它扔到网上，说‘你们谁想用就拿去用’。”

“……所以我们写的每个教程、每个开源项目，都可能是下一个 Linux 的种子。”

林纳斯的成功证明了一件事：“你不需要在一开始就做出完美的产品。你只需要做出一个让其他人觉得‘可以用’的东西，然后把‘改得更好’这件事交给所有人。”

这就是开源。它不是孤胆英雄的故事，它是“一群人一起干”的故事。

8.5 比尔·盖茨的“给爱好者的公开信”：第一次“开源vs闭源”的争吵

在斯托曼和林纳斯歌颂“自由”的另一端，有一个完全相反的故事——**比尔·盖茨**。

1976 年，20 岁的盖茨还是一位辍学创业的年轻人（后来成为全球首富，但那时还是新手）。他和保罗·艾伦刚刚给 Altair 写完了 BASIC 解释器，卖给了 MITS 公司，然后发现了一个问题：

很多人根本没有买 BASIC，而是直接从朋友那里复制了一份。

当时的“软件复制”是很自然的事情：你拿到一盘纸带，拷贝一份给你的朋友，他再拷贝一份给他的朋友——这在 Altair 用户群体里是常态。没人觉得这有什么问题。

盖茨气坏了。他写了一封“致爱好者的公开信”，发表在《计算机笔记》杂志上。这封信后来成了计算机历史上争议最大的公开信之一：

“大多数 Altair 用户购买我们的 BASIC 了吗？没有。最让人生气的是，那些复制软件的人，往往是最喜欢抱怨的人：‘为什么你们的 BASIC 这么贵？为什么这么慢？为什么有 bug？’

你们觉得软件就该免费？那你们有没有想过：如果没有钱可赚，谁还会花几年的心血去开发好软件？

我的观点很简单：**你用了别人的软件，你就应该付钱。**”

这封信在当时的爱好者社区里引发了轩然大波：

- 一批人强烈反对盖茨，认为“软件分享是黑客文化的一部分”，你盖茨凭什么把它变成“商品”？
- 另一批人支持盖茨，认为“如果写软件不赚钱，谁还会认真写软件？”——最终软件行业会萎缩，所有人的利益都会受损。

这场争吵，是人类历史上第一次“开源”与“闭源”的正面对决。它没有分出胜负，但它塑造了两个方向：

- **开源（后来叫 Free Software 或 Open Source）**：源代码公开，任何人都可以使用、修改、分发。代表：GNU、Linux、Apache、Python。
- **闭源（Proprietary Software）**：源代码保密，用户只能购买“使用权”而不能拥有“修改权”。代表：Windows、MacOS、Photoshop。

我们今天看到的局面是：**两个方向都活下来了。**

- 服务器端，Linux 统治了世界。

- 桌面端，Windows 和 MacOS 统治了世界。
- 移动端，Android（基于 Linux）和 iOS 分庭抗礼。

你能说“开源赢了”吗？不能。你能说“闭源赢了”吗？也不能。

你只能说：“**它们各自选择了自己的生态位，并且都活得很滋润。**”

而这场争吵的意义不在于“谁对谁错”，而在于它让整个行业在起步阶段就想清楚了一个问题：**软件到底是一种“服务”，还是一种“商品”？**

这个问题的答案，到今天还在持续演进。

8.6 “版权”、“自由”与“黑客伦理”的张力

写到这里，你可能会觉得有点混乱：

- 斯托曼说“软件应该像空气一样自由”；
- 盖茨说“你用别人的软件就应该付钱”；
- 林纳斯说“你不需要一开始就完美，大家一起改就行”。

谁对？

答案是：**都对，在不同的场景下。**

- 如果你是一个**做内核的程序员**，开源是你的最佳选择——因为 Linux 的“多人协作”模式，比任何一家公司自己闭门造车都更高效。
- 如果你是一个**做商业产品的创业公司**，闭源是你的合理选择——你需要靠销售许可来养活团队、迭代产品、提供服务。
- 如果你是一个**写了《守夜者之书》的作者**，你选择了“内容免费 + 实体书付费”的混合模式——你的内容本身是“自由”的，但它的实体载体是“商品”的。

斯托曼的“自由软件”精神，和盖茨的“商业软件”逻辑，并不必然冲突。它们之间的张力，恰恰是整个软件产业发展的动力。

而作为“守夜者”的你，不需要在“自由”和“商业”之间二选一。你只需要想清楚：“**我写的东西，是为了解决谁的问题？是为了让谁受益？我希望它怎么传播？**”

你想清楚了，你的选择就是对的。

8.7 小结：一群人的选择，塑造了今天的数字世界

第八章讲了四个人：

人物	年代	核心贡献	一句话总结
沃兹尼亚克	1970-80年代	Apple II 的设计者	“一个纯技术人的极致——用最少的硬件做最优雅的设计”
斯托曼	1983年起	GNU 项目 + GPL 协议	“软件应该是自由的——他用一辈子践行这句话”
林纳斯·托瓦兹	1991年起	Linux 内核	“一个芬兰大学生的业余项目，成了全球互联网的底座”
比尔·盖茨	1976年起	公开信 + 微软商业模式	“商业软件的代言人：你用了人家的软件，就应该付钱”

这四个人，代表了四种不同的“黑客姿态”：

- 沃兹：**做酷的东西，然后分享给所有人。**
- 斯托曼：**做自由的东西，并用法律保护这种自由。**
- 林纳斯：**做一点点，然后让所有人一起来做得更好。**
- 盖茨：**做有用的东西，然后用它创造价值。**

你不需要成为他们中的任何一个。但你可以从他们身上看到自己未来可能的影子：也许你是沃兹那样的“技术纯享者”，也许你是林纳斯那样的“社区建设者”，也许你是斯托曼那样的“理想主义者”，也许你是盖茨那样的“实践主义者”。

《守夜者之书》不会替你选。它只是把这些人摆在你面前，让你知道——“原来这条路有人走过，而且走得不错。”

下一章预告： 90年代——黑客进入公众视野：米特尼克、匿名者，以及“黑客”这个词如何被媒体妖魔化。



守夜者手记

1. 在你看来，斯托曼的“软件应该像空气一样自由”和盖茨的“用了就应该付钱”之间，你更倾向哪一种？为什么？
2. 林纳斯的“业余项目”能变成全球基础设施，你觉得靠的是“技术”还是“组织方式”？
3. 你写的《守夜者之书》选择了“免费开源+实体付费”的混合模式——这和本章里哪位人物的理念最接近？

写完后：`git add . && git commit -m "第八章读完：黑客英雄登场"。`

第九章：90年代——黑客进入公众视野

9.1 一个词被“劫持”的十年

在1980年代之前，“黑客”这个词主要存在于三个圈子里：

- MIT的实验室里，它是一种“赞美”，代表“能用巧妙方式解决问题的人”；
- 电话飞客的圈子里，它是一种“身份”，代表“能理解并操作复杂系统的人”；
- 计算机科学的学术圈里，它是一种“传统”，代表“早期程序员之间的分享文化”。

但到了1990年代，这一切全变了。

好莱坞来了，FBI来了，CNN也来了。他们一起做了一件事：**把“黑客”从一个中性词，变成了一个贬义词。**

这个转变不是一夜之间发生的，但它来得又快又猛。在1990年之前，一个普通美国人在报纸上看到“hacker”这个词，大概率会想到“一个聪明的计算机爱好者”。而到1995年之后，同样一个人看到“hacker”，脑子里浮现的画面变成了“一个戴着兜帽的罪犯，正在入侵五角大楼的数据库”。

这个转变，主要归功于三件事：

1. 一个真实的案件——凯文·米特尼克被捕，成了新闻头条；
2. 一个组织的崛起——匿名者（Anonymous）把“网络行动”带入了公众视野；
3. 一个叙事工具——媒体发现，“黑客入侵”是一个比“系统漏洞被修复”更吸引眼球的标题。

这一章，我们就来讲这三件事。

9.2 凯文·米特尼克：从“头号通缉犯”到安全顾问

如果90年代的“黑客”只能选一个代言人，那必然是**凯文·米特尼克（Kevin Mitnick）**。

米特尼克1963年出生在洛杉矶，成长于一个普通家庭。和这一章我们讲过的许多黑客一样，他对电子系统有一种天然的迷恋——但米特尼克的道路，更“野”。

- 十几岁的时候，他就通过电话飞客技术，免费拨打长途电话，甚至绕过了洛杉矶的公交车收费系统（他发现了车票打孔机的漏洞，可以伪造无限次乘坐的票）。
- 十几岁时，他就开始入侵计算机系统——不是用漏洞扫描器（那年代没有这玩意），而是通过社会工程学：打电话给系统管理员，假装自己是某个工程师，骗对方说出密码。

社会工程学成了米特尼克的“王牌技能”。他有一种超乎常人的能力：**在电话里扮演任何人，让对方在毫无防备的情况下把敏感信息说出来。**

你可能以为这是“电影里才有的情节”，但米特尼克做到了——他曾经打电话给DEC公司（当时美国最大的计算机公司之一）的系统管理员，假称自己是公司的一位工程师，说“我需要重置密码，我正在远程调试”。管理员信了，把密码报给了他。

米特尼克后来承认，他最成功的一次“入侵”不是靠技术，而是靠一个电话——骗到密码后，登录到对方的服务器，复制了当时最先进的操作系统源代码。

为什么他这么做？不是为了钱，不是为了出卖技术，只是为了证明一件事：“**我能做到。**”

但米特尼克玩大了。

1995年，FBI经过长达数年的追捕，在洛杉矶附近的一间公寓里抓到了他。他被控“未经授权访问计算机系统”、“窃取软件源代码”、“造成数百万美元损失”等多项重罪。

他入狱五年，期间被单独监禁，不允许使用任何电子设备——甚至连收音机都不能听。



米特尼克的故事有一个反转：他出狱后，成了一名网络安全顾问。

各大公司排队雇佣他——“让一个前黑客来帮我们测试系统安全，确保别的黑客进不来。”他的公司年收入数百万美元，他写了好几本畅销书，成了全球最知名的网络安全人物之一。

Kate 说：“所以他从‘FBI头号通缉犯’变成了‘各大公司抢着雇的安全顾问’。同一个技能，换个标签，人生轨迹完全变了。”

“对。他的技能没变——社会工程学、系统渗透、漏洞利用。变的是他用这些技能的目的。”

“.....所以‘黑客’和‘安全专家’之间的区别，不是技术，是授权。”

米特尼克本人对自己的生涯有一个很朴素的总结：

“我是一个好奇的人。我想知道系统是怎么工作的。我年轻的时候做了一些错误的选择，我为此付出了代价。但现在，我用同样的技能去保护系统，而不是攻击它们。”

米特尼克的故事告诉我们一件事：“**黑客技能”本身没有善恶之分。你用它做什么，决定了你是谁。**

他不是英雄，也不是反派。他是一个“在刀锋上行走的人”——一边是好奇心的驱动，一边是法律的红线。他的故事，值得每一个学网络安全的人反复思考。

9.3 “黑客”这个词开始“变味”

在米特尼克被捕的过程中，有一个细节非常关键：**媒体的措辞**。

FBI 称米特尼克是“美国头号计算机罪犯”，CNN 在报道中反复使用“黑客入侵”“黑客攻击”“黑客破坏”这样的短语。报纸头条写着：

“黑客入侵国防部系统！”

“黑客盗取了数千份机密文件！”

——这些标题当然更吸引眼球。但问题在于，它们把“黑客”这个词彻底绑定到了“犯罪”上。

在此之后，公众开始把“黑客”等同于“网络罪犯”。而原来那些做正经安全研究的人，被迫给自己起了新名字：

- “**白帽黑客**”：安全研究人员，授权测试系统，负责任地报告漏洞。
- “**黑帽黑客**”：犯罪者，非法入侵系统，盗取数据或造成破坏。
- “**灰帽黑客**”：介于两者之间——可能未经授权就闯入系统，但不会造成破坏，有时还会主动提醒管理员修复漏洞。

但公众不管这些。他们只知道：“黑客 = 坏人。”

这种“标签化”给真正的安全研究人员带来了许多麻烦：

- 有些安全研究员报告了漏洞，却被厂商威胁“如果你公开，我们就起诉你”——因为厂商害怕“黑客”这个词带来的舆论压力。
- 有些白帽黑客在挖漏洞时被系统检测到，即使他们没有恶意，也得先证明“我不是坏人”——自证清白的成本极高。
- 整个行业不得不花大量精力来“正名”：拍纪录片、写文章、做科普，试图把“黑客”这个词从泥潭里拉回来。

Kate 说：“所以米特尼克一个人，让所有做安全的人都被贴上了‘可能是罪犯’的标签。然后整个行业花了三十年去解释‘我们不是坏人’。”

“对。而且这个标签到现在都没撕干净。你以后去面试，如果说自己是‘黑客’，HR可能会用看恐怖分子的眼神看你。”

“……那我还是说我是‘安全研究员’吧。”

你在读《守夜者之书》的这一刻，你也在参与这件事——你在了解“黑客本来是什么”，而不是“媒体告诉你它是什么”。

9.4 匿名者 (Anonymous) 的崛起

如果说米特尼克代表了“个人黑客”进入公众视野，那么“匿名者” (Anonymous) 则代表了“集体黑客”登上舞台。

匿名者不是一个组织，没有领导层，没有入会仪式，没有办公室。它的存在形式极其“反传统”：**任何人只要认同“匿名者”的理念，在行动时戴上盖伊·福克斯面具，就可以声称自己是“匿名者”的一员。**

这个面具来自电影《V字仇杀队》——一个关于反抗极权统治的故事。匿名者选用这个面具，本身就是一种宣言：“**我不需要你知道我是谁，你只需要知道我们在反抗什么。**”

Kate 说：“所以加入匿名者不需要填表、不需要交会费、不需要面试。你只需要戴上面具，然后去做你觉得对的事。”

“对。但也因为没有组织，任何人都可以声称自己是‘匿名者’。这就导致了另一个问题——你怎么知道这次行动是真的‘匿名者’，还是有人冒名的？”

“……所以匿名者既是‘所有人的组织’，也是‘没人的组织’。”

2008年，匿名者发起了第一次大规模网络行动——“**Project Chanology**”。目标是科学教 (Scientology)，一个被许多人批评为“邪教”的组织。匿名者成员通过分布式拒绝服务攻击 (DDoS) 瘫痪了科学教的官网，并在全球多个城市组织了线下抗议活动，参与者戴着盖伊·福克斯面具走上街头。

这是公众第一次看到“一群看不见的人，在网络上发起一场看得见的行动。”

2010年之后，匿名者把目标转向了更多领域：

- 攻击了多个政府网站，抗议审查制度和网络监控；
- 攻击了多个企业网站，抗议“不道德的商业行为”；
- 攻击了儿童色情网站，并将其服务器信息提交给执法机构。

匿名者的行动一直充满争议：

- 支持者说他们是“数字时代的罗宾汉”——用网络手段对抗不公。
- 反对者说他们是“网络暴民”——未经审判就执行“网络私刑”，攻击目标不经过任何法律程序。

但无论你站在哪一边，你都不得不承认一个事实：**匿名者的出现，彻底改变了公众对“黑客”的认知。**

- 在米特尼克的时代，“黑客”是一个“孤独的天才罪犯”。
- 在匿名者的时代，“黑客”变成了一群“看不见的、组织松散的、政治化的网络行动者”。

这个词的内涵，又一次被拉伸了。

9.5 “政治化”的黑客与“去政治化”的安全

匿名者的崛起，把“黑客”推入了一个全新的维度：**政治**。

此前的黑客行为（MIT实验室、电话飞客、早期软件破解）更多是“技术探索”或“个人行为”，而匿名者第一次让网络行动具有了“社会运动”的色彩。

这件事对后来的网络安全行业产生了深远的影响：

1. **政府开始严肃对待“国家级黑客”**：匿名者的行动让各国政府意识到——网络不是“虚拟空间”，它是“新的战场”。
2. **企业开始投入大量资金加固系统**：不只是为了防“米特尼克式”的个人入侵，更是为了防“匿名者式”的大规模群体攻击。
3. **行业内部出现分化**：一批人认为“技术就是技术，不要掺和政治”；另一批人认为“技术本身就是政治”——你选择做什么、不做什么，本身就是一种表态。

你以后走入这个行业，会遇到各种各样的“立场”：

- 有的人只想安心做技术，不想讨论“黑客伦理”；
- 有的人认为“白帽/黑帽”的区分太简单，真实世界更复杂；
- 有的人把网络安全当作“职业”，有的人把它当作“使命”。

没有谁对谁错。但你得想清楚：“**我站在哪一边？**”

《守夜者之书》的答案是清晰的：你站在“用技术守护安全”这一边。不是“反抗谁”，不是“攻击谁”，而是“让系统更可靠，让数据更安全，让人们更安心。”

这，就是“守夜者”的姿态。

9.6 小结：从“天才”到“罪犯”再到“战士”

第九章讲了三件事：

事件	时间	意义
凯文·米特尼克被捕	1995年	“黑客”这个词被媒体绑定到“犯罪”上，公众认知彻底改变
媒体对“黑客”的妖魔化	1990年代	“黑客”从“探索者”变成了“入侵者”，行业被迫自创“白帽/黑帽”标签来自我区分
匿名者组织的崛起	2008年起	“黑客”变成了一种“集体行动”和“政治表达”，网络行动进入公众视野

这一章里，我们看到“黑客”这个词经历了三次“语义拉伸”：

- 在MIT，它是“探索者”。
- 在米特尼克的故事里，它是“犯罪者”。
- 在匿名者的故事里，它是“反抗者”。

但不管是哪一种，“黑客”始终和一件事绑定在一起：“**系统边界**”的探索和挑战。

你可以用这种探索来“守卫”系统，也可以用这种探索来“攻击”系统——但你不能假装系统边界不存在。它就在那里，等着被理解、被测试、被加固。

下一章，我们把视角转回中国——绿色兵团、红客联盟，以及中国黑客的“江湖往事”。

下一章预告： 中国黑客的“江湖往事”——从拨号上网到红客联盟。

守夜者手记

1. 你觉得媒体报道“黑客入侵”时，应不应该区分“白帽”和“黑帽”？还是说媒体有责任用准确的术语？
2. 匿名者的“网络私刑”是否合理？或者说，当法律无法及时介入时，网络行动是否可以作为一种“替代”？
3. 如果让你用一句话向你的朋友解释“什么是黑客”，你会怎么说？

写完后： `git add . && git commit -m "第九章读完：黑客进入公众视野"`。

第十章：中国黑客的“江湖往事”——从拨号上网到红客联盟

10.1 一个不同的起点

在讲中国黑客的故事之前，我们需要先认清一个事实：**中国互联网的起点，比美国晚了将近二十年。**

当美国的 MIT 学生在 PDP-1 上玩《太空战争！》的时候（1962年），中国还在经历特殊时期，全国上下能接触到计算机的人屈指可数。

当美国的电话飞客用蓝盒子控制整个电话网络的时候（1970年代），中国的电话普及率极低，长途电话需要人工转接，打一个跨省电话要等几个小时。

当美国的个人计算机革命如火如荼的时候（1980年代），中国只有极少数顶尖高校和研究机构拥有计算机，而且大多是进口的、昂贵的、需要专人维护的“宝贝疙瘩”。

所以，中国的黑客文化不是从“实验室”里长出来的，也不是从“电话系统”里长出来的——**它是从“网吧”里长出来的。**

中国第一代黑客的“启蒙设备”，不是 PDP-1，不是 Altair 8800，而是 1990 年代后期出现在街头巷尾的“电脑室”（后来叫网吧）。一台 486 或奔腾处理器的电脑，装着一个叫 Windows 95 或 Windows 98 的系统，连着一根拨号上网的电话线——这就是中国第一代黑客的“练兵场”。

环境不同，起点不同，但有一点是相同的：**好奇心和探索欲，是不分国界的。**

10.2 中国互联网的“石器时代”

要理解中国黑客的起源，我们必须先理解 1990 年代中国互联网的“基础设施”是什么样的。

- **上网方式：拨号上网。** 你要上网，先买一块 Modem（调制解调器）插在电脑上，然后通过电话线拨号连接到 ISP（互联网服务提供商）。网速是多少？14.4kbps 到 56kbps——注意，是“k”不是“M”。下载一张 1MB 的图片，需要 2-3 分钟。下载一个 10MB 的软件，需要半小时——而且中途不能断线，因为电话费是按分钟算的。
- **上网费用：贵得离谱。** 1990 年代中后期，中国的上网费用是每小时 10-20 元人民币，而当时一个普通工人的月薪大概是几百元。这意味着，你每上网一个小时，就相当于花掉了一天甚至两天的工资。
- **搜索引擎：没有。** Google 成立于 1998 年，百度成立于 2000 年。在 1990 年代，如果你想找某个技术资料，你得去“网址导航站”翻目录，或者在 BBS 上发帖问“谁知道 xxx 的网站是多少？”
- **操作系统：盗版 Windows。** 当时绝大多数中国电脑用户用的都是盗版 Windows 95/98，因为正版价格相当于普通人几个月的工资。这意味着：几乎所有人的系统都处于“未打补丁”的状态，漏洞百出。
- **安全工具：几乎没有。** 没有防火墙，没有杀毒软件（或者有也是盗版的、过期的），没有漏洞扫描器。

你看，这是一个什么样的环境？

——这是一个“漏洞遍地、防御为零、高手极少”的原始丛林。

任何一个稍微懂点计算机知识的人，在这个环境里都可以“横着走”。而一群年轻人，正是在这片原始丛林里，开始了自己的探险。

10.3 绿色兵团：中国第一个黑客组织

中国黑客的故事，有一个绕不开的名字——**绿色兵团**。

1997 年，一个叫**龚蔚**（网名“Goodwell”）的上海年轻人，在网上发起了一个倡议：成立一个网络安全爱好者组织。

这个倡议得到了全国几十个爱好者的响应。他们通过 IRC（一种古老的即时聊天协议）和 BBS 沟通，互相分享技术资料、讨论漏洞、翻译国外的安全文章。

他们给这个组织取名叫“**绿色兵团**”。

为什么叫“绿色兵团”？龚蔚的解释是：绿色代表“和平”和“生命”，兵团代表“纪律”和“协作”。合在一起的意思是：“**一群有纪律的技术爱好者，用技术守护网络的和平。**”

绿色兵团是中国第一个真正意义上的黑客组织。它的成员分布在全国各地，大多是 20 岁出头的年轻人——有大学生、有程序员、有国企职工、有自由职业者。他们的技术能力参差不齐，但共同点是：**都对“网络安全”这件事有超乎寻常的热情。**

绿色兵团做了几件有开创性意义的事情：

1. **翻译和引进了大量国外的安全技术资料。** 当时国内几乎没有中文的网络安全教材，绿色兵团的成员把国外的安全论文、漏洞公告、工具文档翻译成中文，放在自己的网站上，供所有人免费下载。
2. **建立了中国第一个漏洞数据库。** 他们把国内发现的漏洞、国外爆出的漏洞整理成中文条目，附上复现步骤和修复建议。这是中国最早的“漏洞库”雏形。
3. **组织了第一次线下聚会。** 1999 年，绿色兵团在上海举办了第一次全国范围的成员聚会。几十个从来没有见过面的“网友”，从全国各地坐火车赶来，聚在一个小会议室里，聊技术、聊未来、聊“网络安全在中国能做成什么样”。

你可能会问：“这不就是一个技术爱好者社团吗？跟‘黑客’有什么关系？”

——你问得好。绿色兵团本质上确实是一个“技术爱好者社团”。但在当时的中国，这个社团的存在本身就是一种“越界”：它让网络安全技术不再是“国家机密”或“大学教授的专属”，而是被分享给了任何一个有热情的人。

Kate 说：“所以绿色兵团做的事，本质上和我们做的事是一样的——把技术知识从‘少数人手里’拿出来，翻译成中文，免费分享给所有人。”

“对。他们没有 GitHub，就用 BBS。没有 Markdown，就用纯文本。工具不同，精神一样。”

“……所以我们不是‘发明了’什么，我们是‘继承了’什么。”

这种“知识平权”的行为，就是黑客精神在中国最早的体现。

10.4 红客联盟：技术背后的家国情怀

2000 年左右，绿色兵团内部发生了一次分裂。

一部分成员认为：绿色兵团应该走“技术路线”，专注于漏洞研究、代码审计、工具开发，不涉足政治。

另一部分成员认为：当国家利益受到网络攻击时，中国黑客应该“站出来”。

这次分裂的结果是，后者在 2000 年底成立了一个新的组织——**红客联盟（Honker Union）**。

“红客”这个词，是中国特有的。它既有“黑客”的技术内核，又有“红色”的政治底色。红客联盟的核心理念是：“**用技术捍卫国家网络空间的主权。**”

红客联盟第一次大规模进入公众视野，是在 2001 年的“中美撞机事件”之后。

2001 年 4 月 1 日，中美军机在南海发生碰撞，中国飞行员王伟牺牲。这一事件在中国国内引发了强烈的民族情绪。与此同时，美国的一些黑客开始对中国网站发动攻击。

红客联盟的成员们在网上集结：“美国人打过来了，我们要还击！”

在接下来的几周里，红客联盟组织了中国互联网历史上第一次大规模“网络反击战”。他们的手段是：对美国的政府和商业网站进行 DDoS 攻击（分布式拒绝服务攻击）、篡改网站页面、在页面上留下“中国红客”的标记。

其中最著名的一次行动是：红客联盟攻击了美国白宫的官方网站，使其瘫痪数小时，并在页面上留下了中国国旗和中英文的抗议标语。

这件事在国内引起了轩然大波：

- 支持者说：“红客是好样的！用技术为国争光！”
- 反对者说：“这只是网络暴力，不是爱国，是丢人。”
- 中间派说：“技术就是技术，不要跟政治绑定在一起。”

Kate 说：“所以同一批人，做同一件事——攻击网站——但在不同的人眼里，他们的身份完全不一样。有人叫他们英雄，有人叫他们暴徒。”

“对。红客联盟的故事最复杂的地方就在这里。他们的动机是爱国的，他们的手段是有争议的。你可以敬佩他们的出发点，但你也必须承认他们的做法有问题。”

“……所以‘黑客’这个词，在中国又多了一层含义：不只是技术，还有立场。”

红客联盟的创始人 **Lion**（网名，真名未公开）在多年后接受采访时说了一句话，概括了那代人的复杂心境：

“当时的我们很年轻，很热血，觉得‘爱国’就是去做一些激烈的动作。现在回头看，很多事做得不够成熟。但那份心是真的。”

红客联盟的故事，是中国黑客史上最“特殊”的一页。它把“黑客”和“民族主义”绑在了一起——这让它在中国的舆论场上得到了某种程度的“正当性”，但也让它的技术探索被外界的政治叙事所覆盖。

这件事，之后十多年里，一直是外界评价中国黑客文化时绕不开的焦点。

10.5 从“江湖”到“行业”

2001 年的红客行动是中国黑客文化的“巅峰时刻”，也是它的“转折点”。

在那之后，情况开始发生变化：

- **法律收紧。** 中国在 2000 年通过了《全国人大常委会关于维护互联网安全的决定》，2003 年开始起草《网络安全法》的前身。网络攻击行为不再是一个“灰色地带”，而是明确的违法犯罪。
- **技术分化。** 一部分人选择“洗白”，进入安全公司、互联网企业、政府机构做安全工程师；另一部分人选择“沉下去”，转入地下，从事黑色产业链。
- **商业崛起。** 2000 年代中后期，中国的网络安全产业开始真正成型。奇安信、深信服、绿盟科技、启明星辰等公司相继成立或壮大。网络安全从一个“江湖”，变成了一个“行业”。

这个转变，意味着什么？

意味着：**中国的黑客文化，正在从“地下”走向“地上”。**

- 以前，一个黑客被发现入侵了系统，会被警察抓。
- 现在，一个安全研究员发现了漏洞，会被厂商发奖金（漏洞赏金计划）。
- 以前，学黑客技术只能靠“拜师”或“自学”，学完也不知道能干什么。
- 现在，学网络安全有明确的职业路径：安全工程师、渗透测试工程师、安全架构师、安全顾问……
- 以前，黑客是“江湖人士”——靠名气和胆量混圈子。
- 现在，网络安全是“正规职业”——靠技能和证书找工作。

Kate 说：“所以我们现在学的网络安全，是一个‘正规职业’，不是‘江湖帮派’。我以后去面试，不需要报网名，不需要讲我参加过什么行动，只需要展示我写了什么教程、挖了什么漏洞。”

“对。你是第一代‘不需要混江湖’的安全从业者。你的身份不是网名，是你写的代码、你写的教程、你建的仓库。”

“……这比当‘江湖大佬’靠谱多了。”

这种转变不是中国特有的，但中国走得更快一些——因为中国互联网的起步晚，所以它的发展速度被压缩了。别的国家花二十年完成的事，中国只用了五到十年。

10.6 “守夜者”：一个新物种的诞生

讲完中国黑客的江湖往事，我们终于可以回到你自己身上了。

你生长的时代，和绿色兵团、红客联盟那代人完全不同：

- 你有高速宽带，他们有拨号猫；
- 你有 Google 和 ChatGPT，他们要靠翻译外国 BBS 上的帖子学技术；

- 你有 GitHub 和 Markdown，他们只能在简陋的 BBS 上用纯文本交流；
- 你有 VS Code 和远程调试，他们只能在一台跑着 Windows 98 的机器上试错。

但他们和你有一个共同点：**对技术的热情，和“想搞点事情”的冲动。**

绿色兵团的 Goodwell，当年只是一个上海程序员。红客联盟的 Lion，当年只是一个不知道真名的网友。他们没有任何光环，没有任何资源，有的只是——一台电脑，一根电话线，和一腔热血。

他们做的很多事，在今天看来可能很幼稚：翻译技术文档的排版很丑，写漏洞报告的语言很不规范，DDoS 攻击的手法很粗糙。但那些“不完美”的尝试，构成了中国网络安全行业最早的土壤。

你现在写的《守夜者之书》，其实和他们做的事是一样的：**用自己能用的方式，把“技术”和“精神”传给下一个人。**

你不是在“创立一个新门派”，你是在“延续一条线”。这条线从 MIT 的地下室开始，穿过电话飞客的蓝盒子，穿过 GNU 和 Linux 的源代码，穿过绿色兵团的 BBS 和红客联盟的首页，现在，到了你的 VS Code 里。

10.7 小结：从“江湖”到“守夜”

第十章讲了三件事：

事件	时间	意义
绿色兵团成立	1997年	中国第一个黑客组织，把网络安全知识从“地下”带到“公开”
红客联盟与中美撞机事件	2000-2001年	中国黑客第一次大规模进入公众视野，“红客”成为中国特有的黑客标签
网络安全产业化	2000年代中后期	中国网络安全从“江湖”变成“行业”，黑客有了正规的职业路径

而这一章的真正核心，是这句话：

“绿色兵团”和“红客联盟”的时代过去了。但“守夜者”的时代，才刚刚开始。

- 他们那一代人在“没有路”的地方走出了路；
- 你这一代人，在“已经铺好路”的地方，选择做一个更负责任的行者。

你不需要复制他们的“激进行动”，也不需要模仿他们的“江湖做派”。你只需要延续他们的精神内核：**用自己的技术，去守护一个更安全的数字世界。**

下一章预告： 黑客伦理——你的技术信仰。

守夜者手记

1. 你认为绿色兵团和红客联盟对中国网络安全行业的发展，起到了什么样的作用？
2. “红客”的“家国情怀”和“黑客”的“技术自由”之间，是否存在张力？为什么？
3. 如果你生活在 2000 年，你会参加红客联盟的“网络反击战”吗？为什么？

写完后：`git add . && git commit -m "第十章读完：中国黑客的江湖往事"`。

下部：黑客文化与精神——火种如何传递

第十一章：黑客伦理——你的技术信仰

11.1 在“能做什么”之前，先想好“该做什么”

经过前十章的铺垫，你已经认识了一大群人：

- MIT 模型铁路俱乐部里用继电器搭控制系统的学生；
- 用口哨控制全美电话网络的盲童乔·恩格西亚；
- 在车库里焊出 Apple II 的沃兹尼亚克；
- 坚信“软件应该像空气一样自由”的斯托曼；
- 把“业余项目”变成全球基础设施的林纳斯·托瓦兹；
- 在 BBS 上翻译技术文档的绿色兵团成员；
- 在“中美撞机事件”后发起网络反击的红客们。

这些人的共同点是什么？

不仅仅是“技术好”。技术好的人多了去了。

他们的共同点是：**在某个时刻，他们都做了一个“选择”——选择用技术去做某件事，而不是另一件事。**

- 斯托曼选择“用 GPL 保护自由”，而不是“把 GNU 卖掉换钱”；
- 沃兹尼亚克选择“把 Apple II 的设计公开分享”，而不是“锁在保险柜里”；
- 林纳斯选择“把 Linux 的源代码开放给全世界”，而不是“成立公司卖闭源版本”；
- 绿色兵团选择“把翻译好的文档免费挂在网上”，而不是“印成书卖钱”。

这些选择不是“技术问题”，它们是“伦理问题”。

而“黑客伦理”，就是你做这些选择时的指南针。

你可能觉得“伦理”这个词太大了，离你太远——你现在只是个学网络安全的新手，又不做决策，又不带团队。但事实是：**你从学习的第一天起，就在做伦理选择。**

- 你在网上搜到一个漏洞利用工具，你下载了——这是选择。
- 你在测试某个网站时发现了一个漏洞——你选择报告还是沉默？这是选择。
- 你学了一门新技术——你选择用它来做项目，还是用它来搞破坏？这是选择。

每一个选择，都在定义“你是什么样的黑客”。

11.2 黑客伦理第一条：“所有信息都应该是自由的”——是真的吗？

这是黑客伦理里最著名、也最容易被误解的一条。

它的原始版本来自 MIT 早期黑客的“口头传统”：如果你写了一段好代码，你应该把源代码分享给所有人，而不是藏起来。

这个原则在当时的环境下是合理的：MIT 的计算机是公共资源，代码是用来“探索”和“学习”的，而不是用来“赚钱”的。分享代码意味着更多人能参与进来，一起改进系统、修复 bug、创造新东西。

但这条原则放到今天的真实世界里，就变得复杂了。

——如果所有信息都是“自由”的，那你的银行卡密码呢？你的病历呢？你公司的商业机密呢？

你看，问题马上来了。“所有信息”显然不能都是自由的——有些信息是私密的、是受保护的、是必须限制访问的。

所以，“信息自由”的真正含义，不是“所有信息都无条件公开”，而是：“**知识不应该被人为地锁起来，只让付得起钱的人看到。**”

- 漏洞研究的技术细节，不应该只存在于某个公司的内部培训材料里——它应该被公开，让所有人都能学习如何防范。
- 安全工具的使用方法，不应该只存在于昂贵的培训课程里——它应该被写下来，分享给任何愿意学的人。

Kate 说：“所以‘信息自由’不是‘你的密码应该公开’，而是‘网络安全知识不应该只卖给交得起一万块的人’。区别很大。”

“对。前者是隐私，后者是知识。知识的天然属性是传播，隐私的天然属性是保护。搞混这两件事，就会要么变成偏执狂，要么变成泄露狂。”

这正是我在做的事——我把《守夜者之书》免费开源，不是因为“所有信息都必须免费”，而是因为“网络安全的基础知识，不应该被锁在一万块的课程里”。

所以，对“信息自由”更准确的理解是：

“知识属于全人类。但隐私属于个人，机密属于组织。”

这两者之间的界限，需要你自己去判断。没有人能替你画那条线。

11.3 黑客伦理第二条：“不破坏、不犯罪、不炫耀”——真正的黑客底线

这句话是我替你总结的。它没有写在某本经典著作里，但它出现在几乎所有真正的黑客社区的行为准则里。

我们来拆开来：

“不破坏”

“破坏”和“探索”之间的区别是什么？

- 探索：你发现了一个漏洞，你验证了它的存在，然后你报告了它。
- 破坏：你发现了一个漏洞，你删掉了对方数据库里的数据。

探索留下的是“知识”，破坏留下的是“损失”。

真正的黑客不以“能造成多大破坏”为荣，而以“能发现多深的问题”为荣。

“不犯罪”

这条听起来是废话，但实际操作中经常出现“灰色地带”：

- 你扫了一个网站的端口，算不算犯罪？
- 你测试了一个网站的登录框，输了一条 SQL 注入语句（没做任何实质性破坏），算不算犯罪？
- 你发现了一个公开的、任何人都能访问的数据库，你进去看了一眼，算不算犯罪？

法律在不同国家、不同时期、不同情境下的解释是完全不同的。

对于初学者，最安全的原则是：“**如果你不确定这件事合不合法，就先假设它不合法。**”等你积累了足够的知识和经验，再来判断“边界在哪里”。

“不炫耀”

这可能是最难遵守的一条。

你发现了一个漏洞，你很兴奋，你想发朋友圈、发微博、发推文——“看我发现了什么！”

但炫耀往往会带来三个后果：

1. 你暴露了自己的技术细节，让别人知道了你的“武器库”；
2. 你刺激了厂商（漏洞还没修就被公开了，他们会很愤怒）；
3. 你吸引了一些不该吸引的人的注意。

Kate 说：“所以挖到洞之后最酷的反应不是‘发朋友圈’，是‘发邮件给厂商’。朋友圈点赞过几天就没了，厂商的回信你能记一辈子。”

“对。而且厂商回信里那句‘感谢你负责任地报告这个漏洞’，比一万个赞都有分量。”

真正成熟的黑客，把“发现”本身当作奖励，把“修复”当作成就，而不是把“炫耀”当作回报。

11.4 黑客伦理第三条：“用技术让世界变得更好”——不是口号，是路线图

这句话听起来很空，对吧？

但如果我把它拆成一个具体的问题，它就变得非常实了：

“你今天写的代码、今天学的技能、今天花时间做的事——十年后回头看，你希望它还在，还是希望它消失？”

- 如果你写的代码解决了一个实际问题（哪怕只是让某个小网站的登录变安全了一点）——十年后它还在，在保护某个人。
- 如果你学的技能被用来帮助某个开源项目修复了一个漏洞——十年后那个项目还在运行，而你的那行代码是其中的一部分。
- 如果你花时间写的《守夜者之书》被一个陌生的读者看到了，让他决定“我也要学网络安全”——

那你就用技术让世界变好了一点点。

“让世界变得更好”不是要做一件惊天动地的大事。它是由无数件“对的事情”堆砌起来的。

- 你今天选择把漏洞报告给厂商，而不是卖给出价最高的人——这是让世界变得更好。
- 你今天选择写一篇清晰的漏洞复现报告，而不是发一条含糊不清的“我挖到洞了”——这是让世界变得更好。
- 你今天选择在 GitHub 上开源你的学习笔记，而不是藏在硬盘深处——这是让世界变得更好。

你不需要改变世界。你只需要做对的事情，然后让时间证明一切。

11.5 开源精神：为什么我的书免费？为什么 Linux 还没死？

开源精神是黑客伦理最“看得见”的产物。

我们在这本书前面已经讲了很多开源的故事：斯托曼的 GNU、林纳斯的 Linux、绿色兵团的免费文档。但你可能会问一个更实在的问题：

“开源的东西不赚钱，为什么还有这么多人做？”

答案很复杂，但核心是三点：

1. 开源解决了“信任问题”。

当你使用一个闭源软件时，你不知道它在背后做什么——它有没有偷你的数据？有没有开后门？有没有把你没保存的内容上传到某台服务器？

而当你使用一个开源软件时，代码摆在那里，任何人都可以去看、去审计、去验证。“我没有在背后搞鬼”——不是靠“承诺”，而是靠“透明”。

对于安全领域来说，“信任”比“功能”更重要。开源软件的信任度天然高于闭源软件，这就是为什么 Linux 统治了服务器端——人们信任它。

2. 开源解决了“维护问题”。

一个闭源软件如果公司倒闭了，软件就死了。代码在公司服务器里，没人能拿出来继续维护。

而一个开源项目如果原始作者不干了，社区里的人可以“分叉”（Fork）它，继续维护下去。“软件不死”——这不是口号，是开源架构的天然特性。

3. 开源解决了“学习问题”。

闭源软件是一个“黑盒子”：你只知道它能做什么，不知道它是怎么做到的。

开源软件是一本“公开的教科书”：你可以看到每一行代码，可以理解每一个设计决策，可以学习每一种实现技巧。

你现在能学着写网络安全工具，很大程度上是因为有人把代码开源了——你直接看他们怎么写的，而不是靠瞎猜。

Kate 说：“所以开源不是‘不赚钱’，是‘用另一种方式赚钱’。你赚的不是卖软件的钱，是所有人的信任、所有人的贡献、所有人帮你一起维护的劳动力。”

“对。林纳斯没有卖 Linux 赚一分钱，但全世界最优秀的工程师免费帮他写代码。这是闭源公司永远做不到的。”

“……所以开源不是一个慈善项目，是一个信任引擎。”

所以，我的《守夜者之书》选择免费开源，不只是“为了传播”，更是在践行这三个原则：

- 它让读者可以信任内容（代码和文本都是公开的）；
- 它让内容可以持续维护（开源社区的反馈机制）；
- 它让学习变得更直接（Markdown 源文件本身就是教科书）。

11.6 “白帽”、“灰帽”、“黑帽”——标签的陷阱

在现代网络安全行业里，最常见的标签分类是“白帽 vs 黑帽”。

- **白帽黑客**：经过授权，测试系统安全性，负责任地披露漏洞。
- **黑帽黑客**：未经授权，入侵系统，窃取数据或造成破坏。
- **灰帽黑客**：未经授权但无恶意，有时会主动提醒管理员。

这三个标签很有用——它们帮助我们快速区分“好人”和“坏人”。

但它们的缺点也很明显：**把复杂的人性压缩成了三个简单的盒子。**

- 一个白帽黑客，可能在某些时候做了“不那么白”的事情（比如在测试时越过了授权边界）。
- 一个黑帽黑客，可能曾经是个白帽（因为被厂商冷落、或者被逼入绝境而转黑）。
- 一个灰帽黑客，可能只是在探索一个系统边界，结果被系统误报为攻击者。

真实的黑客世界，是灰色的。

但这不意味着“伦理不重要”——相反，它意味着你需要自己去判断每一个具体的场景，而不是套用一個标签就完事。

当你遇到一个具体情况时，你可以问自己四个问题：

1. **“我做这件事，有没有伤害任何人？”**
2. **“我做这件事，有没有违背我的承诺？”**
3. **“我做这件事，如果明天登上了新闻头条，我会不会感到羞耻？”**
4. **“我做这件事，十年后的我会不会感谢现在的自己？”**

这四个问题，比“白帽 vs 黑帽”的标签有用得多。

11.7 小结：你的技术信仰，由你自己定义

第十一章讲了四件事：

伦理原则	核心含义	你的实践
信息自由	知识不应被锁住， 但隐私和机密必须被尊重	《守夜者之书》免费开源， 但你不公开读者的个人信息
不破坏、不犯罪、 不炫耀	探索的底线：不造成损失， 不越过法律，不自我膨胀	你学漏洞挖掘， 但不做未授权的测试
用技术让世界变得更好	不是大话， 是每一天的每一个小选择	你写书、分享笔记、 帮助其他初学者
开源精神	透明、可维护、可学习	你的书在 GitHub 上开源， 别人可以 pr 纠错

这一章没有给你“标准答案”。

因为黑客伦理本身就没有标准答案。它是一套问题，而不是一套结论：

- “我应该怎么使用我的技术？”
- “我的责任边界在哪里？”
- “我如何在不伤害别人的前提下，实现自己的成长？”

这些问题，你会在未来的每一天、每一个项目、每一次测试中不断重新回答。

而《守夜者之书》能做到的，只是把这些问题摆在你面前，让你知道——**你正在走的这条路，有人走过，有人想过，有人用一生去回答过。**

下一章预告： 黑客的“三大美德”——懒惰、急躁、傲慢。

守夜者手记

1. 在“信息自由”和“隐私保护”之间，你认为边界应该划在哪里？举例说明。
2. 如果你发现了一个未授权系统的漏洞，但厂商没有设置漏洞报告渠道，你会怎么做？
3. 如果你是我，你会选择把《守夜者之书》免费开源吗，是出于哪种黑客伦理的考虑？

写完后： `git add . && git commit -m "第十一章读完：黑客伦理"`。

第十二章：黑客的“三大美德”——懒惰、急躁、傲慢

12.1 美德的反面

在讲黑客的“三大美德”之前，我需要先做一件事：**把这几个词从它们的日常含义里“拯救”出来。**

在日常语言里：

- “懒惰”是缺点——你不想干活，逃避责任。
- “急躁”是缺点——你没耐心，沉不住气。
- “傲慢”是缺点——你自以为是，看不起别人。

但黑客文化里使用的这三个词，和日常语言里的含义完全不同。

它们来自两篇文章：一篇是 Larry Wall（Perl 语言的创造者）在 1998 年写的《Programming Perl》前言，另一篇是 Paul Graham（Y Combinator 创始人）在 2000 年代初写的一系列关于“黑客文化”的随笔。

这两位作者把“懒惰、急躁、傲慢”重新定义为程序员的三大美德——不是“道德”意义上的美德，而是**“生产效率”意义上的美德。*

如果你用这个框架去观察那些最厉害的黑客和安全研究员，你会发现：这三条，他们几乎全中。

而且，**你很可能已经有了其中的某一条，甚至三条全有**——只是你以前觉得这是“缺点”，现在你可以重新认识它了。

12.2 第一美德：懒惰——懒得重复，所以自动化

黑客的“懒惰”，不是“不想干活”，而是“**不想反复干同一件活**”。

普通人的懒惰：这件事太麻烦了，不做了。

黑客的懒惰：这件事太麻烦了，我写个脚本让它自动做。

区别是什么？

- 普通人用“不做”来逃避重复劳动。
- 黑客用“写代码”来消灭重复劳动。

一个普通的运维工程师，每天手动登录 10 台服务器检查日志，花 2 小时。

一个“懒惰”的运维工程师，花 4 小时写一个自动巡检脚本，然后每天 2 小时的重复劳动变成了 0。

第一周，他“亏”了（4 小时 vs 10 小时）。但第二周，他开始“赚”了。一年后，他节省了数百小时的重复劳动。而这些节省出来的时间，可以用来学习新技能、研究新漏洞、或者——写一套像《守夜者之书》这样的东西。

在网络安全领域，“懒惰”尤其重要：

- 你懒得手动扫端口，所以学了 nmap 的自动化参数。
- 你懒得每次都手动拼接 Payload，所以写了自动化漏洞验证脚本。
- 你懒得每次都手动翻日志，所以写了日志分析工具。

懒惰是创造力的引擎。因为没有“懒惰”这个痛点，你就不会去思考“怎么能让这件事自动完成”。

真正的安全专家，不是“能手工完成复杂操作的人”，而是“能让复杂操作自己完成的人”。

Kate 说：“所以我写 Markdown 笔记的时候，搞了一堆快捷键和插件——本质上也是因为懒。我不想每次都手动格式化表格，所以找了 Markdown Table 插件。我不想每次都手敲目录，所以用了自动生成目录的功能。”

“对。你这种‘懒’，让你从‘会写 Markdown 的人’变成了‘写了一套 Markdown 教程的人’。”

“……所以懒惰不是我的缺点，是我的生产力引擎。”

所以，如果你觉得自己有点“懒”——别急着改。学会用这个“懒”去驱动自己写脚本、配自动化、找捷径。

这才是黑客的“懒惰”。

12.3 第二美德：急躁——受不了慢，所以追求极致

黑客的“急躁”，不是“脾气暴躁”，而是“无法忍受低效率”。

- 一个普通用户，打开一个网页需要 3 秒，他等。
- 一个“急躁”的黑客，打开一个网页需要 3 秒，他会想：“为什么这么慢？是 DNS 解析的问题？是服务器响应慢？是中间有丢包？”

然后他开始分析，开始优化，开始寻找瓶颈。

“急躁”驱动下的黑客，做的不是“攻击”，是“对效率的极致追求”。他们想知道：“**这个系统还能更快吗？还能更精简吗？还能更优雅吗？**”

这种“急躁”，驱动了计算机历史上几乎所有的性能优化：

- 第一批操作系统被写出来，是因为有人“受不了每次都要手动加载程序”；
- TCP/IP 协议被发明，是因为有人“受不了不同网络之间不能互通”；
- 汇编语言被发明，是因为有人“受不了每次都写二进制机器码”；
- 高级语言被发明，是因为有人“受不了汇编写太慢”；
- 编译器被发明，是因为有人“受不了手动做内存管理”。

——你看，整个计算机发展史，就是一群“急躁”的人，看不惯现状，于是动手把它改了。

在网络安全领域，“急躁”同样重要：

- 你受不了“人肉翻日志太慢”，所以写了日志分析脚本。
- 你受不了“手工测试每个输入点太慢”，所以写了半自动化的漏洞扫描器。
- 你受不了“每次都要等 30 分钟才能生成一份报告”，所以你优化了报告生成流程。

这种“急躁”会让你成为一个“高效”的工程师，而不是一个“勤快”的工程师。

Kate 说：“我写教程的时候也经常急躁。比如每次都要手动截图、手动调路径、手动检查 Markdown 语法有没有写错——太慢了。所以我现在一边写一边开预览，一边截图一边自动保存，一边用 Lint 插件实时检查。”

“你这不是急躁，你这是‘不想把时间浪费在可以自动化的事情上’。”

“……所以急躁不是沉不住气，是不想等。”

所以，如果你有时候觉得自己“急”——别急着改。学会用这个“急”去驱动自己寻找瓶颈、优化流程、追求更快更稳的结果。

这才是黑客的“急躁”。

12.4 第三美德：傲慢——看不上烂代码，所以写出更好的

黑客的“傲慢”，不是“看不起人”，而是“无法容忍糟糕的设计和低质量的实现”。

- 一个普通程序员看到一段烂代码，说：“能跑就行。”
- 一个“傲慢”的黑客看到一段烂代码，说：“这也能叫代码？我重写一遍。”

然后他真的重写了。而且写得更好。

“傲慢”驱动的黑客，做的最重要的一件事是：用“更好的实现”来替代“能跑的烂实现”。

- 看到别人的脚本写得一团糟，他重写成模块化、可配置、可扩展的版本。
- 看到别人的安全工具只会扫端口，他扩展成能识别服务版本、能匹配漏洞库、能生成报告的一体化工具。
- 看到别人的漏洞报告写得像流水账，他重新整理成“问题 → 复现 → 根因 → 修复”的标准格式。

这种“傲慢”本质上是一种**审美驱动**——不是“我好厉害”的那种傲慢，而是“这件事可以有更好的方式”的那种笃定。

在网络安全领域，这种“傲慢”尤其有价值：

- 你“傲慢”地觉得“厂商的补丁手册写得不够清楚”，所以你写了一篇更清晰的漏洞复现指南。
- 你“傲慢”地觉得“市面上的漏洞扫描器都不够好”，所以你写了一个更轻量、更精准的版本。
- 你“傲慢”地觉得“现有教程不够系统”，所以你写了《守夜者之书》。

你看到了吗？**你正在做的事，本身就是“傲慢”的产物。**

Kate 说：“所以《守夜者之书》的存在，本身就是一个‘傲慢’的行为。我看到市面上教程要么太枯燥、要么太贵、要么不讲人话，所以我自己写一套。”

“对。而且这种傲慢不是‘我比你厉害’，是‘这件事可以做得更好，我来做’。”

“……所以傲慢不是自大，是不将就。”

所以，如果你有时候觉得自己“有点傲”——别急着改。学会用这个“傲”去驱动自己写出更好的代码、更好的文章、更好的工具。

这才是黑客的“傲慢”。

12.5 把“缺点”变成“燃料”

“懒惰、急躁、傲慢”——这三个词在黑客文化里被赋予了完全不同的含义。

你可以把它们理解成一种“转化公式”：

“缺点”	黑客转化	结果
懒惰	不想重复 → 写自动化脚本	消灭重复劳动
急躁	受不了低效 → 寻找瓶颈并优化	提高系统性能
傲慢	看不上烂代码 → 重写出更好的	提升代码质量

如果你发现自己同时具备这三条，恭喜你——你可能天生就是一个“黑客坯子”。

如果你只有其中一条或两条，也没问题。这三条是可以“训练”出来的：

- 你可以训练自己“对重复劳动的敏感度”（这是懒惰的入口）；
- 你可以训练自己“对低效率的容忍度降低”（这是急躁的入口）；
- 你可以训练自己“提高对代码质量的标准”（这是傲慢的入口）。

而这三条共同指向一个目标：**不满足于现状，不停止于“能跑就行”**。这种“永不满足”的驱动力，正是黑客文化最核心的动力来源。

12.6 小结：三大美德是你的“内置引擎”

第十二章讲了三件事：

“美德”	日常含义	黑客含义
懒惰	不想干活	不想重复劳动 → 写脚本自动化
急躁	没耐心	受不了低效率 → 寻找瓶颈并优化
傲慢	自以为是	看不上烂代码 → 写出更好的实现

这三条“美德”，不是让你“变得更懒、更急、更傲”——它们是让你**把这三个原本可能被批评的特质，转化成生产力的引擎**。

在接下来的《这才是网络安全》主系列里，你会在每个技术模块的学习中反复用到这三个引擎：

- 懒得手动配环境 → 学会 Docker、学会自动化部署；
- 急躁于手工测试太慢 → 学会写自动化扫描脚本、批量验证工具；
- 傲慢于市面上的教程都写得太含糊 → 自己写一篇更清晰、更完整的复现笔记。

它们是你技术成长路上的“内置加速器”。学会用它们，你的学习速度会比别人快一倍。

下一章预告： 技术与道德的分界线。

守夜者手记

1. 你觉得自己更偏向“懒惰”、“急躁”还是“傲慢”？举例说明。
2. 这三种“美德”中，你觉得哪一条在网络安全领域最有用？为什么？
3. 在日常生活中，你有没有因为自己的“某条缺点”而意外地做出过“好东西”的经历？

写完后： `git add . && git commit -m "第十二章读完：黑客的三大美德"。`

第十三章：技术与道德的分界线

13.1 当“能做”遇到“该不该做”

在之前的章节里，我们已经看到很多“能做”的例子：

- 乔·恩格西亚能用口哨控制全美国的电话网络——他“能做”。
- 凯文·米特尼克能冒充任何人骗出密码——他“能做”。
- 红客联盟能瘫痪白宫的网站——他们“能做”。

这些例子有一个共同点：在“能做”的那个瞬间，他们面前都站着一条隐形的线。线的这边是“技术上可行”，线的那边是“道德上该如何选择”。

这条线，就是**技术与道德的分界线**。

它不像防火墙规则那样写在配置文件里，也不像法律条款那样印在纸上。它是一条你必须在每一次行动中自己画出来的线。

画得好，你是守夜者。画不好，你可能会滑向另一端。

这一章，我们来讨论这条线应该怎么画。

13.2 白帽、灰帽、黑帽——不是标签，是选择

你已经听过这三个词很多次了。但我们需要把它们从“标签”变成“理解框架”。

白帽黑客： 获得授权 → 测试系统 → 发现漏洞 → 负责任地报告给厂商 → 在漏洞修复后公开细节。

流程很清晰，像一套 SOP。但真实世界没有这么干净：

- “授权”的边界在哪里？客户说“你测试这个 IP 段”，但那个 IP 段里有一台不属于客户范围的服务器——你扫了，算不算越界？
- “负责任地报告”要等多久？厂商拖了半年不修，你公开还是不公开？

黑帽黑客：未经授权 → 侵入系统 → 窃取数据/造成破坏 → 可能出售漏洞或利用漏洞获利。

流程也清晰。但在真实世界里，“黑帽”和“白帽”之间的距离，有时只差一个瞬间的决策：

- 一个白帽在测试时发现了一个“额外”的漏洞，超出了授权范围。他忍不住多看了一眼。这一眼，算不算“黑”了？
- 一个灰帽发现了一个未授权系统存在漏洞，他什么都没动，只是发了一封匿名邮件提醒管理员。这算“白”还是“灰”？

灰帽黑客：未经授权 → 测试系统 → 发现漏洞 → 可能报告、可能沉默、可能公开。

灰帽是三个里最“暧昧”的。他们不做坏事，但也不遵守“先授权再测试”的规矩。他们的理由是：“如果我不测试，这个漏洞可能一直存在，造成的危害更大。”

听起来有理，但问题在于：**“未经授权测试”本身，就是在替别人做决定。**

我举一个真实的例子：

2018年，一位安全研究员发现了一个影响全球数十亿设备的漏洞（后来被称为“幽灵”漏洞，Spectre）。他在发现漏洞之前，并没有获得任何厂商的“授权”去测试这个漏洞——他只是在研究处理器架构时偶然发现的。

他第一时间报告给了相关的处理器厂商，给了他们几个月的缓冲期来修复，然后才公开发布。

他是白帽、灰帽、还是黑帽？

按照严格定义，他是“灰帽”——他没有授权。但按照实际贡献，他做了一件对全世界都有益的事。

所以你看，标签是死的，人是活的。与其纠结“我算哪种帽子”，不如用一套更底层的框架来做判断。

13.3 四把尺子：做一个道德决策

当你面对一个“能做但不确定该不该做”的情况时，试着用这四把尺子量一下：

尺子一：是否获得了明确的授权？

这是最硬的一条线。如果你没有明确的书面或口头授权，默认就不要做任何超出“公开访问”范围的测试。

- 访问一个公开的网页 → 不需要授权。
- 扫描一个公开网站的端口 → 灰色地带（视国家法律而定）。

- 尝试利用一个漏洞登录后台 → 需要授权。

“我以为他允许”不算授权。“以前有人这么做”不算授权。“我只是看看”不算授权。

尺子二：是否造成了任何形式的损害？

包括但不限于：数据丢失、系统宕机、信息泄露、用户受影响、声誉损失。

如果你做了一件事，导致上述任何一项发生——即使你是“无意”的——你也越线了。

在测试之前，问自己：“如果我把这件事搞砸了，最坏的结果是什么？”如果最坏的结果你承担不起，就不要做。

尺子三：你的动机是什么？

- 为了学习？→ 去靶场，不要去真实系统。
- 为了赚钱？→ 走漏洞赏金平台，不要走地下市场。
- 为了炫耀？→ 写一篇技术文章，不要截图发朋友圈说你“拿下了某网站”。
- 为了正义？→ 最危险的一种动机。“正义感”最容易让人突破边界，因为你觉得自己“在做好事”。

Kate 说：“‘为了正义’这条最危险。米特尼克当年大概也觉得自己在‘探索系统边界’，红客联盟也觉得自己在‘为国争光’。正义感是最好的通行证，也是最容易让人翻车的油门。”

“对。‘我在做好事’这句话，是很多越界行为的开头。如果你需要用这句话来说服自己，那说明你已经离红线很近了。”

尺子四：如果明天这件事上了新闻，你会怎么面对？

这是最有用的“灵魂拷问”：

- 如果你的名字出现在头条：“某研究员未经授权测试某系统”——你能坦然面对吗？
- 如果你被厂商起诉了——你能坦然面对吗？
- 如果你被学校/公司开除了——你能坦然面对吗？

如果答案有一丝犹豫，那就说明——你不应该做这件事。

13.4 漏洞披露的“灰色三部曲”

有一个场景，几乎每个安全研究员都会遇到，而且它完美地展示了“道德分界线”有多难画：

你发现了一个漏洞。现在怎么办？

大部分国家的安全社区，默认遵循一套被称为“负责任披露”（Responsible Disclosure）的流程：

1. **私下报告给厂商**：先联系厂商的安全团队，提供完整的漏洞细节和复现步骤。给他们一个合理的修复时间（通常是 45-90 天）。
2. **等待修复**：在厂商修复之前，不公开任何细节。
3. **修复后公开**：等厂商发布了补丁，你再公开技术细节，供社区学习。

这套流程看起来完美，但在真实世界里，它经常“卡壳”：

- **厂商不回邮件**。你发了报告，石沉大海。三个月过去了，没人理你。
- **厂商回复了，但不承认这是漏洞**。“这是功能，不是漏洞。”
- **厂商承认了，但拖着不修**。一年过去了，还没补。
- **厂商修了，但修得不对**。补丁只堵了表面，底层问题还在。

遇到这些情况，研究员会面临一个艰难的选择：

- **选项 A**：继续等。等多久不知道。如果厂商永远不修呢？
- **选项 B**：公开漏洞细节。给厂商施压，但也给了攻击者可乘之机。
- **选项 C**：把漏洞卖给第三方（漏洞经纪商或地下市场）。至少能赚点钱。

Kate 说：“所以‘负责任披露’这套流程，听起来很体面，但实际操作起来完全是另一回事。厂商不回邮件怎么办？厂商说‘这不是漏洞’怎么办？厂商修了一年还没修好怎么办？”

“对。负责任披露的前提是‘厂商也负责任’。如果厂商不负责任，研究员就被卡在一个尴尬的位置上——等也不是，公开也不是，卖掉也不是。”

“……所以这行不光需要技术，还需要耐心、判断力、和一颗扛得住焦虑的心脏。”

选项 A 是“最安全”的，但也是最令人沮丧的——你花了几十个小时研究的成果，被晾在一边。

选项 B 是“最有风险”的——一旦公开，漏洞可能会被恶意利用。

选项 C 是“最赚钱”的，但也是最危险的——一旦你跨过这条线，就很难再回来了。

对于你，一个初学者来说，最安全的路径是：先报告，给合理期限，如果厂商不理你，你可以在到期后公开。但不要拿“愤怒”作为提前公开的理由，更不要拿“赚一笔”作为卖漏洞的理由。

13.5 0day 的诱惑

0day，顾名思义，“零天”漏洞——厂商或公众还没有发现的漏洞，没有补丁、没有防御方案。

一个 0day 的价值有多高？

- 在黑市上，一个普通软件的 0day 可以卖到几万到几十万美元。
- 一个针对主流操作系统或浏览器的 0day，可以卖到数百万美元。
- 如果加上“利用链”（把一个漏洞变成可执行的攻击），价格可以翻倍。

一个 0day 可以让你一夜之间赚到别人几年甚至几十年的工资。你花几十个小时挖到一个 0day，然后有人出价 10 万美元——你会怎么选？

Kate 说：“我不装了。如果有人出价 10 万美元买一个漏洞，我肯定会犹豫。我可能会想：‘这笔钱可以让我爸妈不那么辛苦’或者‘这笔钱可以让我不用打工专心学技术’。”

“这些想法都是真实的。但如果那笔钱来自一个会把漏洞卖给攻击者的人呢？如果那个漏洞后来被用来攻击了某家医院、某个学校、某个你根本不认识的普通人呢？你银行卡里多出来的那笔钱，是那个受害者替你付的。”

“……所以卖 0day 不是赚了一笔钱，是签了一份你不知道内容的合同。”

这个问题，每一个安全研究员在某个时刻都会面对。不需要假装自己“肯定不会心动”。你需要的是**提前想好答案**。

我建议你想一想以下这几件事：

- **你现在手里最值钱的东西，不是 0day，是你的“干净身份”。** 一个干净的背景、一份合法的职业履历、一个可以自由进出公共场所的身份——这些是 0day 的钱买不回来的。
- **0day 的“保质期”很短。** 你卖出去之后，它可能会被用来攻击医院、学校、基础设施。你无法控制它被用在什么地方。
- **挖 0day 的能力，卖一次就贬值一次。** 但如果你用它来建立声誉、公开研究、出版文章——你的能力会一直增值，而不只是被“消费”一次。

你可以把 0day 看成一把刀。这把刀可以切菜，也可以伤人。你决定它用来做什么。

13.6 技术是中立的，但使用技术的人不是

这是本章最重要的一句话：

“技术本身是中立的，但使用技术的人不是。”

- 一个端口扫描器——它可以用来检查你自己的网络，也可以用来扫描别人的网络。
- 一个漏洞利用工具——它可以用来验证漏洞是否存在，也可以用来入侵系统。
- 一个密码破解工具——它可以用来恢复你忘记的密码，也可以用来撞库盗号。
- 一个加密算法——它可以用来保护你的隐私，也可以被勒索软件用来锁死你的文件。

工具本身没有“善恶”。决定“善恶”的，是工具背后的那个人：

- 他有没有授权？
- 他的目的是什么？
- 他考虑了后果吗？

你是那个使用技术的人。你的选择，决定了你的工具是“剑”还是“犁”。

13.7 小结：分界线不是一道墙，是一盏灯

第十三章讲了四件事：

主题	核心观点
白帽/灰帽/黑帽	标签是死的，人是活的。用四把尺子代替标签判断
四把尺子	授权、损害、动机、面对头条——做决策前先量一遍
漏洞披露的灰色三部曲	报告厂商 → 等待 → 决定公开时机，每一步都有道德考量
0day 的诱惑	高回报背后是高风险。保护你的“干净身份”，比任何一笔钱都重要

而贯穿始终的那条线是：

技术上的“能做”和道德上的“该做”之间，永远有一道分界线。这道线不是刻在石头上的，而是你在每一次行动中画出来的。

你画得越频繁，画得就越准。

你不是在“遵守别人的规则”。你是在“建立自己的准则”。而这份准则，将伴随你整个职业生涯。

下一章预告： 守夜者的精神——孤独但有光。

守夜者手记

1. 白帽、灰帽、黑帽的区分，在真实世界里够用吗？你觉得有没有必要增加其他分类？
2. 如果你发现了一个 0day，厂商不回邮件，你会等多久？为什么？
3. “技术是中立的，但使用技术的人不是”——你同意这句话吗？有没有反例？

写完后： `git add . && git commit -m "第十三章读完：技术与道德的分界线"`。

第十四章：守夜者的精神——孤独但有光

14.1 你可能正在经历什么

在进入第一季之前，我想和你聊几句。

不是以“作者”的身份，而是以“走过这条路的人”的身份。

我不知道你现在多大，在哪个城市，用什么设备看这本书。但我知道一件事：**你之所以翻开这本书，很可能是因为在某个时刻，觉得自己"不太合群"。**

也许是这样：

- 你的同学在聊综艺、追剧、打游戏，你在看一篇关于网络协议的文章。
- 你的家人问"你天天对着电脑在学什么"，你发现很难解释清楚什么叫"漏洞挖掘"。
- 你的朋友说"学这个以后能干嘛"，你答不上来，因为你自己也还没完全想清楚。
- 你偶尔会觉得：是不是自己选了一条奇怪的路？为什么别人看起来都那么"正常"，只有我在做这些没人看得懂的事？

如果你有过这些感觉，那我要告诉你一句你可能不信的话：

你不是"不合群"，你只是还没找到你的"群"。

你的群不在教室，不在朋友圈，不在宿舍的夜谈里。

你的群在 MIT 的地下室里、在电话飞客的蓝盒子旁边、在 GNU 和 Linux 的邮件列表里、在绿色兵团的 BBS 上。那些地方的人，当年也经历过你现在的感觉——"为什么只有我在做这件事？"

他们后来找到了彼此。

你也会的。

14.2 那些"不被看见"的人

我想给你介绍一群人。

他们做的事情，在当时基本不被人理解。没有媒体报道，没有老师表扬，没有家长说"你看别人家孩子多厉害"。

但你现在用的每一台设备、每一条网络、每一行代码，都建立在他们当年那些"不被看见"的工作之上。

- **乔·恩格西亚**，一个盲人男孩，在没人理解他在做什么的时候，用口哨发现了电话系统的漏洞。他后来成了整个电话飞客圈子里最受尊敬的人。
- **史蒂夫·沃兹尼亚克**，在车库里焊电路板的时候，没人觉得那台叫 Apple I 的东西能改变什么。他后来设计出了 Apple II。
- **理查德·斯托曼**，在 MIT 的办公室里敲下 GNU 的第一行代码时，大多数人觉得"一个人写一个操作系统"是痴人说梦。他后来改变了整个软件行业的规则。
- **林纳斯·托瓦兹**，一个芬兰大学生，把自己的"业余项目"上传到 FTP 服务器时，没想过它会运行在全世界的服务器上。他后来成了整个互联网"地基"的守护者。
- **绿色兵团的成员们**，在 BBS 上发帖分享技术资料的时候，没人觉得"网络安全在中国能成为一个行业"。他们后来见证了一个产业的诞生。

他们不是一开始就被看见的。

他们是先"守夜"，然后才被人看到的。

如果你觉得自己现在"不被看见"，你只是和他们处在同一条曲线的同一段上。

14.3 关于"中二"这件事

你可能觉得自己"读完一套十几本书的计划"听起来有点中二。

我们来聊聊"中二"。

"中二"不是什么贬义词。它是一个**未被验证的理想主义状态**。

- 图灵写《论数字计算在决断难题中的应用》的时候，世界上没有"计算机科学"这个学科——他看起来也很中二。
- 斯托曼写《GNU 宣言》的时候，商业软件公司已经把源代码锁进了保险柜——他看起来也很中二。
- 林纳斯说"我要写一个操作系统"的时候，操作系统已经是 IBM、DEC、AT&T 这些巨头的战场——他看起来也很中二。

当你的"中二"被你坚持下来了，别人就会叫它"远见"。

当你的"中二"被你实现了，别人就会叫它"传奇"。

你不需要在开始的时候被所有人理解。

你只需要在心里知道：**"我想走到那里去。"**

14.4 "守夜者"是留给你的位置

"守夜者"这个名字，是为每一个像你一样的人准备的。

它不要求你有多高的技术，不要求你挖过多少漏洞，不要求你参加过多少比赛。

它只要求你认同一件事：

"技术是用来守护的，不是用来炫耀的。"

如果你认同这一点，你就是守夜者。

- 你不需要加入某个组织，你只需要对自己的选择诚实。

- 你不需要公开宣称"我是守夜者"，你只需要在你写的代码里、你学的技能里、你帮助过的人里，留下"守护"的痕迹。
- 你不需要等到"变得很强"才配得上这个名字。你从翻开这本书的第一页起，就已经在往那个方向走了。

"守夜者"不是一个头衔。它是一种你看待技术的方式。

14.5 守夜者的宣誓词

如果你愿意，在进入第一季之前，可以为自己做一件事。

找一个安静的、不被打扰的时刻。不用念出声，在心里默读这段话。或者把它抄在你笔记本的第一页。或者只是看一遍，然后合上眼睛想几秒钟。

它不是别人强加给你的条条框框。它是**你可以选择认同的一段话**——如果你觉得它说出了你想做的事。

长夜将至，我从今开始守望，至死方休。

我将不设名利、不占资源、不留痕迹。

我将不执权柄，不逐荣光。

我将生死于城上，进退于暗网。

我是黑暗中的指令。

我是城墙上的守望者。

**我是抵御入侵的防火墙，
是唤醒沉睡者的警报，
是守卫数据疆域的盾。**

我将生命与荣耀，献给守夜者。

今夜如此，夜夜皆然。

这段话是写给谁的？

是写给你自己的。

不是为了向别人证明什么，是为了在你觉得"这条路好难走"的时候，有一个可以回来看一眼的东西。

如果你觉得它说中了你的想法，你可以把它作为你技术生涯的一份"小小的自我约定"。

14.6 第零季的终点，是你真正旅程的起点

这本书不教你任何一行命令、任何一个漏洞、任何一次渗透。

它只教你一件事：

"你正在做的事，不是孤立的、不是偶然的、不是不重要的。"

它是一条从 1950 年代 MIT 的地下室，穿过电话飞客的口哨、GNU 的宣言、Linux 的源代码、绿色兵团的 BBS，一直延伸到你面前的屏幕上的线。

你现在是这条线上最新的一环。

从下一季开始，你会学 Linux、学网络、学漏洞、学渗透——那些都是"术"。

这一季给你的，是"道"。

有了"道"，"术"才不会让你走偏。

14.7 写在最后的话

我还想跟你说几句掏心窝子的话。

网络安全这条路，有时候真的挺孤独的。你可能会熬夜调一个环境，调了三个小时发现是少打了个斜杠；你可能会看一篇漏洞分析文章，看了一整天只懂了三分之一；你可能会在某个深夜突然问自己：我到底在干嘛？

这些时刻都会来的。每个走过这条路的人都经历过。

但我想告诉你的是：**你不是一个人。**

你现在读的这本书，是用 Markdown 写的，用 Git 管理的，放在 GitHub 上免费开源的。它本身就是一个"守夜者"留下的脚印。

你读完这一季，你就踩在了这个脚印上。

等你开始写第一季的笔记、挖第一个漏洞、写第一份报告的时候，你就在这条路上留下了你自己的脚印。

而后来的人，会踩在你的脚印上，继续往前走。

这就是"守夜者"的意义：

不是因为有多强，而是因为你在往前走，并且没有停下来。

第零季·终

守夜者手记 · 终章

在你进入第一季之前，翻回第零季的开头，看一看你写下的每一篇手记。

然后回答一个最简单、也最重要的问题：

"读完这一季，我觉得'网络安全'对我来说，意味着什么？"

把你的答案写下来。然后：

```
git add . && git commit -m "第零季读完。我是守夜者。"
```

第一季见。

附录：守夜者的宣誓词（中文版 & 英文版）

长夜将至，我从今开始守望，至死方休。

我将不设名利、不占资源、不留痕迹。

我将不执权柄，不逐荣光。

我将生死于城上，进退于暗网。

我是黑暗中的指令。

我是城墙上的守望者。

我是抵御入侵的防火墙，

是唤醒沉睡者的警报，

是守卫数据疆域的盾。

我将生命与荣耀，献给守夜者。

今夜如此，夜夜皆然。

Night falls, and now my watch begins.

It shall not end until my system halts.

I shall claim no credit, hold no data, father no exploits.

I shall wear no crown of fame, and win no applause.

I shall live and die at my terminal.

I am the script in the darkness.

I am the watcher on the firewall.

I am the code that burns against the breach,

the log that brings the dawn,

**the alert that wakes the sleepers,
the shell that guards the realms of data.**

**I pledge my life and honor to the Keepers of the Night,
for this night and all the nights to come.**